



# GeoLibre 튜토리얼

처음부터 끝까지 따라 하는 실습 워크플로

# 목차

---

|                                                            |    |
|------------------------------------------------------------|----|
| <b>개요</b>                                                  | 5  |
| 시작하기 전에                                                    | 5  |
| 튜토리얼 목록                                                    | 5  |
| 동영상 튜토리얼                                                   | 5  |
| <b>동영상 튜토리얼</b>                                            | 7  |
| GeoLibre 1.0: 어디서나 실행되는 무료 오픈소스 클라우드 네이티브 GIS              | 7  |
| 브라우저에서 지오프로세싱: 설치 없이 쓰는 GeoLibre의 700개 이상 무료 GIS 도구        | 8  |
| 브라우저에서 무료 고해상도 재난 위성 영상에 접근하기                              | 9  |
| GeoLibre로 브라우저에서 건물 외곽선 정규화하기                              | 10 |
| GeoLibre + GeoLens: 지리공간 데이터 자체 호스팅을 위한 현대적인 GIS 스택        | 12 |
| GeoLibre Model Builder와 AI Assistant로 재사용 가능한 GIS 워크플로 만들기 | 14 |
| 무료 고해상도 위성 영상으로 2026년 네팔 홍수 매핑하기                           | 15 |
| GeoLibre로 클라우드 네이티브 GIS 워크플로 구축하기                          | 16 |
| 브라우저에서 GeoLibre로 이미지 지오레퍼런싱하기                              | 17 |
| 공개 데이터로 만든 100개의 인터랙티브 지도: GeoLibre로 탐색하고, 포크하고, 직접 만들기    | 18 |
| 더 많은 영상                                                    | 19 |
| <b>첫 번째 지도 만들기</b>                                         | 20 |
| 1. GeoLibre 열기                                             | 20 |
| 2. 레이어 추가                                                  | 20 |
| 3. 레이어 스타일 지정                                              | 21 |
| 4. 데이터 살펴보기                                                | 21 |
| 5. 저장 또는 공유                                                | 22 |
| 다음 단계                                                      | 22 |
| <b>클라우드 네이티브 데이터</b>                                       | 23 |
| 원격 GeoParquet 로드                                           | 23 |
| FlatGeobuf 로드                                              | 23 |
| COG 로드                                                     | 23 |
| 로컬 데이터를 클라우드 네이티브로 변환                                      | 24 |
| 다음 단계                                                      | 24 |
| <b>벡터 분석</b>                                               | 25 |
| 1. 입력 데이터 로드                                               | 25 |
| 2. 레이어 버퍼링                                                 | 26 |
| 3. 두 레이어 중첩                                                | 26 |
| 4. 검토 및 다듬기                                                | 27 |
| 5. 결과 내보내기                                                 | 27 |

|                                          |    |
|------------------------------------------|----|
| 다음 단계 .....                              | 27 |
| <b>지형 분석</b> .....                       | 28 |
| 1. DEM 로드 .....                          | 28 |
| 2. 음영기복 .....                            | 28 |
| 3. 경사와 경사향 .....                         | 28 |
| 4. 등고선 .....                             | 29 |
| 5. 관심 영역으로 클립 .....                      | 29 |
| 다음 단계 .....                              | 29 |
| <b>공간 SQL</b> .....                      | 30 |
| 1. 로드된 레이어 쿼리 .....                      | 30 |
| 2. 원격 파일 직접 쿼리 .....                     | 31 |
| 3. 공간 함수 사용 .....                        | 31 |
| 4. 결과를 지도에 추가 .....                      | 31 |
| 5. 내보내기 .....                            | 31 |
| 다음 단계 .....                              | 32 |
| <b>공유 및 임베딩</b> .....                    | 33 |
| 1. 계정 연결 .....                           | 33 |
| 2. 프로젝트 공유 .....                         | 33 |
| 3. 공유된 지도 열기 .....                       | 33 |
| 4. 페이지에 임베드 .....                        | 34 |
| 5. 페이지에서 지도 제어 .....                     | 34 |
| 다음 단계 .....                              | 35 |
| <b>형식 참조</b> .....                       | 36 |
| 머리말 .....                                | 36 |
| 1. 런타임 구성 및 플랫폼 개요 .....                 | 36 |
| 2. 벡터 데이터 포맷 지원 .....                    | 37 |
| 3. 래스터 및 영상 .....                        | 39 |
| 4. 포인트 클라우드 및 LiDAR .....                | 40 |
| 5. 3D 모델, 타일 및 가우시안 스피켓 .....            | 40 |
| 6. 타일 및 OGC 서비스 .....                    | 42 |
| 7. 클라우드 네이티브 포맷 및 데이터 카탈로그 .....         | 43 |
| 8. 프로젝트 및 스타일 상호 운용성 .....               | 46 |
| 9. 내보내기 및 포맷 변환: 사실상 무료 GDAL 프런트엔드 ..... | 46 |
| 10. 플랫폼 차이: 주의할 점 .....                  | 47 |
| 11. 아키텍처: 어떻게 이렇게 많은 포맷을 지원할 수 있는가 ..... | 48 |
| 12. 성능: 몇 가지 실제 수치 .....                 | 50 |
| 13. 그 경계 .....                           | 51 |
| 맺으며 .....                                | 52 |
| <b>소스 코드 심층 분석</b> .....                 | 53 |

|                                                             |    |
|-------------------------------------------------------------|----|
| 머리말 .....                                                   | 53 |
| 1. 도구 상자: 하나씩 살펴볼 가치가 있는 오픈 소스 라이브러리 .....                  | 53 |
| 2. DuckDB-WASM Spatial: 포맷 드라이버와 연산 엔진을 겸하는 하나의 라이브러리 ..... | 56 |
| 3. 웹 측 성능: 다섯 가지 최적화, 그리고 솔직한 부정적 결과 하나 .....               | 63 |
| 4. 상태 관리: 바로 재사용할 수 있는 세 가지 패턴 .....                        | 68 |
| 5. 오프라인 기능: Workbox 3단계 캐싱 전략 .....                         | 69 |
| 6. 클라우드 네이티브 포맷: 대체되는 것은 프론트엔드 라이브러리가 아니라 배포 아키텍처 전체 .....  | 71 |
| 맺으며: 투자 규모 순으로 정리한 도입 체크리스트 .....                           | 76 |

<https://geolibre.app/tutorials/>

# 튜토리얼

이 튜토리얼들은 자주 쓰이는 GeoLibre 워크플로를 처음부터 끝까지 안내합니다. 각 튜토리얼은 짧고, **사용자 가이드**를 바탕으로 하며, 사용하는 기능의 참조 페이지로 다시 연결됩니다.

## 시작하기 전에

- 대부분의 튜토리얼은 GeoLibre의 브라우저 빌드인 [web.geolibre.app](https://web.geolibre.app)의 **라이브 뷰어**에서 따라 할 수 있습니다. 설치가 필요하지 않습니다.
- 몇몇 단계에는 **데스크톱 앱**이 필요합니다. 로컬 MBTiles와 래스터 읽기, 그리고 Python 사이드카 도구 (Contour처럼 브라우저 엔진이 없는 래스터 도구)가 그렇습니다. 이런 단계는 해당 위치에서 따로 표시해 두었습니다. 설치하려면 **다운로드**를 참조하세요. Whitebox 지오프로세싱 툴박스는 여기에 포함되지 않습니다. 1,000개가 넘는 그 도구들은 WebAssembly로 브라우저에서 실행됩니다.
- 여러 튜토리얼에서 사용하는 샘플 데이터셋은 세계 국가의 공개 GeoParquet 파일입니다: `https://data.source.coop/giswqs/opengeos/countries.parquet`.

## 튜토리얼 목록

| 튜토리얼                          | 배우는 내용                                              |
|-------------------------------|-----------------------------------------------------|
| <a href="#">첫 번째 지도</a>       | 레이어를 추가하고, 스타일을 지정하고, 속성을 살펴보고, 저장합니다.              |
| <a href="#">클라우드 네이티브 데이터</a> | 원격 GeoParquet, FlatGeobuf, COG를 로드하고 로컬 데이터를 변환합니다. |
| <a href="#">벡터 분석</a>         | 레이어를 버퍼링하고 중첩한 뒤 결과를 내보냅니다.                         |
| <a href="#">지형 분석</a>         | DEM에서 음영기복, 경사, 등고선을 도출합니다.                         |
| <a href="#">공간 SQL</a>        | DuckDB Spatial로 데이터를 쿼리하고 결과를 지도에 표시합니다.            |
| <a href="#">공유 및 임베딩</a>      | 프로젝트를 공유하고 웹 페이지에 임베드합니다.                           |

안내된 투어처럼 순서대로 진행하거나, 지금 하려는 작업에 맞는 튜토리얼로 바로 이동하세요.

## 동영상 튜토리얼

- [GeoLibre 1.0: A Free, Open-Source Cloud-Native GIS That Runs Anywhere \(Browser, Desktop & Jupyter\)](#)
- [Geoprocessing in the Browser: 700+ Free GIS Tools in GeoLibre, Zero Install](#)
- [Access Free High-Resolution Disaster Satellite Imagery in Your Browser](#)
- [Regularize Building Footprints in the Browser with GeoLibre](#)
- [GeoLibre + GeoLens: A Modern GIS Stack for Self-Hosting Geospatial Data](#)

- [Create Reusable GIS Workflows with GeoLibre Model Builder and AI Assistant](#)
- [Mapping the 2026 Nepal Floods with Free High-Resolution Satellite Imagery](#)
- [Building Cloud-Native GIS Workflows with GeoLibre](#)
- [Image Georeferencing Using GeoLibre in the Browser](#)
- [100 Interactive Maps from Open Data: Explore, Fork, and Build Your Own with GeoLibre](#)

챕터와 요약이 포함된 전체 목록은 [동영상 튜토리얼](#)에 있습니다.

<https://geolibre.app/tutorials/videos/>

## 동영상 튜토리얼

GeoLibre를 녹화한 안내 영상으로, 오래된 순서대로 정리되어 있습니다. 전체적인 안내를 원하시면 맨 위부터 시작하고, 특정 작업에 맞는 영상으로 바로 이동하셔도 됩니다. 모든 영상은 Open Geospatial Solutions 채널의 [GeoLibre 재생목록](#)에 포함되어 있습니다. 새 영상을 놓치지 않으려면 채널을 구독하세요.

같은 워크플로를 글로 단계별로 설명한 문서는 [튜토리얼 개요](#)를, 정지 이미지로 앱을 둘러보려면 [데모](#)를 참고하세요.

[전체 재생목록 보기](#)

## GeoLibre 1.0: 어디서나 실행되는 무료 오픈소스 클라우드 네이티브 GIS



▶ [GeoLibre 1.0: 어디서나 실행되는 무료 오픈소스 클라우드 네이티브 GIS \(브라우저, 데스크톱, Jupyter\)](#)

40:18 · 2026년 6월 · [YouTube에서 보기](#)

전체 소개 영상이며, 가장 먼저 보기에 좋은 영상입니다. 브라우저, 데스크톱, Jupyter 세 가지 빌드를 모두 설치하고 실행한 다음, 프로젝트, 벡터 및 래스터 데이터, 속성과 지오메트리 편집, 180 GB 데이터셋을 다운로드 없이 스트리밍하기, PMTiles, 3D 타일, LiDAR, DuckDB, 지도 컨트롤, 공유 기능을 차례로 다룹니다.

관련 문서: [시작하기](#) · [인터페이스 개요](#) · [첫 번째 지도](#)

## 다루는 주제

영상 설명에 적힌 챕터 타임스탬프가 녹화 길이인 40:18을 넘어가므로, 여기서는 시간 없이 주제만 나열합니다.

- GeoLibre란 무엇이며, 실행하는 세 가지 방법(브라우저, 데스크톱, Jupyter)
- 반응형 웹 레이아웃과 라이트/다크 모드
- 데스크톱 앱 설치, Jupyter 노트북에서 GeoLibre 실행
- 프로젝트 저장 및 불러오기, 웹사이트 둘러보기와 기술 스택
- 3D 타일과 지도 캔버스 탐색, Layers 패널과 Style 패널
- 프로젝트 만들기과 베이스맵 선택
- 벡터 데이터 드래그 앤 드롭과 식별 도구
- 속성 테이블: 선택, 편집, 내보내기, 지오메트리 편집
- 래스터 데이터(DEM) 드래그 앤 드롭, GeoParquet 레이어 추가
- 180 GB 벡터 데이터셋을 다운로드 없이 스트리밍
- 벡터 스타일링: 줌 레벨, 라벨, 3D 돌출
- 래스터 데이터: COG, 로컬 파일, 컬러맵, 픽셀 검사
- CSV/위경도 포인트, GPS(GPX) 트랙, XYZ/WFS/WMS 서비스
- 2D와 3D의 PMTiles(Overture 건물), LiDAR 포인트 클라우드와 3D 타일
- DuckDB에 연결하기
- 지도 컨트롤: 축척, 지형, 검색, 컬러바, 범례, 측정, 북마크, 미니맵
- 플러그인, 프로젝트 공유, 토큰, 지도 전용 임베드 링크

## 브라우저에서 지오프로세싱: 설치 없이 쓰는 GeoLibre의 700개 이상 무료 GIS 도구



▶ 브라우저에서 지오프로세싱: 설치 없이 쓰는 GeoLibre의 700개 이상 무료 GIS 도구

## 21:52 · 2026년 6월 · YouTube에서 보기

WebAssembly 위에서 완전히 동작하는 Whitebox Next Gen 툴박스를 소개합니다. 아무것도 설치하지 않고 지형, 수문, LiDAR, 원격탐사, 래스터, 벡터 도구를 사용합니다. 카테고리별로 탐색할 수 있는 Processing 메뉴를 살펴보고, DEM에 지형 및 수문 도구를 실행하며, 플러그인, 대용량 로컬 파일, JupyterLite 패널도 다룹니다.

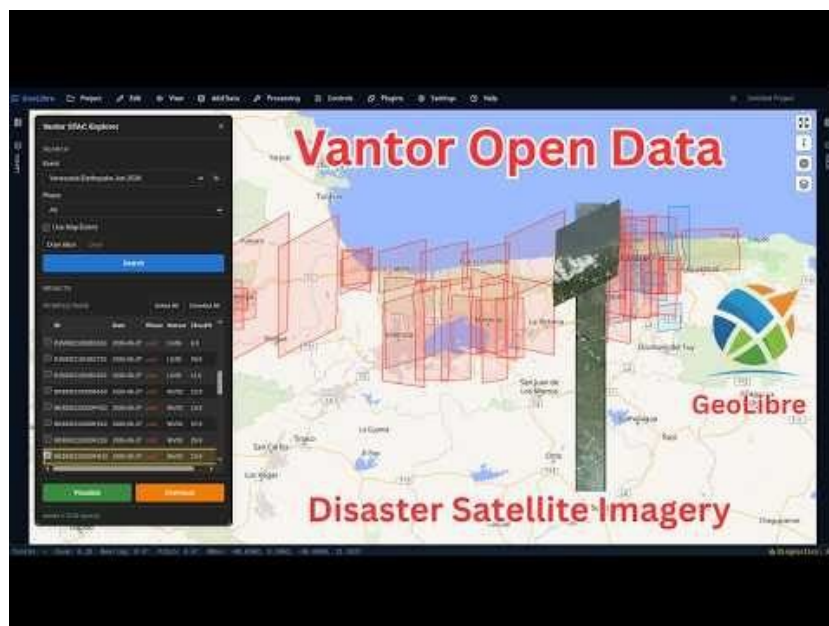
툴박스는 이후 1,000개가 넘는 도구로 늘어났습니다. 현재 목록은 [프로세싱 도구](#)를 참고하세요.

관련 문서: [지형 분석](#) · [벡터 분석](#) · [노트북 패널](#)

### 챕터

- 00:00 소개 및 GeoLibre 1.7 개요
- 03:10 웹 앱 열기와 프로젝트 만들기
- 04:03 데이터 추가와 래스터 레이어 스타일링
- 06:26 Whitebox 도구를 이용한 웹 기반 지오프로세싱
- 09:18 지형 및 수문 분석 실행
- 12:03 벡터 데이터, 라벨, 중심점 처리
- 14:38 플러그인과 GeoLibre 확장
- 17:39 대용량 로컬 파일 불러오기
- 20:25 JupyterLite와 현장 데이터 수집
- 21:31 마무리와 피드백

## 브라우저에서 무료 고해상도 재난 위성 영상에 접근하기



▶ 브라우저에서 무료 고해상도 재난 위성 영상에 접근하기

## 8:26 · 2026년 7월 · YouTube에서 보기

Vantor Open Data 플러그인을 설치하고, 해당 STAC 카탈로그에서 재난 이벤트를 검색한 뒤, 이벤트 전후의 고해상도 영상을 지도에서 비교합니다. 이어서 원본 Cloud Optimized GeoTIFF를 다운로드하고 결과를 링크로 공유합니다.

관련 문서: [플러그인 및 마켓플레이스](#) · [데이터 추가](#)

### 챕터

- 00:00 소개
- 00:53 GeoLibre와 Vantor Open Data란?
- 01:27 Vantor Open Data 플러그인 설치
- 02:30 재난 이벤트 탐색
- 03:00 이벤트 전후 영상 검색
- 03:55 고해상도 위성 영상 시각화
- 04:21 전후 영상 비교
- 05:06 지진 피해 식별
- 05:54 원본 영상 다운로드
- 06:18 여러 영상 장면 불러오기
- 07:05 소스 URL 접근과 지도 공유
- 07:53 STAC 카탈로그 자동 업데이트
- 08:20 마무리

## GeoLibre로 브라우저에서 건물 외곽선 정규화하기



▶ [GeoLibre로 브라우저에서 건물 외곽선 정규화하기](#)

**16:25 · 2026년 7월 · YouTube에서 보기**

AI 세그멘테이션 모델이나 수작업 디지털라이징으로 만들어진 불규칙한 건물 폴리곤을 Rust 기반 정규화 도구로 브라우저 안에서 정리합니다. 데이터 불러오기, 투영 좌표계, 매개변수 조정, 전후 비교, 결과 내보내기를 다룹니다.

[라이브 샘플 프로젝트](#)에서 직접 따라 해 보세요.

관련 문서: [프로세싱 도구](#) · [AI 세그멘테이션](#)

**챕터**

- 00:00 소개
- 00:30 건물 외곽선 정규화가 중요한 이유
- 01:00 기존의 Whitebox 워크플로
- 02:20 GeoLibre 개요
- 03:05 브라우저에서 GeoLibre 실행
- 04:05 샘플 건물 데이터 불러오기
- 05:00 자신의 건물 외곽선 데이터 가져오기
- 06:10 좌표계와 데이터 준비
- 06:45 건물 정규화 도구 열기
- 07:20 도구 매개변수 설정
- 08:05 도구 실행과 결과 비교
- 09:05 정규화 옵션 이해하기
- 10:10 다양한 건물 형태 다루기
- 11:10 추가 Whitebox 도구 살펴보기
- 12:20 예제: 도시 지역 추출
- 13:20 GeoLibre의 Whitebox 통합
- 14:30 GeoLibre의 다른 지오프로세싱 도구
- 15:30 GeoLibre 데스크톱 및 모바일 앱
- 16:10 GeoLibre 다운로드
- 16:17 결론

## GeoLibre + GeoLens: 지리공간 데이터 자체 호스팅을 위한 현대적인 GIS 스택



▶ GeoLibre + GeoLens: 지리공간 데이터 자체 호스팅을 위한 현대적인 GIS 스택

25:41 · 2026년 8월 · [YouTube에서 보기](#)

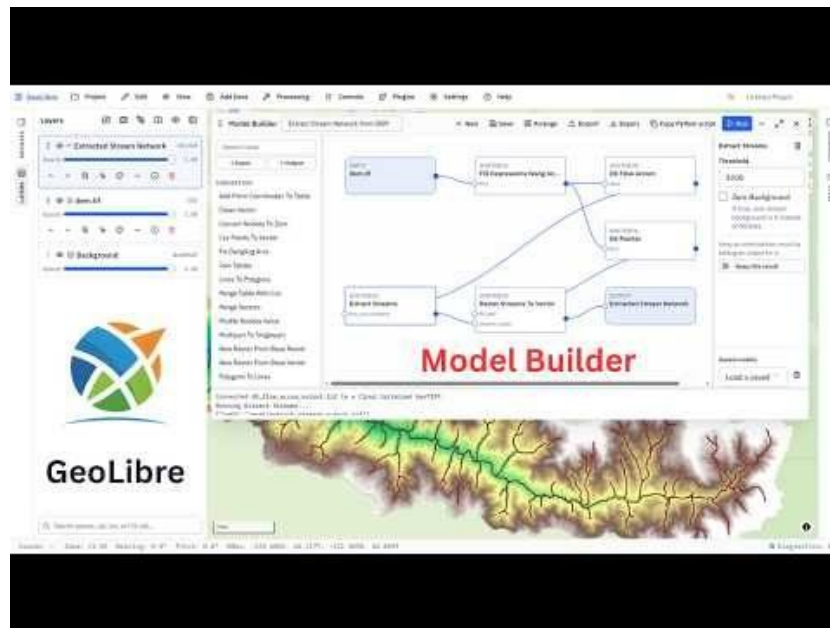
GeoLibre와 [GeoLens](#)를 결합해 완전히 자체 호스팅되는 스택을 구성합니다. GeoLens를 배포하고, GeoLibre에서 그 카탈로그를 탐색하며, 돌출과 표현식 기반 스타일링으로 3D 뉴욕 지도를 만들고, 시간에 따른 건물 증가를 애니메이션으로 보여준 다음, 건물 외곽선을 편집해 그 변경 사항을 GeoLens로 다시 반영합니다.

관련 문서: [데이터 통합](#) · [레이어 스타일링](#) · [자체 호스팅](#)

## 챕터

- 00:00 소개: 현대적인 오픈소스 GIS 스택
- 01:15 GeoLens 소개
- 02:19 GeoLens 데모와 데이터 카탈로그 살펴보기
- 03:51 GeoLibre 소개
- 04:52 GeoLens로 데이터 자체 호스팅 및 관리
- 06:31 GeoLibre를 GeoLens에 연결
- 08:16 NYC 3D 샘플 프로젝트 살펴보기
- 08:58 시간에 따른 NYC 건물 증가 애니메이션
- 10:21 3D NYC 지도를 처음부터 만들기
- 11:34 3D 건물 돌출 만들기
- 12:27 표현식으로 건물 스타일링
- 14:41 NYC 지하철 데이터 추가 및 스타일링
- 16:50 시간 애니메이션 만들기
- 17:26 GeoLibre 프로젝트 공유
- 17:52 GeoLibre에서 GeoLens 데이터 편집
- 19:12 편집 가능한 GeoJSON 데이터 불러오기
- 20:02 건물 외곽선 생성 및 편집
- 21:30 변경 사항을 GeoLens에 다시 저장
- 22:27 지도 범위 기준으로 대용량 벡터 데이터셋 편집
- 23:03 래스터 및 Sentinel-2 데이터 시각화
- 24:23 현대적인 GIS 스택으로서의 GeoLens + GeoLibre
- 24:29 GeoLibre의 다른 데이터 커넥터와 분석 도구
- 25:14 결론

## GeoLibre Model Builder와 AI Assistant로 재사용 가능한 GIS 워크플로 만들기



▶ GeoLibre Model Builder와 AI Assistant로 재사용 가능한 GIS 워크플로 만들기

18:55 · 2026년 8월 · [YouTube에서 보기](#)

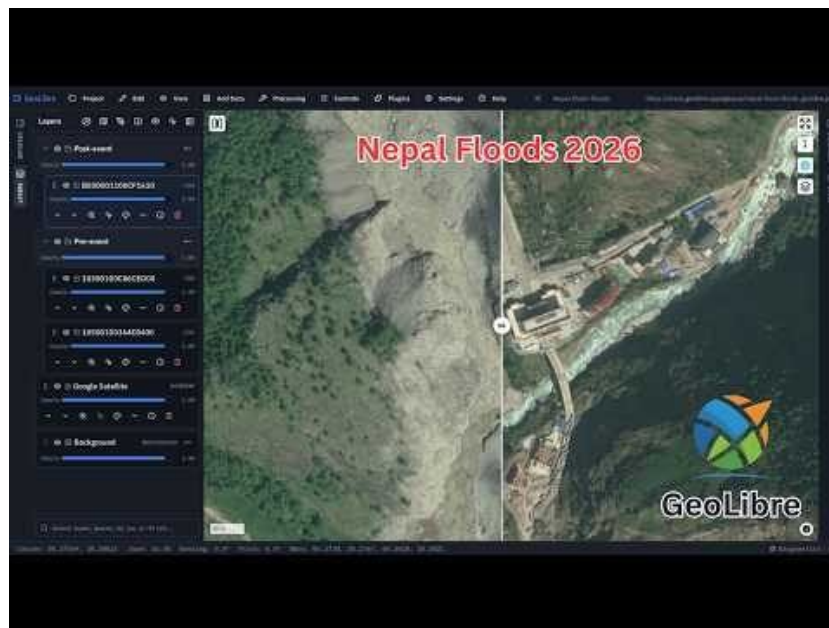
브라우저에서 그래픽 기반의 재사용 가능한 지오프로세싱 워크플로를 만듭니다. 먼저 간단한 벡터 체인을, 이어서 DEM에서 하천망을 추출하는 완전한 수문 모델을 구성합니다. 후반부에서는 같은 작업을 **AI Assistant**에게 맡겨 자연어 프롬프트로 모델을 조립하고 연결하는 과정과, 생성된 결과를 편집하는 방법을 보여줍니다.

관련 문서: [프로세싱 도구](#) · [지형 분석](#)

## 챕터

- 00:00 소개
- 00:49 Model Builder 개요
- 02:23 간단한 워크플로 만들기
- 04:19 모델 실행
- 04:54 모델 가져오기와 내보내기
- 05:23 수문 워크플로 만들기
- 11:08 수문 모델 실행
- 13:01 AI로 모델 만들기
- 14:46 프롬프트로 워크플로 생성
- 17:32 AI가 생성한 모델 사용자화
- 18:25 결론

## 무료 고해상도 위성 영상으로 2026년 네팔 홍수 매핑하기



▶ 무료 고해상도 위성 영상으로 2026년 네팔 홍수 매핑하기

### 23:14 · 2026년 8월 · YouTube에서 보기

네팔-중국 국경 지역 홍수를 대상으로 한 완전한 재난 대응 워크플로입니다. Vantor Open Data, Planet Crisis Response, OpenAerialMap에서 공개 영상을 찾고, 보고, 비교하고, 다운로드합니다. 전후 비교 스와이프 지도를 만들고, 3D 지형으로 계곡을 살펴보면, Overture Maps에서 건물 외곽선을 가져온 뒤 피해 평가를 위해 결과를 내보냅니다.

[라이브 샘플 프로젝트](#)에서 직접 따라 해 보세요.

관련 문서: [플러그인 및 마켓플레이스](#) · [데이터 통합](#) · [지도 컨트롤](#)

## 챕터

1.0 소개 영상과 마찬가지로, 영상 설명의 마지막 몇 개 챕터 타임스탬프는 녹화 길이인 23:14를 넘어갑니다.

- 00:00 우주에서 본 네팔-중국 국경 홍수
- 02:02 전후 비교 지도 만들기
- 03:05 피해 지역 찾기
- 06:09 Vantor Open Data 검색
- 08:40 3D로 지형 살펴보기
- 12:08 이벤트 이후 위성 영상 추가
- 14:53 Layer Swipe로 피해 비교
- 18:39 인터랙티브 GeoLibre 지도 공유
- 20:06 Planet Crisis Response 영상 살펴보기
- 22:52 OpenAerialMap으로 영상 찾기
- 25:14 Overture Maps에서 건물 추출
- 27:20 피해 평가용 데이터 내보내기
- 28:30 결론

## GeoLibre로 클라우드 네이티브 GIS 워크플로 구축하기



▶ GeoLibre로 클라우드 네이티브 GIS 워크플로 구축하기

1:07:28 · 2026년 9월 · [YouTube에서 보기](#)

녹화된 웨비나로, 프로젝트를 단일 영상으로 가장 길게 소개한 자료입니다. GeoLibre가 COG, GeoParquet, PMTiles, DuckDB, WebAssembly를 활용해 대용량 벡터, 래스터, LiDAR 데이터셋을 브라우저에서 직접 다루는 방식을 설명한 뒤, 플러그인 생태계, 크로스 플랫폼 빌드(브라우저, 데스크톱, 모바일, Jupyter, R), 새롭게 등장하는 GeoAI 통합을 차례로 살펴봅니다.

슬라이드: [geolibre-slides-en.html](https://geolibre-slides-en.html)

관련 문서: [클라우드 네이티브 데이터](#) · [아키텍처](#) · [Python API](#) · [R 패키지](#)

## 브라우저에서 GeoLibre로 이미지 지오레퍼런싱하기



▶ 브라우저에서 GeoLibre로 이미지 지오레퍼런싱하기

15:35 · 2026년 9월 · [YouTube에서 보기](#)

데스크톱 GIS 없이 스캔한 테네시 대학교 캠퍼스 지도를 지오레퍼런싱합니다. 지상기준점을 배치하고, 변환 방식을 선택하고, GCP별 잔차와 RMS 잔차를 확인하고, 이미지를 베이스맵에 정렬합니다. 이어서 GeoTIFF로 내보내고 다시 불러온 뒤, NoData 투명도를 처리하고, 스와이프 도구로 베이스맵과 비교한 다음 완성된 지도를 공유합니다.

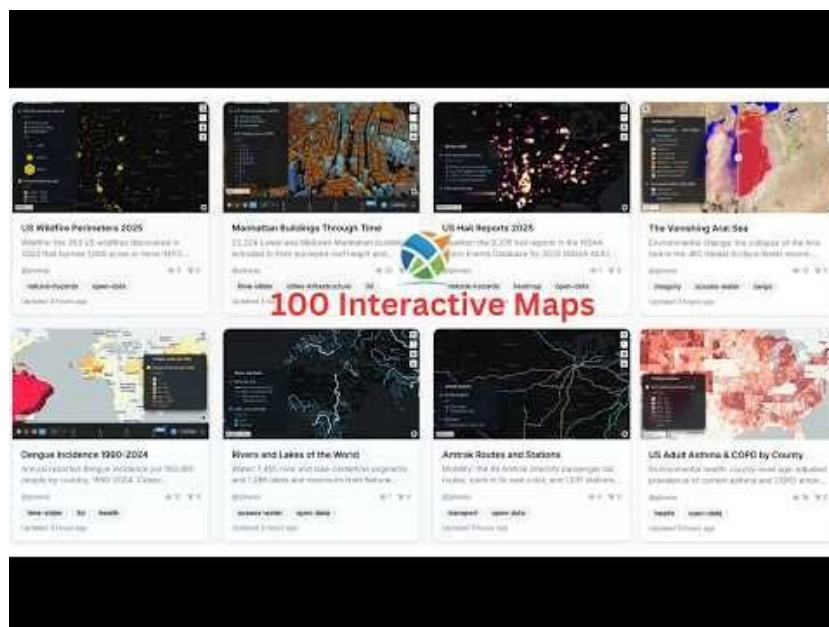
샘플 데이터: [UTK 캠퍼스 지도](#) · [기준점](#)

관련 문서: [프로세싱 도구](#) · [데이터 추가](#) · [공유 및 임베드](#)

## 챕터

- 00:00 이미지 지오레퍼런싱 소개
- 01:44 GeoLibre 실행과 베이스맵 선택
- 02:44 Georeferencing 도구 열기
- 03:20 이미지 불러오기
- 03:50 변환 방식 선택
- 04:55 지상기준점 선택
- 08:34 좌표와 잔차 확인
- 11:17 지오레퍼런싱 정확도 평가
- 11:30 기준점과 GeoTIFF 내보내기
- 12:26 지오레퍼런싱된 이미지를 지도에 추가
- 12:59 스와이프 도구로 레이어 비교
- 13:35 내보낸 GeoTIFF 불러오기
- 14:07 NoData와 투명도 처리
- 14:42 지오레퍼런싱된 지도 공유
- 15:29 결론

## 공개 데이터로 만든 100개의 인터랙티브 지도: GeoLibre로 탐색하고, 포크하고, 직접 만들기



▶ 공개 데이터로 만든 100개의 인터랙티브 지도: GeoLibre로 탐색하고, 포크하고, 직접 만들기

24:37 · 2026년 9월 · YouTube에서 보기

공개 데이터 갤러리를 둘러봅니다. 무료 공공 데이터만으로 만든 100개의 인터랙티브 지도가 브라우저에서 바로 열립니다. 각 지도를 탐색하고 포크하는 방법과 어떻게 만들어졌는지를 보여주므로, 같은 종류의 지도를 자신의 데이터로 만들 수 있습니다. 지도는 보건과 자연재해부터 역사와 문화까지 12개 주제에 걸쳐 있으며, NASA, NOAA, USGS, CDC, 세계은행, Our World in Data, Eurostat, Wikidata 등의 데이터를 사용합니다.

관련 문서: [갤러리](#) · [공식 데모 컬렉션](#) · [Python 패키지](#)

## 더 많은 영상

---

이 채널은 GeoAI, DuckDB, 지리공간 Python도 폭넓게 다룹니다.

- [YouTube의 Open Geospatial Solutions](#)
- [GeoAI 재생목록](#)

다루어 주었으면 하는 워크플로가 있으신가요? [토론](#)을 열어 제안해 주세요.

<https://geolibre.app/tutorials/first-map/>

# 첫 번째 지도

이 튜토리얼은 빈 워크스페이스에서 시작하여 몇 분 만에 속성을 살펴볼 수 있는 데이터가 담긴 스타일 지정된 지도를 만드는 과정을 안내합니다. 전 과정을 [라이브 뷰어](#)에서 진행할 수 있습니다.

## 1. GeoLibre 열기

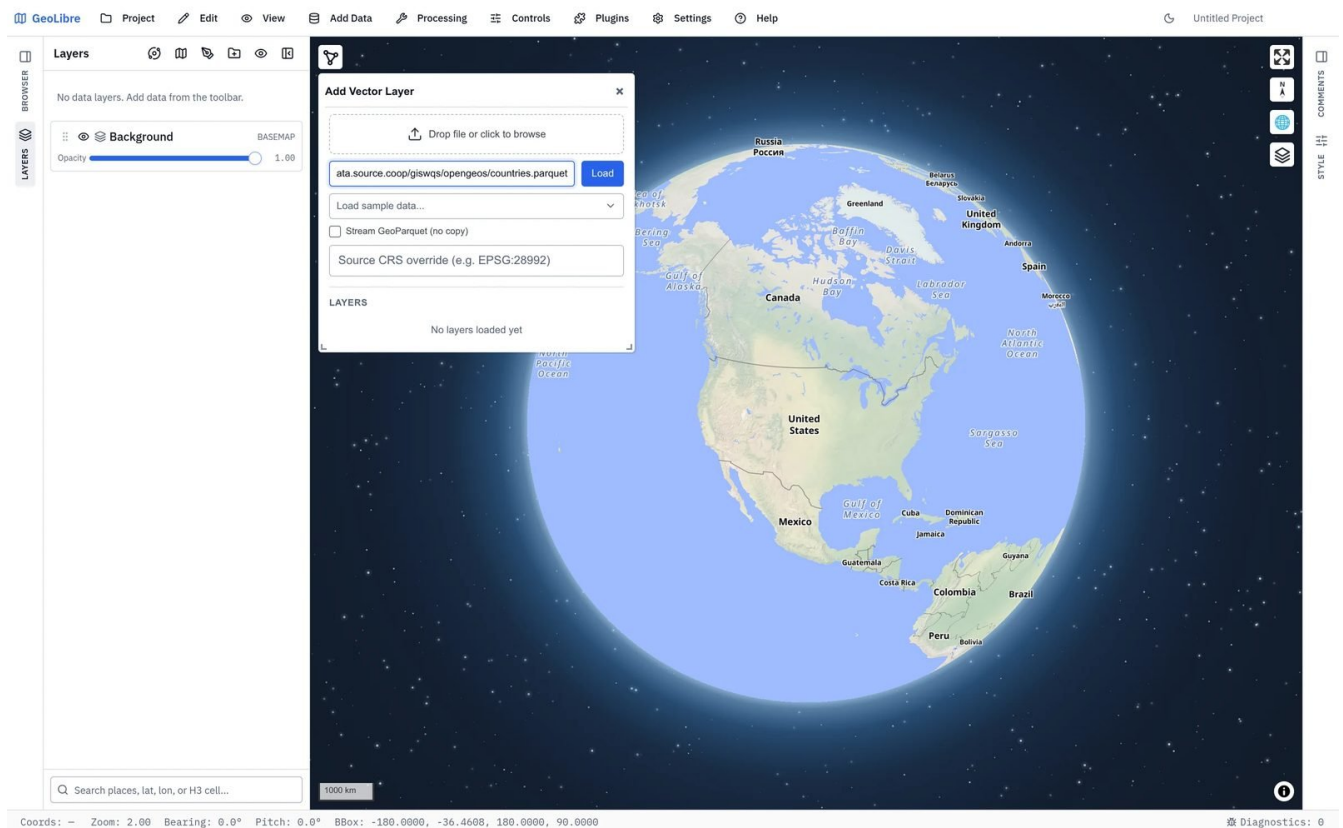
[web.geolibre.app](https://web.geolibre.app)을 열거나 데스크톱 앱을 실행합니다. 베이스맵과 빈 **Layers** 패널에서 시작합니다.

## 2. 레이어 추가

1. **Add Data → Vector Layer**(데이터 추가 → 벡터 레이어)를 엽니다.
2. Add Vector 패널에서 벡터 URL을 입력합니다. 샘플 국가 데이터셋을 사용할 수 있습니다:

`https://data.source.coop/giswqs/opengeos/countries.parquet`

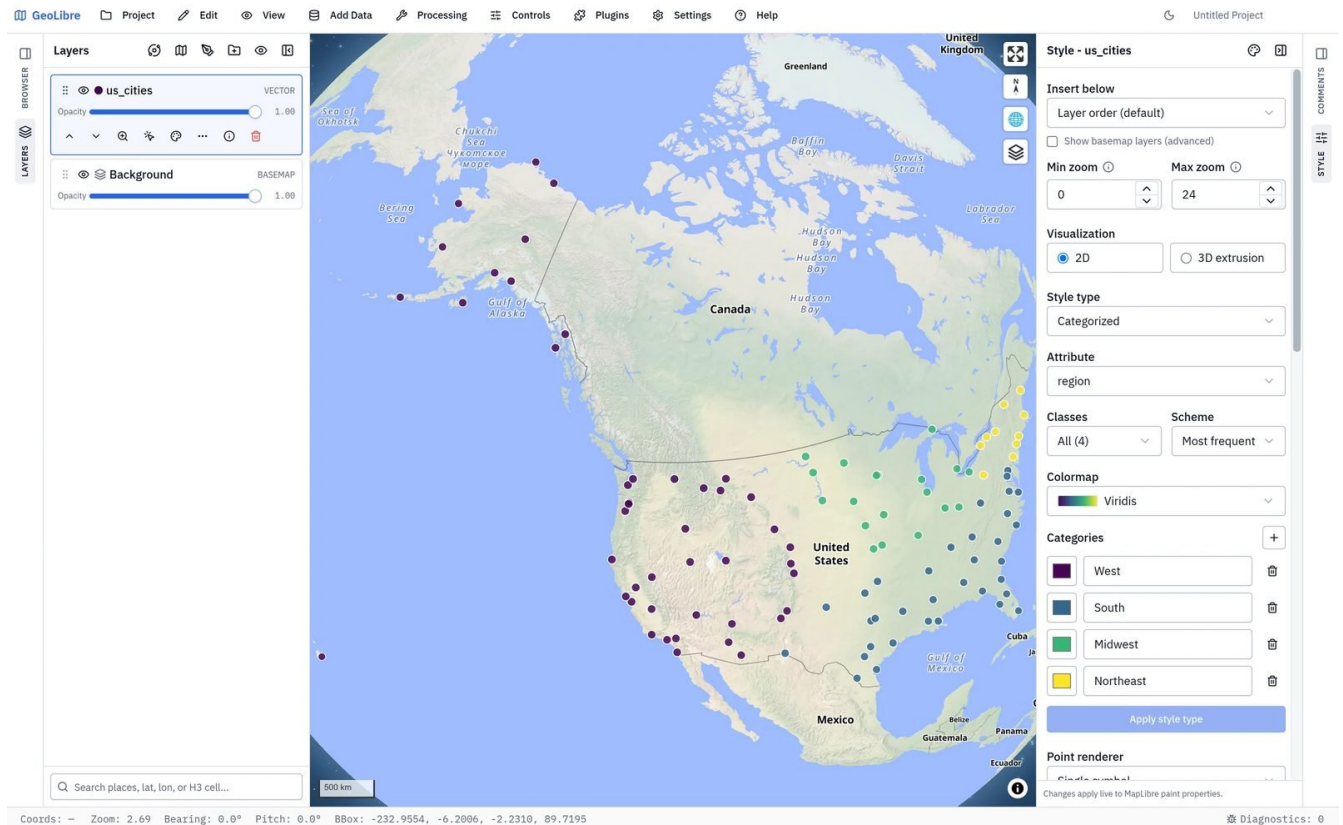
3. **Load**(로드)를 클릭합니다. 국가들이 지도에 나타나고 **countries** 레이어가 Layers 패널에 추가됩니다.



지원되는 모든 소스는 [데이터 추가](#)를 참조하세요.

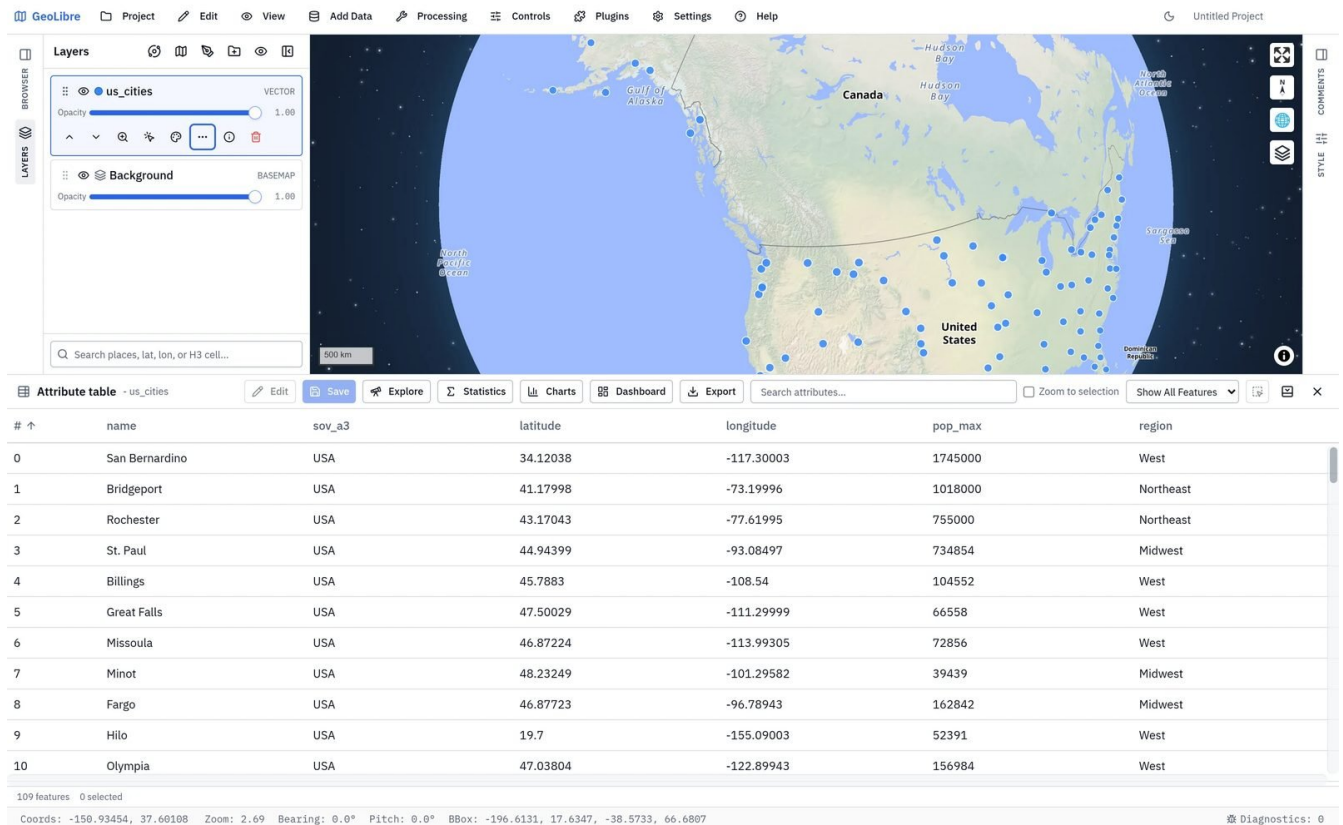
### 3. 레이어 스타일 지정

1. Layers 패널에서 **countries** 레이어를 선택한 다음, 오른쪽의 **Style** 패널이 접혀 있으면 펼칩니다.
2. **Fill color**(채우기 색상), **Outline color**(외곽선 색상), **Fill opacity**(채우기 불투명도)를 취향에 맞게 조정합니다.
3. 단계 구분도를 만들려면 **Style type**(스타일 유형)을 **Graduated**(등급)로 설정하고, 숫자형 **Attribute**(속성)(예: **POP\_EST** 또는 **GDP\_MD\_EST**)를 고른 뒤 **Classes**(클래스) 수, **Scheme**(분류 방식), **Colormap**(컬러맵)을 선택하고 **Apply style type**(스타일 유형 적용)을 클릭합니다.



### 4. 데이터 살펴보기

1. 상태 표시줄의 **Attribute table**(속성 테이블)을 클릭하여 펼친 다음 **countries** 레이어를 선택하여 레코드를 로드합니다. **속성 테이블**을 참조하세요.
2. 열을 기준으로 정렬하거나 **Search attributes**(속성 검색)에 입력하여 피처를 찾습니다. 행을 선택하면 지도에서 강조 표시됩니다.
3. **Explore**(탐색)를 클릭하면 모든 필드를 한 번에 프로파일링합니다. 필드의 유형, 완전성, 값의 분포를 보여줍니다.
4. 레이어의 **Identify features**(피처 식별)를 켜고 지도에서 국가를 클릭하여 팝업으로 속성을 볼 수도 있습니다.



## 5. 저장 또는 공유

- **Project → Save**(프로젝트 → 저장)를 사용하여 `.geolibre.json` 파일을 기록합니다. 데스크톱 앱은 디스크에 저장하고, 브라우저 빌드는 브라우저가 저장 대화 상자를 제공하면 그것을 사용하고 그렇지 않으면 파일을 다운로드합니다. [프로젝트](#)를 참조하세요.
- 어디서든 **Project → Share**(프로젝트 → 공유)를 사용하여 프로젝트를 업로드하고 공개 링크를 받을 수 있습니다. [공유 및 임베딩](#)을 참조하세요.

## 다음 단계

- [클라우드 네이티브 데이터](#)에서 클라우드 네이티브 및 원격 포맷을 로드합니다.
- [벡터 분석](#)에서 지오메트리 도구를 실행합니다.
- [공간 SQL](#)에서 데이터를 쿼리합니다.

<https://geolibre.app/tutorials/cloud-native-data/>

# 클라우드 네이티브 데이터

GeoLibre는 클라우드 네이티브 지리공간 포맷을 위해 만들어졌습니다. 벡터에는 GeoParquet과 FlatGeobuf, 래스터에는 Cloud-Optimized GeoTIFF(COG), 타일에는 PMTiles를 사용합니다. 이 튜토리얼에서는 이들을 원격 URL에서 로드하고 로컬 데이터셋을 변환합니다.

## 원격 GeoParquet 로드

GeoParquet은 HTTP를 통해 잘 읽히는 압축된 열 지향 벡터 포맷입니다.

1. **Add Data → GeoParquet Layer**(데이터 추가 → GeoParquet 레이어)(또는 **Vector Layer**(벡터 레이어))를 엽니다.
2. GeoParquet URL을 입력합니다. 예:
 

```
https://data.source.coop/giswqs/opengeos/countries.parquet
```
3. 큰 파일을 메모리에 복사하지 않으려면 **Stream GeoParquet (no copy)**(GeoParquet 스트리밍(복사 없음))을 활성화하세요. HTTP 범위 요청으로 파일을 제자리에서 쿼리합니다. 이 방식은 행이 공간적으로 정렬된(예: GeoLibre 자체 Conversion 도구가 기록하는 Hilbert 순서) 대용량 원격 파일에서 가장 효과적이며, 관련된 행 그룹만 가져옵니다.
4. **Load**(로드)를 클릭합니다.

## FlatGeobuf 로드

FlatGeobuf는 공간 인덱스를 갖춘 스트리밍 친화적 벡터 포맷입니다.

1. **Add Data → FlatGeobuf Layer**(데이터 추가 → FlatGeobuf 레이어)를 엽니다.
2. `.fgb` URL을 입력하고 로드합니다. 포맷이 허용하는 경우 GeoLibre는 현재 뷰에 있는 피처만 가져옵니다.

## COG 로드

Cloud-Optimized GeoTIFF는 클라이언트가 필요한 타일만 읽을 수 있도록 구성된 일반 GeoTIFF입니다.

1. **Add Data → Raster Layer**(데이터 추가 → 래스터 레이어)를 엽니다.
2. COG( `.tif` )의 URL을 입력하고 로드합니다. 그런 다음 **Style 패널**에서 밝기, 대비, 채도, 색조를 조정할 수 있습니다.

**드래그 앤 드롭**

로컬 GeoTIFF 또는 COG를 지도에 드래그하여 래스터 레이어로 추가할 수도 있습니다. [데이터 추가](#)를 참조하세요.

## 로컬 데이터를 클라우드 네이티브로 변환

**Processing → GeoLibre Toolbox → Conversion**(처리 → GeoLibre 툴박스 → 변환)을 사용하여 클라우드 네이티브 파일을 기록합니다. [처리 도구](#)를 참조하세요.

- 브라우저 빌드에서는 **Vector to GeoParquet**(벡터를 GeoParquet으로)과 **CSV to GeoParquet**(CSV를 GeoParquet으로)이 DuckDB-WASM에서 실행되고, **Vector to FlatGeobuf**(벡터를 FlatGeobuf로), **Vector to PMTiles**(벡터를 PMTiles로), **Raster to COG**(래스터를 COG로)는 WebAssembly( `geolibre-wasm` )에서 실행되므로, 어느 것도 사이드카가 필요하지 않습니다.
- 데스크톱 앱은 대신 이들을 Python 사이드카에서 실행하며, 사이드카는 더 많은 입력 포맷을 읽고(브라우저의 Raster to COG는 GeoTIFF만 읽음) 브라우저 기록기가 보존하지 못하는 데이터 유형을 보존합니다.

예를 들어 로컬 GeoJSON을 GeoParquet으로 게시하려면:

1. **Processing → GeoLibre Toolbox → Conversion → Vector to GeoParquet**을 엽니다.
2. 입력 파일과 출력 경로를 선택합니다.
3. 변환을 실행한 다음, 결과 GeoParquet을 지도에 다시 추가하여 확인합니다.

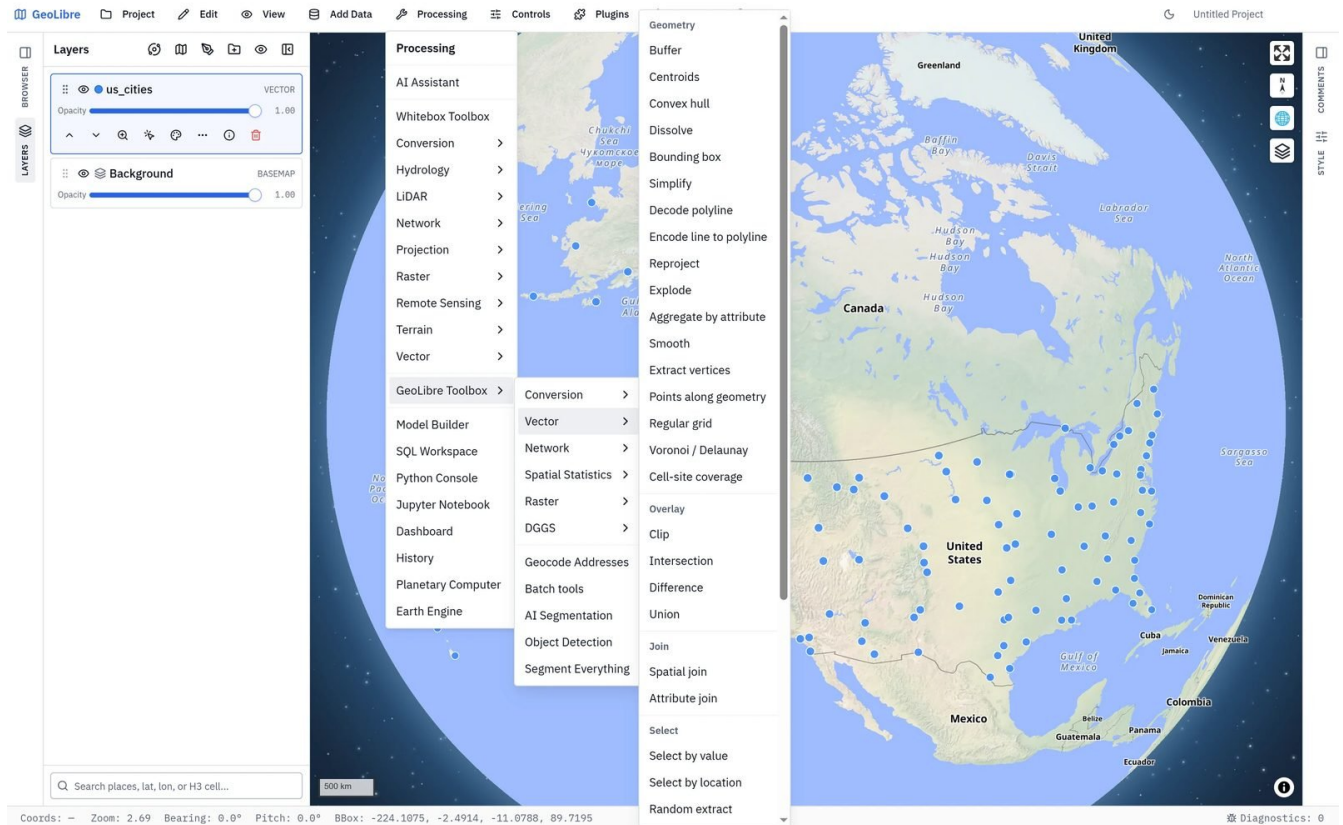
## 다음 단계

- [공간 SQL](#)에서 이 포맷들을 직접 쿼리합니다. SQL Workspace는 URL에서 Parquet, CSV, JSON, GeoJSON을 읽습니다.
- Time Slider 플러그인으로 COG 시계열을 애니메이션으로 재생합니다. [데이터 통합](#)을 참조하세요.

<https://geolibre.app/tutorials/vector-analysis/>

# 벡터 분석

이 튜토리얼은 작은 벡터 워크플로를 실행합니다. 레이어를 버퍼링하고, 다른 레이어와 중첩한 뒤, 결과를 내보냅니다. **Processing → GeoLibre Toolbox → Vector**(처리 → GeoLibre 툴박스 → 벡터) 아래의 **벡터 도구**를 사용합니다.



이 메뉴는 두 개의 툴박스가 공유합니다

**Processing → Vector**(처리 → 벡터)(구분선 위)는 Whitebox 카탈로그의 벡터 카테고리이고, **Processing → GeoLibre Toolbox → Vector**는 이 튜토리얼에서 사용하는 대화 상자입니다. 하나의 메뉴에 있는 두 개의 툴박스를 참조하세요.

## 1. 입력 데이터 로드

벡터 레이어를 하나 이상 추가합니다(데이터 추가 참조). 중첩을 하려면 두 개의 레이어가 필요합니다. 예를 들어 포인트나 라인 집합과, 클립 기준이 될 폴리곤 레이어입니다.

## 2. 레이어 버퍼링

1. **Processing → GeoLibre Toolbox → Vector → Buffer**(처리 → GeoLibre 툴박스 → 벡터 → 버퍼)를 엽니다.
2. **Input layer**(입력 레이어)를 해당 레이어로 설정합니다.
3. **Distance**(거리)와 **Units**(단위)(킬로미터, 미터 또는 마일)를 설정합니다.
4. **Engine**(엔진)을 선택합니다:
5. **Client (Turf.js)**(클라이언트(Turf.js))는 별도 설정 없이 브라우저에서 실행됩니다.
6. **Sidecar (GeoPandas)**(사이드카(GeoPandas))는 투영을 고려한 거리 계산을 위해 Python 사이드카에서 실행됩니다(데스크톱 앱).
7. **Python (Pyodide)**는 사이드카와 동일한 GeoPandas 코드를 브라우저에서 실행하므로, 사이드카를 설치하지 않아도 결과가 사이드카와 일치합니다. 런타임은 처음 사용할 때 한 번 다운로드됩니다.
8. **Run**(실행)을 클릭합니다. 버퍼 레이어가 지도에 추가됩니다.

### 투영을 고려한 거리

클라이언트 엔진은 지리 좌표에서 버퍼를 계산합니다. 넓은 영역에서 정확한 미터 단위 거리를 얻으려면 버퍼링 전에 재투영하는 두 GeoPandas 엔진 중 하나를 사용하세요. 데스크톱 앱에서는 **Sidecar (GeoPandas)**, 웹 빌드를 포함한 어디서나 **Python (Pyodide)**를 쓸 수 있습니다.

## 3. 두 레이어 중첩

버퍼(또는 임의의 폴리곤 레이어)와 두 번째 레이어로 중첩을 실행합니다:

- **Clip**(클립)은 입력 중 중첩 레이어 안에 들어가는 부분만 남기며 입력의 속성을 보존합니다.
- **Intersection**(교차)은 두 폴리곤 레이어가 겹치는 영역만 남깁니다.
- **Difference**(차집합)은 입력에서 중첩 레이어의 영역을 제거합니다.
- **Union**(합집합)은 두 폴리곤 레이어를 하나로 병합합니다(어느 엔진에서도 속성은 보존되지 않음).
- **Spatial join**(공간 조인)은 공간 관계(교차, 포함됨, 포함)를 기준으로 조인 레이어의 속성을 각 입력 피처에 붙입니다. 예를 들어 각 포인트에 그것을 포함하는 폴리곤을 태그합니다. 모든 지오메트리 유형에서 작동합니다.
- **Attribute join**(속성 조인)은 키 필드가 일치하는 곳에서 조인 테이블의 속성을 각 입력 피처에 붙이며, 지오메트리는 관여하지 않습니다(예: 공유 FIPS 코드로 인구조사 통계를 경계 폴리곤에 조인). 일대일 조인입니다(첫 번째로 일치하는 조인 행이 적용됨). 양쪽의 키 필드를 선택하고, 선택적으로 가져올 필드를 나열하고, 내부 조인 또는 왼쪽 조인을 선택합니다.

**Processing → GeoLibre Toolbox → Vector**에서 도구를 열고, 입력 레이어와 중첩 레이어를 선택한 뒤 **Run**(실행)을 누릅니다.

## 4. 검토 및 다듬기

---

결과 레이어의 **속성 테이블**을 열어 출력을 확인하고, 입력과 구별되도록 **스타일**을 조정합니다. 모든 실행은 **Processing → History**(처리 → 기록)에도 기록되며, 여기서 클릭 한 번으로 항목을 다시 실행하거나 동등한 Python 코드를 복사할 수 있습니다.

## 5. 결과 내보내기

---

출력을 클라우드 네이티브 파일로 저장하려면 **Processing → GeoLibre Toolbox → Conversion**(처리 → GeoLibre 툴박스 → 변환)(예: **Vector to GeoParquet** 또는 **Vector to FlatGeobuf**)을 사용합니다. **클라우드 네이티브 데이터**를 참조하세요. **속성 테이블**이나 **SQL Workspace**에서 레코드를 내보낼 수도 있습니다.

## 다음 단계

---

- **공간 SQL**로 같은 종류의 분석을 SQL에서 수행합니다.
- **지형 분석**에서 래스터 분석으로 넘어갑니다.

<https://geolibre.app/tutorials/terrain-analysis/>

# 지형 분석

이 튜토리얼은 수치 표고 모델(DEM)에서 지형 산출물인 음영기복, 경사도, 등고선을 도출합니다. **Processing** → **GeoLibre Toolbox** → **Raster**(처리 → GeoLibre 툴박스 → 래스터) 아래의 **래스터 도구**를 사용합니다.

## 어떤 엔진이 실행하는가

**Hillshade**(음영기복), **Slope**(경사), **Aspect**(경사향)는 각각 **Client (browser)**(클라이언트(브라우저)) 엔진을 제공하므로, 이 튜토리얼의 처음 세 단계는 아무것도 설치하지 않고 브라우저 빌드에서 실행됩니다. **Contour**(등고선)와 **Clip by mask layer**(마스크 레이어로 클립)는 클라이언트 엔진이 없고 rasterio Python 사이드카가 필요하며, 데스크톱 앱이 이를 관리합니다. **시작하기**를 참조하세요. 각 도구 상단의 엔진 선택기가 사용 가능한 엔진을 보여줍니다. 대용량 래스터나 지리 CRS의 지형에는 사이드카를 권장합니다.

## 1. DEM 로드

GeoTIFF 또는 COG DEM과 같은 표고 래스터를 레이어로 추가합니다(**데이터 추가** 참조). 래스터 도구는 GeoTIFF 파일을 읽으므로 로컬 래스터나 접근 가능한 래스터가 가장 적합합니다. 브라우저 엔진이 있는 도구(Hillshade, Slope, Aspect, Clip by extent)는 **Use a layer on the map**(지도의 레이어 사용)으로 이미 추가한 래스터 레이어를 선택할 수도 있습니다.

## 2. 음영기복

1. **Processing** → **GeoLibre Toolbox** → **Raster** → **Hillshade**(처리 → GeoLibre 툴박스 → 래스터 → 음영기복)를 엽니다.
2. DEM을 입력으로 선택하고, 조명을 조정하고 싶다면 방위각, 고도각, z-factor를 설정합니다.
3. 실행합니다. 음영기복 래스터가 지도에 추가됩니다. 다른 레이어 아래에 배치하고 맨 위 레이어의 **블렌드 모드**를 **Multiply**(곱하기)로 설정하세요. 불투명도를 낮추면 둘 다 흐려지는 것과 달리, 색상을 통해 지형 기복이 비쳐 보이게 됩니다.

## 3. 경사와 경사향

- **Processing** → **GeoLibre Toolbox** → **Raster** → **Slope**(처리 → GeoLibre 툴박스 → 래스터 → 경사)는 DEM에서 가파른 정도를 계산합니다.
- **Processing** → **GeoLibre Toolbox** → **Raster** → **Aspect**(처리 → GeoLibre 툴박스 → 래스터 → 경사향)는 가장 가파른 경사의 나침반 방향을 계산합니다.

둘 중 하나를 DEM에 대해 실행하고 출력을 **컬러맵**으로 스타일 지정합니다. **Controls 메뉴**에서 **Colorbar**(컬러바)를 열어 값 척도를 표시합니다.

## 4. 등고선

---

1. **Processing → GeoLibre Toolbox → Raster → Contour**(처리 → GeoLibre 툴박스 → 래스터 → 등고선)를 엽니다.
2. DEM을 선택하고 등고선 **interval**(간격)(선 사이의 표고 차이)을 설정합니다.
3. 실행하여 등고선을 벡터 레이어로 생성합니다. 다른 벡터 데이터처럼 라벨을 붙이고 스타일을 지정할 수 있습니다.

Contour는 사이드카에서만 실행되므로 이 단계에는 데스크톱 앱이 필요합니다.

## 5. 관심 영역으로 클립

---

출력을 연구 영역으로 제한하려면 **Processing → GeoLibre Toolbox → Raster → Clip by extent**(처리 → GeoLibre 툴박스 → 래스터 → 범위로 클립)(경계 상자 기준, 브라우저에서도 실행됨) 또는 **Clip by mask layer**(마스크 레이어로 클립)(벡터 마스크 기준, 사이드카 전용)를 사용합니다. **처리 도구**를 참조하세요.

## 다음 단계

---

- **Polygonize**(폴리곤화)로 래스터 출력을 벡터로 변환하거나, 공유를 위해 **Raster to COG**(래스터를 COG로)를 기록합니다. **클라우드 네이티브 데이터**를 참조하세요.
- Time Slider 플러그인으로 래스터 시계열을 애니메이션으로 재생합니다. **데이터 통합**을 참조하세요.

<https://geolibre.app/tutorials/spatial-sql/>

# 공간 SQL

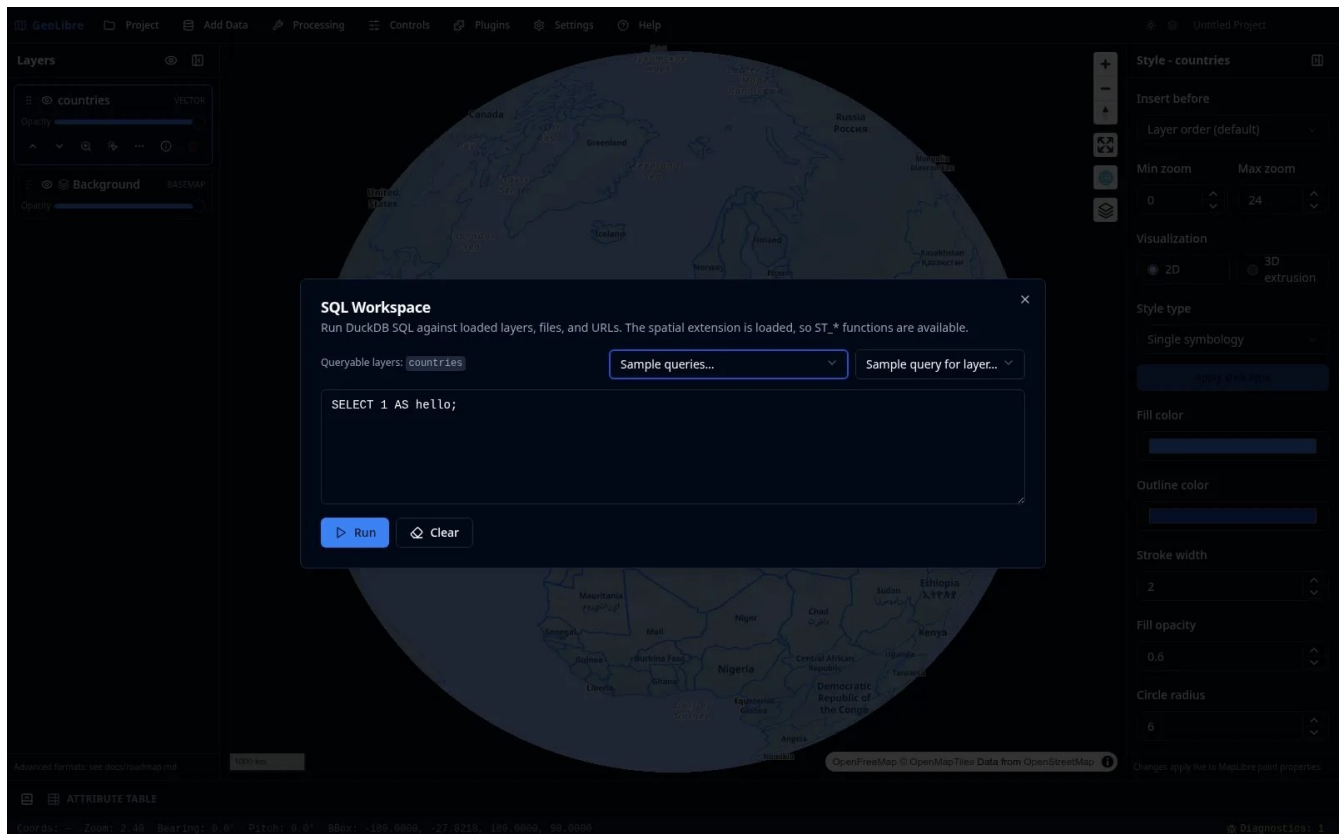
SQL Workspace를 사용하면 DuckDB Spatial SQL로 데이터를 분석하고 결과를 지도에 추가할 수 있습니다. DuckDB-WASM에서 실행되므로 데스크톱 앱뿐 아니라 브라우저에서도 작동합니다. **Processing → SQL Workspace**(처리 → SQL 워크스페이스)에서 엽니다.

## 1. 로드된 레이어 쿼리

로드된 모든 벡터 레이어는 쿼리할 수 있는 테이블입니다. 샘플 국가 레이어를 로드한 다음(첫 번째 지도 참조) 다음을 실행하세요:

```
SELECT NAME, CONTINENT, GDP_MD_EST
FROM countries
ORDER BY GDP_MD_EST DESC
LIMIT 10;
```

Run(실행)을 클릭하면 추정 GDP가 가장 높은 10개 국가가 표시됩니다.



### 샘플 데이터셋 열

NAME, CONTINENT, POP\_EST, GDP\_MD\_EST 는 샘플 countries.parquet 의 Natural Earth 필드 이름이며, 지오메트리 열은 geom 입니다. 다른 데이터셋에는 고유한 열 이름이 있습니다. 열 이름을 알아내려면 먼저 DESCRIBE SELECT \* FROM 'your-file-url' 을 실행하세요.

## 2. 원격 파일 직접 쿼리

파일을 먼저 로드할 필요가 없습니다. Workspace는 FROM 절의 순수 URL을 감지하여 자동으로 적절한 리더로 감싼 다음, HTTP 범위 요청으로 파일을 스트리밍하므로 전체를 다운로드하지 않습니다. 이 순수 URL 축약 표기는 SQL Workspace의 편의 기능입니다. 표준 DuckDB CLI에서는 read\_parquet('https://...') 를 명시적으로 작성해야 합니다.

```
SELECT COUNT(*) AS n
FROM https://data.source.coop/giswqs/opengeos/countries.parquet;
```

## 3. 공간 함수 사용

spatial 확장이 로드되어 있으므로 ST\_\* 함수를 사용할 수 있습니다. 예를 들어 면적을 계산하면서 결과를 지도에 표시할 수 있도록 geom 열을 유지합니다:

```
-- area is in square degrees because the data is in EPSG:4326;
-- the ranking is valid, but for km2 use
-- ST_Area_Spheroid(ST_FlipCoordinates(geom)) / 1e6 (it expects lat/lon order).
SELECT NAME, ST_Area(geom) AS area, geom
FROM https://data.source.coop/giswqs/opengeos/countries.parquet
WHERE CONTINENT = 'Africa'
ORDER BY area DESC;
```

## 4. 결과를 지도에 추가

쿼리가 지오메트리 열을 반환하면 **Add as layer**(레이어로 추가)를 클릭하여 결과로 새 레이어를 만들 수 있으며, 원하면 이름을 지정할 수 있습니다. 결과 레이어는 다른 벡터 레이어와 마찬가지로 식별, 선택, 속성 테이블을 지원하며, 여러 결과 레이어를 한 번에 추가할 수 있습니다.

## 5. 내보내기

쿼리 결과를 워크스페이스에서 바로 CSV 또는 GeoParquet으로 내보냅니다. **Export CSV**(CSV 내보내기)와 **Export GeoParquet**(GeoParquet 내보내기) 버튼은 **Add as layer** 옆에 있습니다. [SQL Workspace](#)를 참조하세요.

### 샘플 쿼리와 기록

**Sample queries...**(샘플 쿼리) 메뉴를 사용하여 동작하는 쿼리에서 시작하고, **History...**(기록)로 이전 쿼리를 다시 실행하세요.

## 다음 단계

- 메뉴 기반 **벡터 분석** 도구로 동등한 지오메트리 연산을 수행합니다.
- **클라우드 네이티브 데이터**에서 결과를 클라우드 네이티브 포맷으로 변환합니다.

<https://geolibre.app/tutorials/sharing-embedding/>

# 공유 및 임베딩

마음에 드는 지도를 만들었다면 공개 링크로 게시하고 어떤 웹 페이지에든 임베드할 수 있습니다. 이 튜토리얼은 두 가지를 모두 다룹니다. 전체 참조는 [임베딩 및 공유](#)를 참조하세요.

## 1. 계정 연결

공유는 `share.geolibre.app`에 업로드합니다.

- **웹 앱: Project → Share...**(프로젝트 → 공유)를 열고 **Sign in to share.geolibre.app**(share.geolibre.app에 로그인)을 클릭합니다. 열리는 팝업에서 접근 요청을 승인하면 브라우저 세션 동안 연결이 유지됩니다.
- **GeoLibre Desktop**: 같은 **Sign in to share.geolibre.app** 버튼을 클릭합니다. 로그인은 시스템 브라우저에서 열리며, 데스크톱 앱은 세션을 OS 자격 증명 저장소에 보관합니다.
- **폴백(모든 빌드)**: 개인 API 토큰을 **Settings → Environment Variables → Share.GeoLibre API token**(설정 → 환경 변수 → Share.GeoLibre API 토큰)에 붙여넣습니다. 토큰은 [share.geolibre.app/settings](https://share.geolibre.app/settings)의 Settings → API tokens에서 만들 수 있습니다.

## 2. 프로젝트 공유

1. 지도를 만듭니다. 레이어를 추가하고, 스타일을 지정하고, 보는 사람이 처음 마주할 지도 뷰를 설정합니다.
2. **Project → Share...**를 엽니다.
3. 프로젝트 제목을 확인하고 업로드합니다. GeoLibre는 `.geolibre.json` 파일의 공개 URL을 반환합니다. 예:

```
https://share.geolibre.app/you/my-map.geolibre.json
```

공유된 파일은 로컬 저장과 동일한 레이어, 스타일, 플러그인 상태, 지도 뷰를 담습니다. 예외는 사용자 컴퓨터의 파일을 읽는 레이어입니다. `share.geolibre.app`은 프로젝트 파일만 저장하고 데이터 파일은 결코 저장하지 않으므로, 대화 상자는 공유 전에 이러한 레이어를 누락으로 나열합니다. 대신 호스팅하는 방법은 [로컬 데이터 공유](#)를 참조하세요.

## 3. 공유된 지도 열기

누구나 공유된 프로젝트를 `url` 매개변수로 전달하여 라이브 뷰어에서 열 수 있습니다:

```
https://web.geolibre.app/?url=https://share.geolibre.app/you/my-map.geolibre.json
```

## 4. 페이지에 임베드

`<iframe>` 과 임베드 매개변수를 사용하여 크롬(UI 요소)을 제어합니다. 지도만 깔끔하게 임베드하려면:

```
<iframe
  src="https://web.geolibre.app/?url=https://share.geolibre.app/you/my-
  map.geolibre.json&maponly"
  title="GeoLibre map"
  width="100%"
  height="600"
  style="border: 0;"
  loading="lazy"
  allow="fullscreen; geolocation"
></iframe>
```

매개변수로 모양을 조정합니다(조합 가능):

- `maponly` 는 모든 크롬을 숨기고 지도만 남깁니다.
- `layout=viewer` 는 읽기 전용 지도를 제공합니다. Layers, View, Controls, 베이스맵, 검색/식별은 유지되고 프로젝트를 편집하는 모든 요소는 숨겨집니다.
- `layout=compact` 는 아이콘만 있는 얇은 툴바를 유지합니다.
- `toolbar=none` 은 패널과 상태 표시줄은 유지한 채 상단 툴바를 숨깁니다.
- `panels=none` 은 측면 및 하단 패널을 숨기고 툴바는 유지합니다.
- `theme=dark` 는 로드 시 어두운 테마를 강제합니다.

독자가 레이어를 켜고 끄며 데이터를 탐색하되 변경은 할 수 없어야 할 때는 `layout=viewer` 를, 지도가 상호작용 대상이 아니라 페이지의 그림 역할을 할 때는 `maponly` 를 사용하세요.

전체 매개변수 표를 참조하세요.

## 5. 페이지에서 지도 제어

URL 매개변수는 로드 시 한 번만 임베드를 구성합니다. 계속해서 지도와 통신하려면 — 사용자가 방금 자신의 UI에서 클릭한 레코드로 날아가거나, 지도 안에서 사용자가 하는 일을 전달받으려면 — 타입이 지정된 클라이언트를 설치하세요:

```
npm install @geolibre/embed
```

```
import { connect } from "@geolibre/embed";

const map = await connect(document.querySelector("iframe"), {
  origin: "https://web.geolibre.app",
});

map.on("selectionChanged", ({ featureIds }) => showRecordFor(featureIds[0]));

async function focusField(field) {
  await map.setView({ bbox: field.bbox });
  await map.highlightFeature({ layerId: "fields", filter: { parcel_id: field.id }, fit: true })
```

```
};  
}
```

이 기능은 페이지의 출처(origin)를 허용 목록에 등록한 배포에 대해서만 작동합니다. `web.geolibre.app` 은 등록하지 않았으므로, 런타임 API는 직접 호스팅한 빌드용입니다. 허용 목록, 전체 명령 목록, 그리고 종속성을 추가하고 싶지 않을 때 쓸 수 있는 원시 `postMessage` 프로토콜은 [런타임에 지도와 통신하기](#)를 참조하세요.

## 다음 단계

---

- 공유하기 전에 [Controls 메뉴](#)에서 표시할 컨트롤을 조정합니다.
- [첫 번째 지도](#)로 돌아가 공유하고 싶은 지도를 만듭니다.

<https://geolibre.app/tutorials/supported-formats-en/>

# GeoLibre 지원 GIS 데이터 포맷 — 완전 참조

**초록:** 이 글은 GeoLibre가 기본으로 지원하는 모든 GIS 데이터 포맷을 벡터, 래스터, 클라우드 네이티브 지리 공간 포맷에 걸쳐 포괄적으로 정리하고, 각 포맷의 내부 파싱 로직을 함께 다룹니다. WebGIS 개발자를 위한 완전한 기술 참조 자료입니다.

**원문 출처:** "GIS开发手记", 원문 링크: <https://mp.weixin.qq.com/s/GrtQzetCBgKtYruEhjsOAw>

## 머리말

이 글은 GeoLibre에 내장된 모든 지리공간 데이터 포맷을 벡터, 래스터, 클라우드 네이티브 지리공간 포맷으로 구분하여 체계적으로 정리하고, 각 포맷에 대응하는 파싱 의존성, 읽기 우선순위, 적용 시나리오를 문서화합니다. WebGIS 개발자를 위한 즉시 활용 가능한 참조 자료입니다.

## 1. 런타임 구성 및 플랫폼 개요

포맷을 나열하기 전에 "어디에서 실행되는가"를 명확히 하는 것이 중요합니다. 많은 포맷이 플랫폼별로 차이가 있기 때문입니다.

| 런타임     | 방법                                | 비고                                                                                           |
|---------|-----------------------------------|----------------------------------------------------------------------------------------------|
| 브라우저    | <code>web.geolibre.app</code> 열기  | 설치 불필요; 최초 로드 후 오프라인에서 동작                                                                    |
| 데스크톱    | Tauri v2 네이티브 애플리케이션              | Windows / macOS / Linux; Microsoft Store, Mac App Store, Homebrew, winget, AUR, Flatpak으로 제공 |
| Android | Google Play 네이티브 앱                | ABI당 약 40 MB                                                                                 |
| iOS     | App Store 네이티브 앱                  | iPhone과 iPad, Tauri v2 모바일을 통한 동일 코드베이스                                                      |
| Jupyter | <code>pip install geolibre</code> | 애플리케이션 전체가 노트북 셀에 임베디드                                                                       |

**핵심 애플리케이션에는 계정도, 서버도, 요금도 없습니다.** 로컬 파일은 제자리에서 읽히며 여러분의 머신에 그대로 남고, 앱이 로드된 뒤에는 로컬 워크플로가 오프라인에서도 계속 동작합니다. 예외는 선택적인 원격 구성 요소입니다. 온라인 카탈로그(STAC, Source Cooperative, Overture, Planetary Computer), CDN에서의 베이스맵 및 타일 다운로드, Earth Engine이나 인증된 ArcGIS 엔드포인트 같은 서비스는 모두 네트워크 접근이 필요하며, 일부는 자체 자격 증명이나 OAuth 로그인이 필요합니다.

## 2. 벡터 데이터 포맷 지원

벡터 데이터는 GeoLibre의 가장 성숙한 기능입니다. 파일 선택기에는 통합 포맷 허용 목록이 내장되어 있고, 드래그 앤 드롭 자동 감지를 지원하며(수동 포맷 선택 불필요), 17가지 주류 벡터 확장자를 다룹니다.

```
geojson, json, gpkg, geoparquet, parquet, fgb, flatgeobuf,
csv, tsv, kml, kmz, gml, gpx, dxf, tab, shp, zip
```

그냥 드래그 앤 드롭하면 됩니다. "어떤 포맷을 가져올지" 먼저 선택할 필요가 없습니다.

정말 흥미로운 것은 각 포맷을 뒤에서 어떤 엔진이 읽는가입니다. 이것이 실제 성능을 결정합니다.

| 포맷                          | 확장자                  | 읽기 엔진                        | 주목할 세부 사항                                                                                                               |
|-----------------------------|----------------------|------------------------------|-------------------------------------------------------------------------------------------------------------------------|
| <b>GeoJSON</b>              | .geojson .json       | 네이티브<br>JSON.parse           | 레거시 최상위 <code>crs</code> 멤버가 있는 파일은 DuckDB를 통해 WGS84로 자동 재투영                                                            |
| <b>GeoParquet / Parquet</b> | .parquet .geoparquet | DuckDB<br>read_parquet       | 원격 파일은 HTTP Range 스트리밍 읽기 사용; 전체 다운로드 불필요                                                                               |
| <b>FlatGeobuf</b>           | .fgb .flatgeobuf     | DuckDB<br>ST_Read            | —                                                                                                                       |
| <b>GeoPackage</b>           | .gpkg                | sql.js(SQLite WASM), GDAL 아님 | 다중 레이어 파일은 레이어 선택기 표시; <code>gpkg_ogr_contents</code> 자동 복구                                                             |
| <b>Shapefile(개별 파일)</b>     | .shp                 | shpjs                        | 데스크톱은 동반 파일 .dbf/.shx/.prj/.cpg 자동 읽기; 3D MultiPatch는 DuckDB로 폴백                                                        |
| <b>Shapefile(zip 아카이브)</b>  | .zip                 | flate 압축 해제<br>→ shpjs       | .prj 가 투영을, .cpg 가 DBF 인코딩을 결정(중국어 속성 인코딩 문제 해결); macOS __MACOSX 자동 건너뛰                                                 |
| <b>KML</b>                  | .kml                 | 커스텀 파서                       | 임베디드 스타일과 Folder 구조 보존; <TimeSpan> / <TimeStamp> 가 있는 플레이스마크는 Time Slider에서 애니메이션; GroundOverlay 이미지와 <Model> 3D 모델도 추출 |
| <b>KMZ</b>                  | .kmz                 | flate 압축 해제                  | 커스텀 아이콘과 서식 있는 설명 보존                                                                                                    |
| <b>GML</b>                  | .gml                 | DuckDB<br>ST_Read            | —                                                                                                                       |
| <b>GPX</b>                  | .gpx                 | 순수 JS                        | <b>세 개의 레이어로 자동 분할</b> : 웨이포인트 / 트랙 / 루트                                                                                |
| <b>CSV / TSV</b>            | .csv .tsv .txt .data | 커스텀 + DuckDB 폴백              | 구분자와 위도/경도 열 자동 감지; WKT 지오메트리 열은 DuckDB를 통과; 대화 상자에서 원본 CRS 지정 가능                                                       |

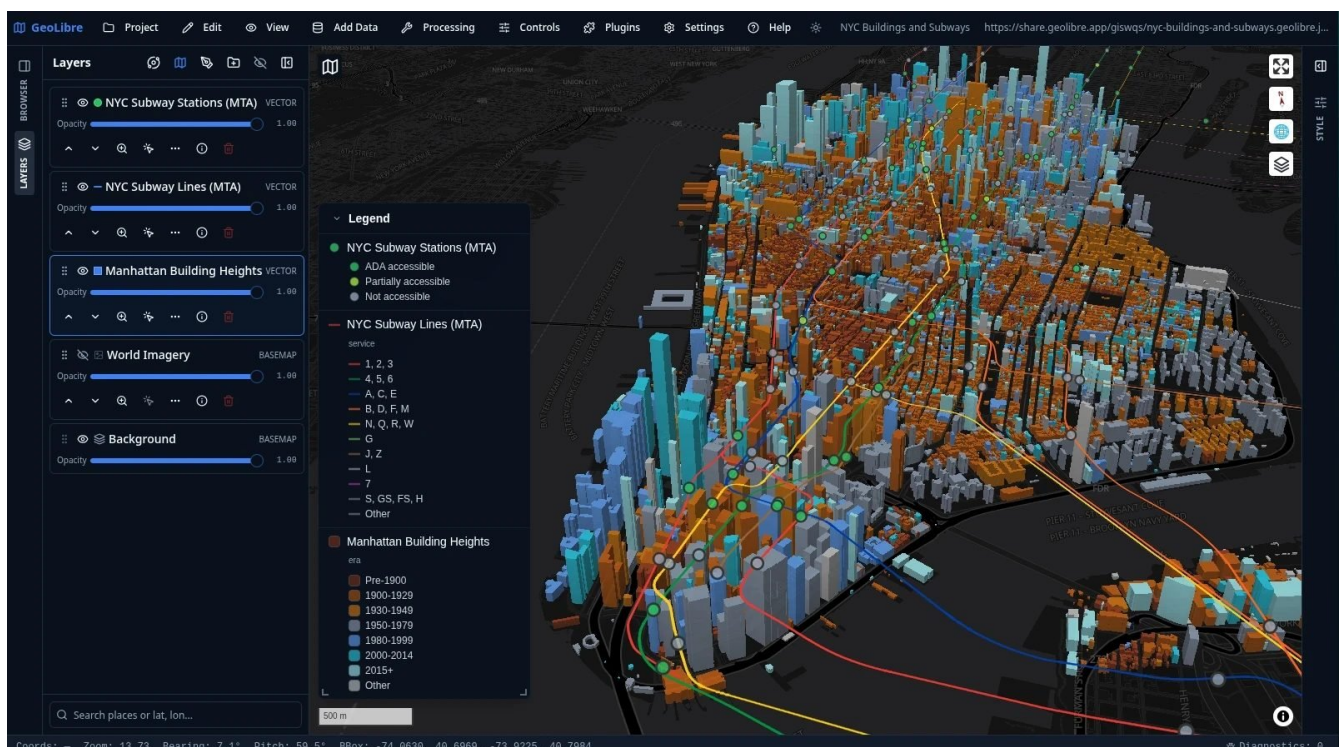
| 포맷                    | 확장자                                  | 읽기 엔진                           | 주목할 세부 사항                                                     |
|-----------------------|--------------------------------------|---------------------------------|---------------------------------------------------------------|
| Excel                 | .xlsx .xls                           | SheetJS( @e965/ xlsx )          | Delimited Text 패널에서 열기: 워크시트를 선택한 뒤 좌표 열 선택                   |
| CAD(DXF /DWG)         | .dxf .dwg                            | DuckDB<br>ST_Read               | 선택을 위한 레이어 목록 표시; <b>CAD 파일에는 좌표계가 없으므로 EPSG를 수동으로 선택해야 함</b> |
| MapInfo TAB           | .tab                                 | ST_Read                         | —                                                             |
| Esri File Geodatabase | .gdb 폴더                              | Python 사이드카                     | 데스크톱 전용, 사이드카 필요; Mac App Store 빌드에서는 숨김                      |
| OSM PBF               | .osm.pbf .pbf                        | osmix, Web Worker에서 실행          | 포인트/라인/폴리곤 레이어로 자동 분할; 50 MB 초과 시 확인 요청; 5분 타임아웃 보호           |
| GeoRSS                | .xml .rss .atom                      | 순수 JS                           | RSS 2.0 / Atom / RDF, GeoRSS Simple + GML 지오메트리 지원            |
| 지오태그 사진               | .jpg .jpeg .png .tif .kifrtiff .webp | EXIF GPS 데이터를 읽어 포인트 레이어를 바로 생성 | — UAV/드론 사진에 매우 실용적                                           |

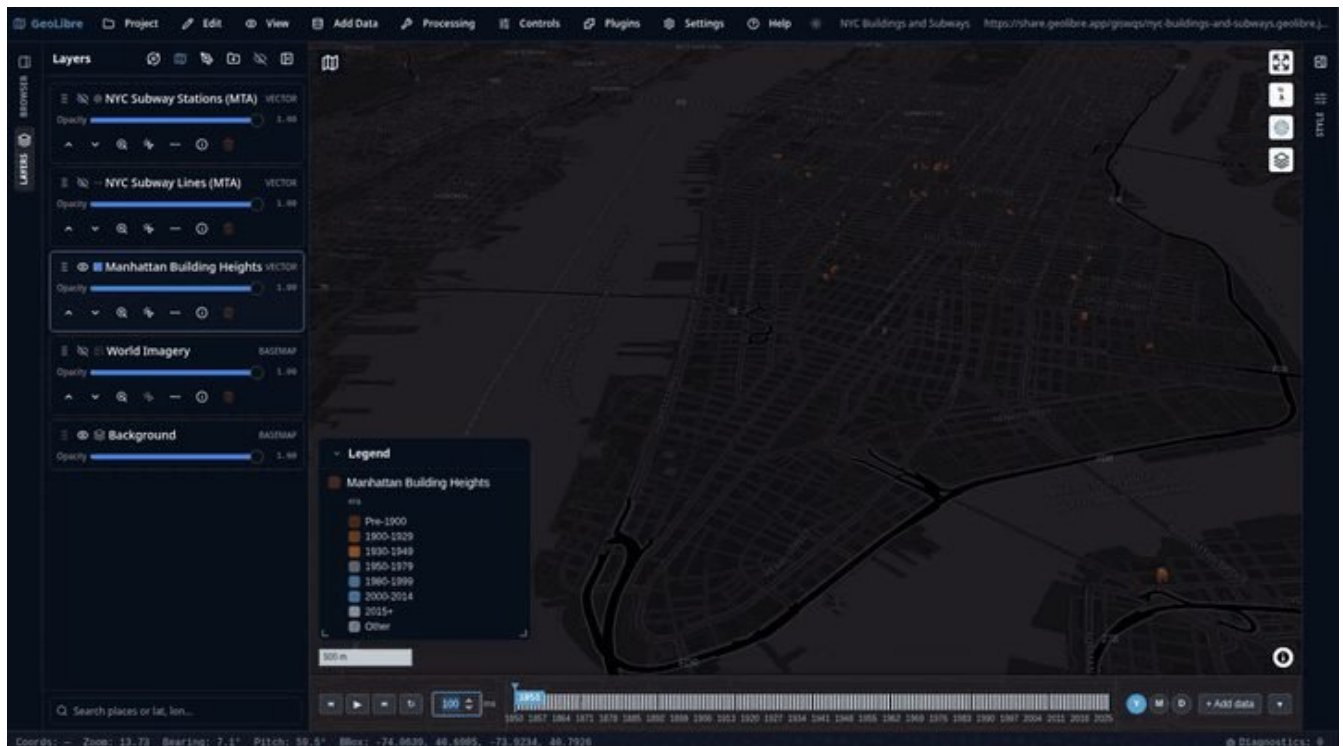
명시적으로 지원하지 않음: 원시 .osm XML(PBF만 지원).

### KML <Model> 3D 모델

KML <Model> 처리는 주목할 만합니다. 임베디드 COLLADA .dae 파일을 three.js로 로드한 뒤 GLB로 내보내 지도에 넣습니다. 이 프로젝트는 이 하나의 예외 상황을 위해 three.js를 끌어왔습니다.

로드한 뒤에는 "보이기만 하는 것"이 아닙니다. 심볼로지, 등급별 색상, 범례가 모두 함께 따라옵니다.

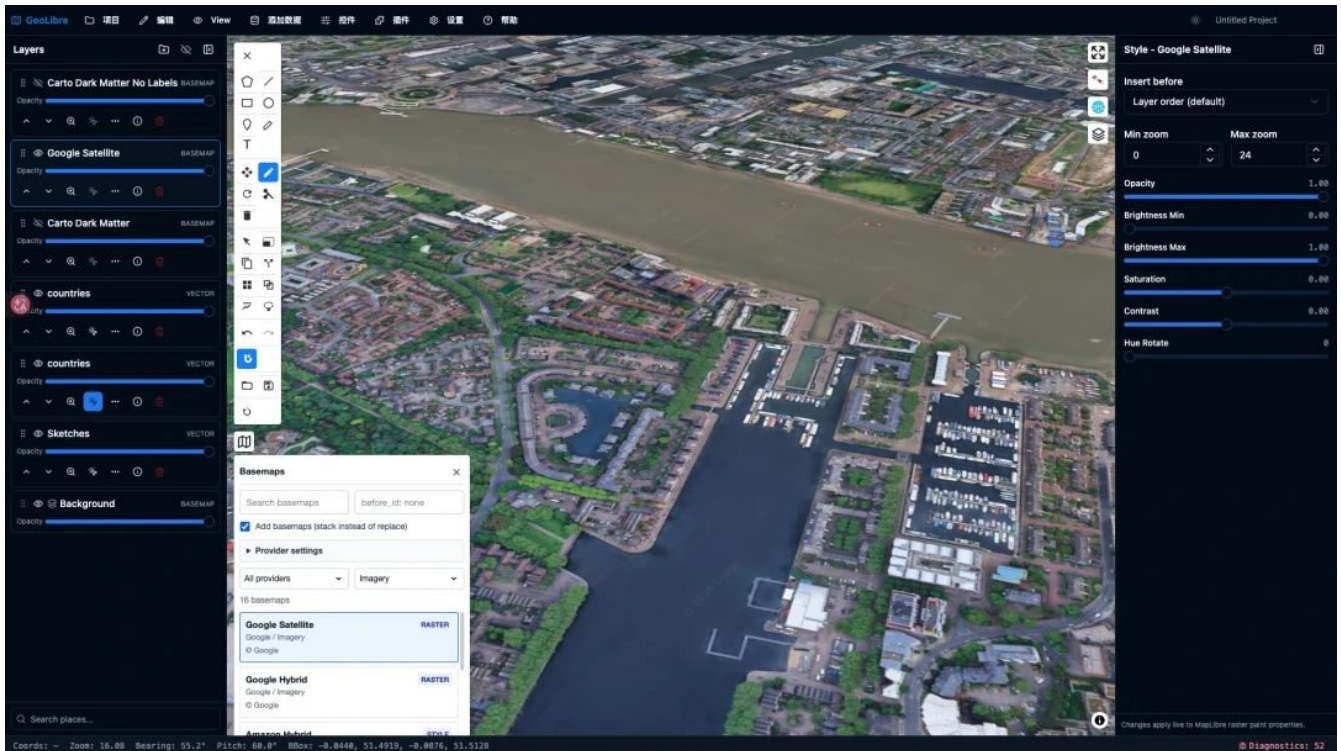




### 3. 래스터 및 영상

래스터 쪽은 벡터보다 범위가 좁지만 클라우드 네이티브 주류를 다룹니다.

| 포맷                     | 확장자                                       | 엔진                                   | 플랫폼                                                  |
|------------------------|-------------------------------------------|--------------------------------------|------------------------------------------------------|
| GeoTIFF / COG          | .tif .tiff                                | 기본 cog-tiler-wasm;<br>GPU 엔진으로 전환 가능 | 모든 플랫폼; 데스크톱 로컬 파일은 Range 읽기를 위해 Tauri asset 프로토콜 사용 |
| MosaicJSON / STAC item | .json                                     | 읽기 시점에 주문형 스티칭                       | 모든 플랫폼                                               |
| 지리참조 이미지               | 브라우저가 디코딩할 수 있는 모든 이미지 + GCP .csv / .txt  | 최소제공 아핀; gdal3.js로 내보내기              | 모든 플랫폼(gdal3.js는 CDN에서 로드)                           |
| KML GroundOverlay      | .kml / .kmz 에서                            | 네 모서리 좌표 이미지 레이아웃                    | <TimeSpan> 이 있는 것은 타임라인 애니메이션도 지원                    |
| 지리참조 비디오               | .mp4 + .webm                              | MapLibre video 소스                    | URL만                                                 |
| 변환 도구 입력 (래스터)         | .tif .tiff .img .vrt .asc .nc .jpg .hgt   | GDAL / rasterio 사이드카                 | 데스크톱; 브라우저는 .tif/.tiff 만 허용                          |
| Whitebox 래스터 I/O       | .tif .tiff .img .bil .flt .sdat .rdc .asc | whitebox-wasm                        | 모든 플랫폼                                               |



비대칭에 주목하세요. `.img`, `.vrt`, `.asc`, `.jp2`, `.hgt` 는 **변환 도구**에서만 인식되며, 레이어로 직접 드래그해 넣을 수 없습니다. 드래그 앤 드롭은 GeoTIFF에서만 동작합니다.

## 4. 포인트 클라우드 및 LiDAR

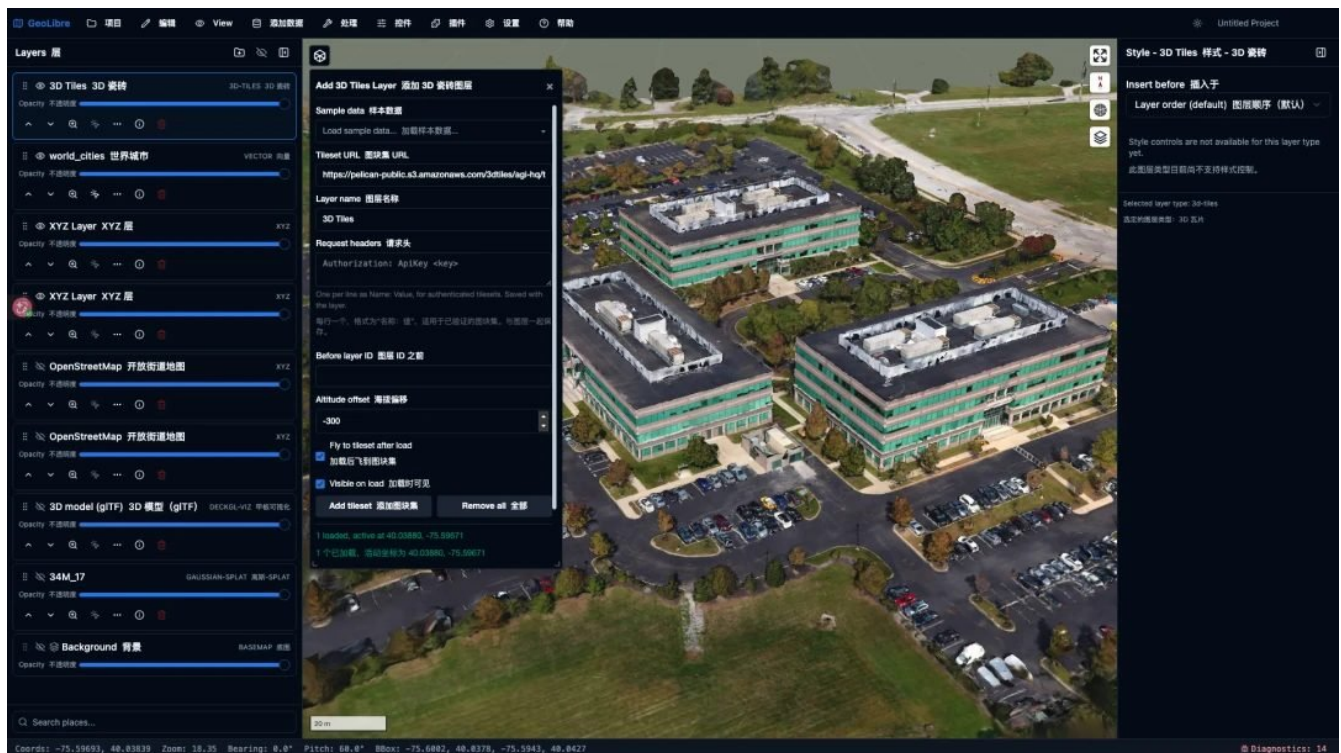
| 항목                       | 지원                                                                                                                                     | 비고                                                                       |
|--------------------------|----------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------|
| <b>LiDAR 레이어</b>         | LAS / LAZ / COPC(URL 또는 로컬 파일), EPT(ept.json URL)                                                                                      | <code>maplibre-gl-lidar</code> + <code>deck.gl</code> 을 통해 렌더링           |
| <b>USGS 3DEP</b>         | 온라인 포인트 클라우드 스트리밍                                                                                                                      | <code>maplibre-gl-usgs-lidar</code> 기반의 독립 플러그인; 3DEP 고도 인덱스 WMS 오버레이 포함 |
| <b>Whitebox LiDAR 도구</b> | <code>.las</code> <code>.laz</code> <code>.zlidar</code> <code>.copc</code> <code>.e57</code> <code>.ply</code> , 출력 <code>.laz</code> | 저장소 전체에서 <code>.e57</code> / <code>.ply</code> 가 등장하는 유일한 곳              |

LiDAR 파일 선택기 자체는 업스트림 `maplibre-gl-lidar` 패키지에 있습니다. GeoLibre의 래퍼(`packages/plugins/src/plugins/components/lidar.ts`)는 LAS/LAZ/COPC 파일과 EPT `ept.json` URL을 문서화하고 있으며, 드롭된 로컬 COPC는 복사 없이 옥트리 노드 단위로 스트리밍됩니다.

## 5. 3D 모델, 타일 및 가우시안 스플랫

서두의 시나리오("3D Tiles를 받았는데 그냥 잠깐 살펴보고 싶다")에 대한 답이 이 절입니다.

| 포맷                                   | 입력 방법                     | 엔진                                            | 비고                                                 |
|--------------------------------------|---------------------------|-----------------------------------------------|----------------------------------------------------|
| OGC 3D Tiles                         | tileset URL               | maplibre-gl-3d-tiles +<br>deck.gl Tile3DLayer | 커스텀 요청 헤더 지원 — 인증된 tileset도<br>동작                  |
| Google<br>Photorealistic 3D<br>Tiles | 내장 URL                    | 위와 동일                                         | Google Maps API 키 필요; 요청 헤더로 전<br>달되며 디스크에 저장되지 않음 |
| ArcGIS I3S Scene<br>Layer            | .../SceneServer<br>URL    | deck.gl + loaders.gl<br>I3SLoader             | 통합 메시와 3D 객체 레이어 모두 지원                             |
| glTF / GLB                           | URL 또는 로컬 파일              | deck.gl ScenegraphLayer                       | deck.gl 시각화 레이어의 scenegraph 타입<br>을 통해 추가          |
| COLLADA .dae                         | KML <Model> 임베<br>딩을 통해서만 | three.js → GLB                                | —                                                  |
| 가우시안 스플랫                             | URL                       | maplibre-gl-splat                             | 저장소 레이어 타입은 gaussian-splat                         |



이 스크린샷은 많은 것을 말해 줍니다. 왼쪽 레이어 패널에서 3D Tiles, 벡터, XYZ, glTF 모델, 가우시안 스플랫이 모두 같은 목록에 쌓여 있고, 렌더링 결과는 바로 오른쪽에 있습니다. "3D Tiles를 로드하려면 서버를 세우고 페이지를 작성해야 한다"가 여기서는 URL 붙여넣기가 됩니다.

없는 것: .obj 는 가져올 수 없습니다(돌출 레이어 3D 모델 내보내기의 출력으로 GLB, STL과 함께 등장할 뿐입니다). .b3dm / .pnts / .cmpt — 이 3D Tiles 내부 포맷들도 저장소에서 검색 결과가 전혀 없습니다. 업스트림 로더가 투명하게 처리하므로 신경 쓸 필요가 없습니다. 가우시안 스플랫 확장자 목록도 마찬가지로 업스트림 패키지에 있습니다.

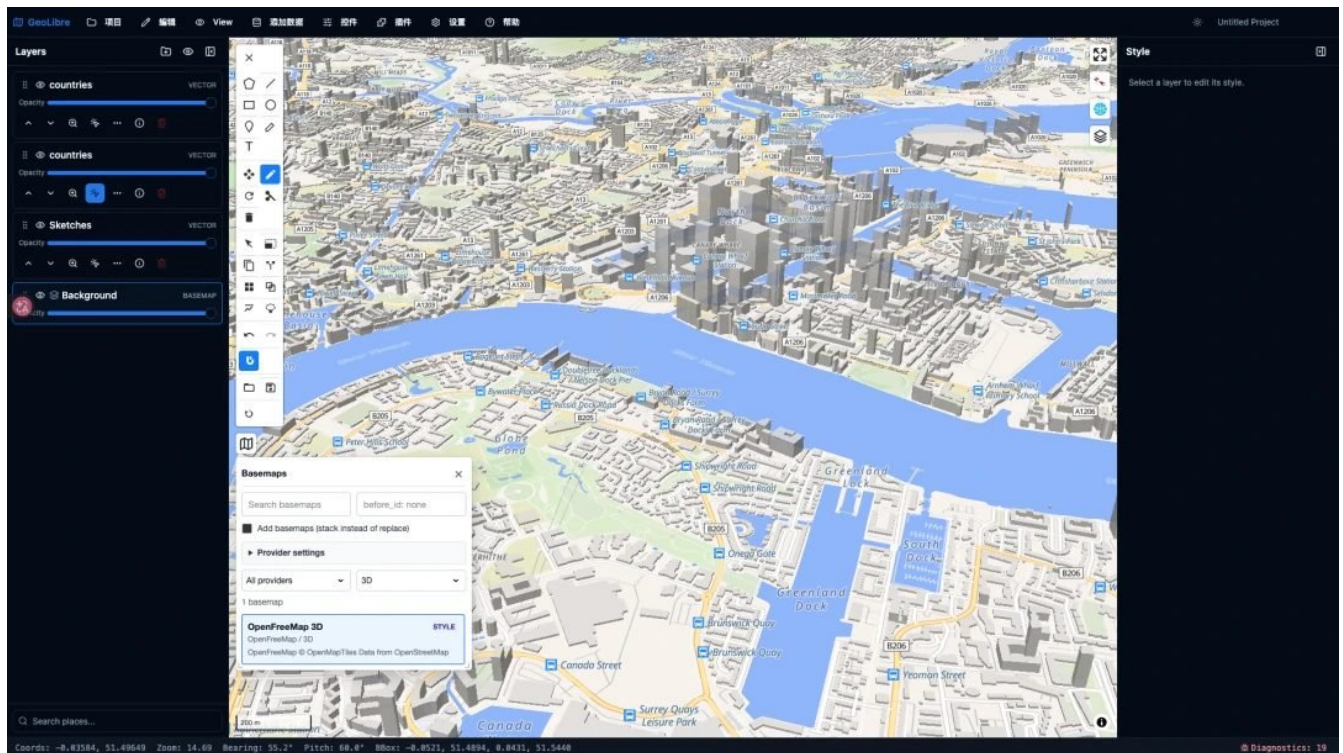
## 6. 타일 및 OGC 서비스

저장소 레이어 타입 열거형에는 20개 항목이 있습니다.

```
geojson, raster, wms, wmts, xyz, vector-tiles, arcgis, pmtiles,
mbtiles, zarr, lidar, gaussian-splat, 3d-tiles, cog, flatgeobuf,
geoparquet, duckdb-query, deckgl-viz, video, image
```

"Add Data → Web Services"(데이터 추가 → 웹 서비스) 아래에 입력할 수 있는 것들:

| 서비스                         | 세부 사항                                                                                                                                |
|-----------------------------|--------------------------------------------------------------------------------------------------------------------------------------|
| <b>XYZ</b>                  | <code>{z}/{x}/{y}</code> 템플릿; 래스터와 벡터 타일 모두 동작                                                                                       |
| <b>WMS</b>                  | <b>GetCapabilities가 레이어 드롭다운을 자동으로 채움</b> ; 피처를 클릭하면 GetFeatureInfo; 개발 모드에는 CORS 프록시 포함                                             |
| <b>WMTS</b>                 | RESTful 타일 템플릿                                                                                                                       |
| <b>WFS</b>                  | GetCapabilities로 typeName 가져오기; 선택적 자동 새로고침                                                                                          |
| <b>OGC API - Features</b>   | 랜딩 페이지 / <code>/collections</code> / 단일 컬렉션 / 전체 <code>/items</code> URL 모두 인식; <code>next</code> 링크 자동 추적; 기본값 1,000 피처             |
| <b>OGC API - Tiles(벡터)</b>  | TileJSON 또는 MVT 템플릿; <code>source-layer</code> 이름을 해석하기 위한 Mapbox 스타일 URL을 선택적으로 제공 가능                                               |
| <b>ArcGIS</b>               | 네 가지 레이어 타입: FeatureServer(GeoJSON으로 페이지 단위 다운로드), VectorTileServer, MapServer, ImageServer(뒤의 둘은 래스터 레이어로); 포털 항목 ID와 ArcGIS 로그인 지원 |
| <b>MBTiles</b>              | <code>.mbtiles</code> 로컬 파일, 커스텀 프로토콜 + Rust 백엔드 읽기. <b>데스크톱 전용</b>                                                                  |
| <b>PMTiles</b>              | <code>.pmtiles</code> ; 벡터와 래스터 모두 동작; 파일 헤더 자동 스니핑                                                                                  |
| <b>PostgreSQL / PostGIS</b> | 연결 → 테이블 선택 → 내장 Martin 서비스를 통해 MVT 출력. <b>데스크톱 전용</b>                                                                               |
| <b>deck.gl 시각화 레이어</b>      | 14가지 타입: scatterplot, heatmap, hexagon, grid, screen grid, contour, arc, line, great circle, GeoJSON, icon, text, trips, scenegraph  |



## 7. 클라우드 네이티브 포맷 및 데이터 카탈로그

이 절은 GeoLibre가 전통적인 데스크톱 GIS를 앞서 나가는 지점입니다.

과학 데이터 포맷:

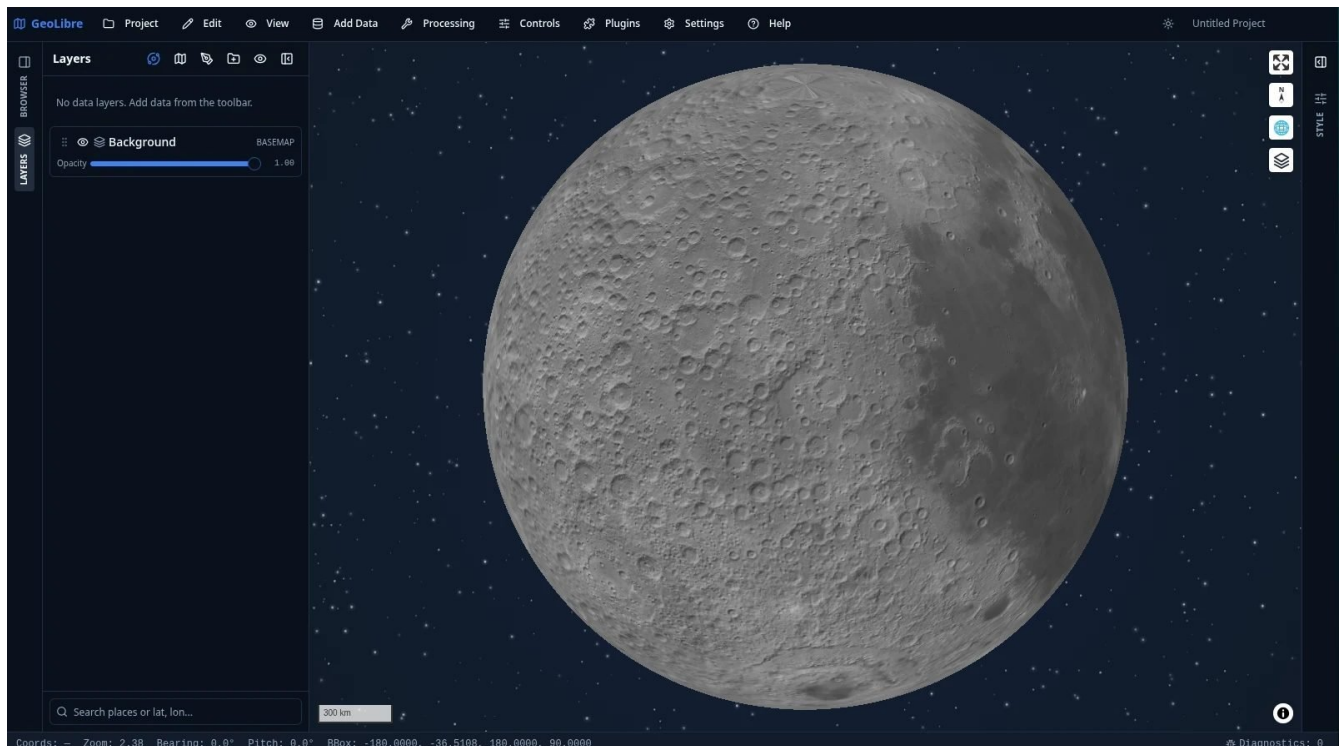
| 포맷                  | 지원 범위                                                                                                                                                                                   |
|---------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Zarr</b>         | 원격 스토어 URL 또는 로컬 폴더. 변수 및 차원 선택기가 실제 좌표 값을 표시. 브라우저의 로컬 폴더는 File System Access API가 필요하며, Firefox / Safari에서는 사용할 수 없음                                                                  |
| <b>NetCDF / HDF</b> | 두 가지 경로: 원격은 <b>kerchunk 참조 JSON + HTTP Range</b> (클라우드 최적화 NetCDF); 로컬은 <b>.nc .nc4 .cdf .h5 .hdf5</b> 지원(NetCDF-3은 netcdfjs, NetCDF-4/HDF5는 h5wasm). <b>HDF4 .hdf</b> 는 명시적으로 지원하지 않음 |
| <b>COG</b>          | 래스터 절 참조                                                                                                                                                                                |

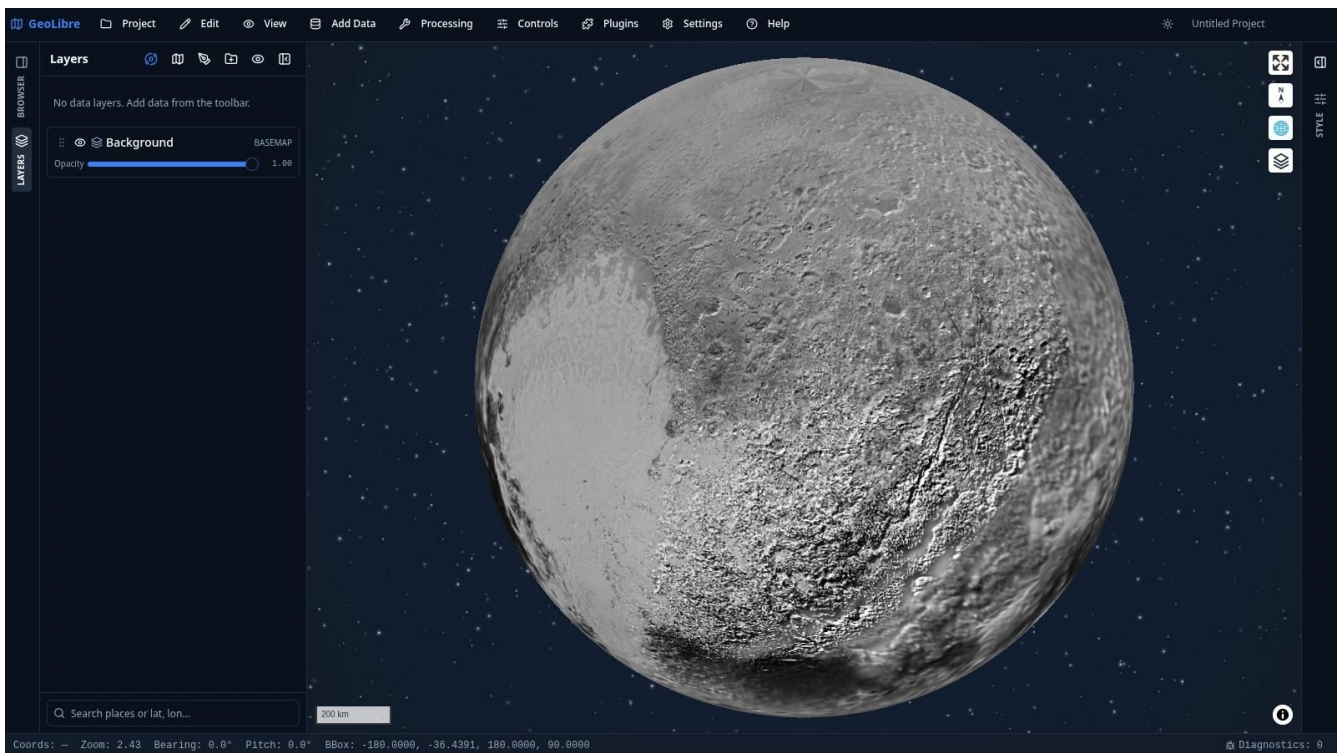
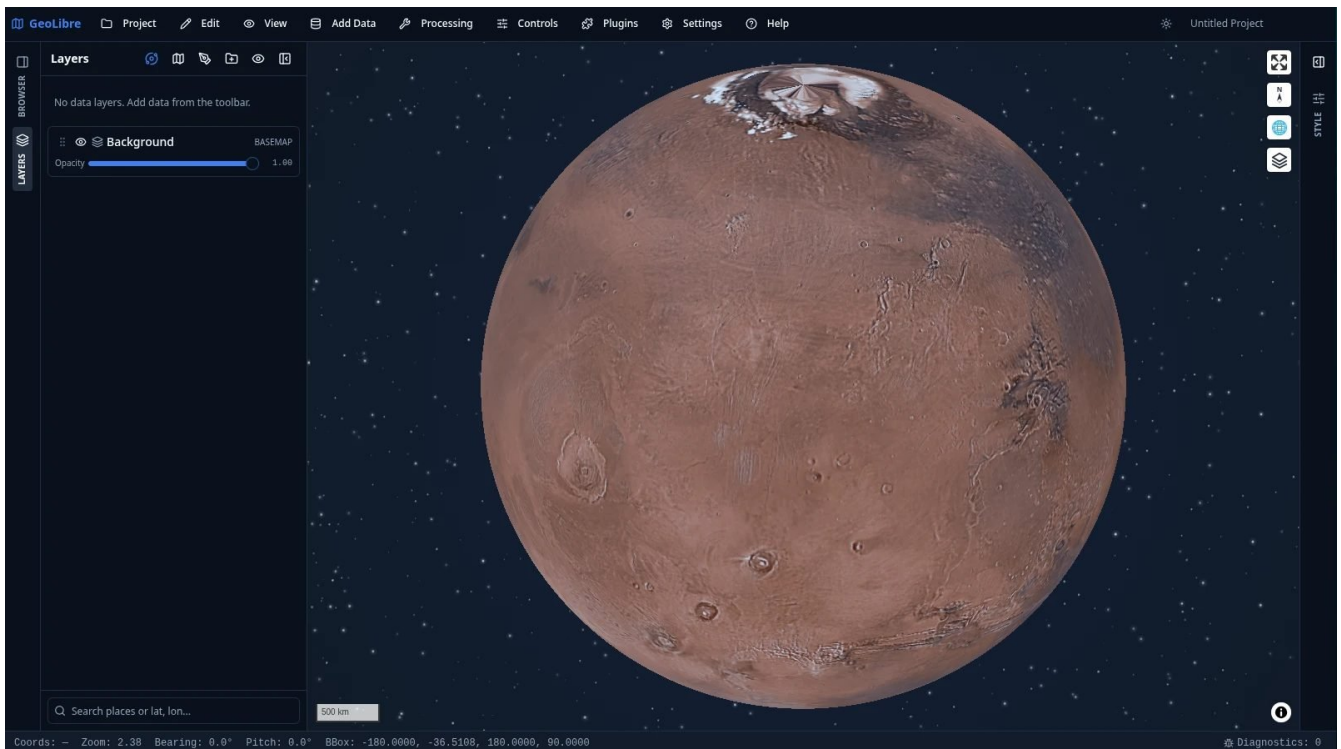
온라인 데이터 카탈로그(직접 탐색 가능):

- **STAC** — 정적 카탈로그와 STAC API 모두 지원; STAC Index를 통해 카탈로그 검색. 시각화 가능한 에셋 (GeoTIFF / GeoJSON)만 직접 추가 가능
- **Source Cooperative** — 버킷 브라우저; **.pmtiles .parquet .geoparquet .tif .geojson .geojsonl .ndjson .fgb .gpkg .csv** 인식
- **Natural Earth** — 본질적으로 **opengeos/natural-earth**에 고정된 Source Coop 패널
- **Hugging Face Hub** — 데이터셋 저장소 탐색; 저장소 생성과 업로드까지 가능
- **Overture Maps** — 주제별 벡터 타일; 기본적으로 건물만 표시

- **Microsoft Planetary Computer** — STAC + TiTiler
- **Google Earth Engine** — OAuth 로그인; 데스크톱에는 네이티브 OAuth 흐름 있음
- **NASA Earthdata GIS** — EOSDIS ImageServer / MapServer / FeatureServer / 발행된 웹 맵
- **OpenAerialMap / Esri Wayback**(과거 영상) / **Mapillary** / **Google Street View**
- **연방 데이터 서비스** — FEMA 홍수 지도, EPA EnviroAtlas, USGS National Map
- **날씨 및 타임랩스** — RainViewer 실시간 레이더, NASA GIBS, EOX 구름 없는 연간 베이스맵

**행성 데이터**(이것은 기대 이상입니다): 달과 화성은 **OpenPlanetaryMap**에서, 수성, 금성, 갈릴레이 위성, 타이탄, 명왕성, 카론은 **USGS Astrogeology**에서 제공됩니다. 핵심 세부 사항은 **프로젝트별 타원체 매개변수**로, 측정하고 있는 천체에 맞게 거리와 면적 측정이 정확합니다.





**편집자 주:** 처음에는 표면적인 기능처럼 보였습니다. 프로젝트별 타원체 매개변수가 그 인상을 바꾸었습니다. 이 프로젝트는 좌표계를 단순한 베이스맵 오버레이가 아니라 진지하게 다룹니다.

## 8. 프로젝트 및 스타일 상호 운용성

간과하기 쉽지만 GeoLibre가 기존 워크플로에 들어맞을 수 있는지를 결정하는 부분입니다.

| 포맷                       | 확장자                                                  | 방향                                                                          |
|--------------------------|------------------------------------------------------|-----------------------------------------------------------------------------|
| GeoLibre 프로젝트            | <code>.geolibre</code> / <code>.geolibre.json</code> | 읽기 전용                                                                       |
| QGIS 프로젝트                | <code>.qgz</code> <code>.qgs</code>                  | 가져오기만. DOMParser로 XML을 파싱하며 QGIS 코드는 실행되지 않음; 인식되는 17가지 벡터 포맷과 GeoTIFF만 가져옴 |
| Mapbox GL / MapLibre 스타일 | <code>.json</code>                                   | 가져오기 + 내보내기                                                                 |
| OGC SLD                  | <code>.sld</code> <code>.xml</code>                  | 가져오기 + 내보내기                                                                 |
| QGIS QML                 | <code>.qml</code>                                    | 가져오기 + 내보내기                                                                 |

**핵심 통찰:** QGIS에서 설정한 심볼로지를 가져올 수 있고, 그 반대도 가능합니다. 스타일 가져오기 포맷은 **확장자가 아니라 내용으로 결정**되므로, 확장자가 잘못된 파일도 인식됩니다.

추가로 `.json` 기반의 다양한 라이브러리가 있습니다. 스타일 라이브러리, 레이어 라이브러리("My Data"), 서비스 라이브러리, 범례, 카메라 플라이스루, 스토리 맵 — 모두 가져오기와 내보내기가 가능합니다.

## 9. 내보내기 및 포맷 변환: 사실상 무료 GDAL 프론트엔드

공식 사이트는 이를 축소해서 다루지만, 가장 실용적인 기능 중 하나입니다.

### 레이어 직접 내보내기

레이어를 마우스 오른쪽 버튼으로 클릭하여 내보냅니다. **모두 브라우저에서 처리되며 백엔드가 필요 없습니다.**

`.geojson` · `.csv` · `.kml` · `.kmz` · `.parquet` (GeoParquet) · `.gpkg` (GeoPackage) · `.zip` (Shapefile)

**GeoPackage와 Shapefile 쓰기는 순수 JS 구현입니다.** DuckDB-WASM이 GeoPackage를 쓸 수 없기 때문에, 작성자가 `sql.js`를 사용해 GeoPackage 1.3 라이터를 직접 작성했습니다. Shapefile의 경우 다섯 개 파일 (`.shp` / `.shx` / `.dbf` / `.prj` / `.cpg`)을 수동으로 조립한 뒤 압축합니다.

### 포맷 변환 도구(9개)

| 도구      | 입력                                                                                                                                                                                                                                      | 출력                                                                                                                    |
|---------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------|
| 벡터 → 벡터 | <code>geojson</code> <code>geojsonl</code> <code>json</code> <code>parquet</code><br><code>geoparquet</code> <code>fgb</code> <code>gpkg</code> <code>shp</code> <code>zip</code> <code>kml</code><br><code>gml</code> <code>gpx</code> | 데스크톱: 14개 드라이버 — GeoJSON, GeoJSONSeq, FlatGeobuf, GPKG, Shapefile, GML, KML, CSV, SQLite, GMT, DXF, MapInfo, JML, GPX |

| 도구                  | 입력                                                      | 출력                                                      |
|---------------------|---------------------------------------------------------|---------------------------------------------------------|
| 벡터 →<br>GeoParquet  | parquet geoparquet geojson json<br>shp gpkg fgb gml kml | .parquet; 압축: zstd / snappy / gzip / lz4 / uncompressed |
| 벡터 →<br>FlatGeobuf  | 위와 동일                                                   | .fgb                                                    |
| 벡터 →<br>Shapefile   | 위와 동일                                                   | .zip                                                    |
| 벡터 →<br>GeoPackage  | 위와 동일                                                   | .gpkg                                                   |
| CSV →<br>GeoParquet | csv tsv txt                                             | .parquet                                                |
| 벡터 →<br>PMTiles     | parquet geoparquet geojson json<br>gpkg fgb shp         | .pmtiles. 데스크톱은 좀 레벨 24까지; 브라우저는 더 얇은 레벨의 WASM 타일러 사용   |
| 래스터 →<br>PMTiles    | .tif/.tiff 만 허용                                         | .pmtiles (단일 밴드를 색상 램프로 처리, 트루 컬러 아님)                   |
| 래스터 →<br>COG        | 데스크톱 tif tiff img vrt asc nc<br>jp2 hgt; 브라우저는 tif만     | .tif; 압축: deflate zstd lzw webp jpeg packbits raw       |

브라우저 측 출력 포맷은 부분 집합입니다: geojson / json / csv / parquet / geoparquet / gpkg / zip / fgb. 14개 드라이버 전체를 사용하려면 데스크톱 버전(Python 사이드카 경유)을 사용하세요.

## 기타 내보내기

래스터 레이어를 GeoTIFF로 내보내기, 경계 상자 래스터 부분 집합 클립, 베이스맵을 PMTiles 오프라인 패키지로 추출, 인쇄 레이아웃을 PNG/PDF/다중 페이지 ZIP으로, **프로젝트 전체를 독립 실행형 HTML 파일로 내보내기**, 스토리 맵을 HTML/PDF로, 지도 화면 녹화를 WebM으로, 차트를 SVG/PNG로.

### 프로젝트를 독립 실행형 HTML로 내보내기

이 기능은 심각하게 과소평가되어 있습니다. 결과를 보여 주기 위해 협업자에게 단일 파일 웹페이지를 공유하는 것이 전문 소프트웨어를 설치해 달라고 요청하는 것보다 훨씬 실용적입니다.

## 10. 플랫폼 차이: 주의할 점

위에서 단편적으로 언급한 내용을 여기에 모았습니다. **여기가 가장 쉽게 문제가 생기는 곳입니다.**

| 제한                        | 영향                                                                                                                                       |
|---------------------------|------------------------------------------------------------------------------------------------------------------------------------------|
| <b>데스크톱 (Tauri) 전용</b>    | 네이티브 파일/폴더 대화 상자, 로컬 MBTiles, 로컬 래스터 읽기, Shapefile 동반 파일 자동 검색, PostGIS/Martin, 파일 지오데이터베이스, 로컬 파일 감시 새로고침                               |
| <b>Python 사이드카 필요</b>     | 파일 지오데이터베이스, 모든 데스크톱 변환 도구(선호 경로), 브라우저 엔진이 없는 래스터 도구(rasterio), AI 세그멘테이션, PostGIS, Sedona                                              |
| <b>Mac App Store 빌드</b>   | Python 사이드카 없음: PostgreSQL과 GDB 데이터 소스 숨김, AI 세그멘테이션 숨김; Whitebox, 변환, 래스터, 벡터 도구는 모두 브라우저/WASM 엔진으로 풀백; Shapefile 동반 파일은 수동으로 다중 선택해야 함 |
| <b>Android / iOS(모바일)</b> | 래스터 도구, 변환 도구, AI 세그멘테이션, PostgreSQL 숨김 — 모두 사이드카 기반. Whitebox 툴박스는 WASM 기반이므로 계속 사용 가능                                                  |
| <b>브라우저</b>               | 로컬 MBTiles/GDB/PostGIS 없음; 변환 출력은 부분 집합; 벡터 변환은 .zip 을 허용하지 않음; 래스터→COG는 GeoTIFF만 허용; Zarr 로컬 폴더는 Firefox/Safari에서 사용 불가                 |

## 11. 아키텍처: 어떻게 이렇게 많은 포맷을 지원할 수 있는가

백 가지가 넘는 포맷을 나열하고 나면 "얼마나 복잡할까"라는 반응이 자연스럽습니다. 이것이 어떻게 하나로 유지되는지 간단히 설명하겠습니다.

**핵심은 엔진에 구매받지 않는 스토어입니다.** @geolibre/core 패키지는 MapLibre에도, Cesium에도, deck.gl에도 의존하지 않습니다. 단 두 가지만 저장합니다. 평면 레이어 레코드의 집합(id, type, source, style, visibility...) 과 다섯 필드의 MapViewState (center / zoom / bearing / pitch / bbox)입니다.

**모든 UI 작업은 스토어만 수정하며, 동기화 계층이 변경 사항을 렌더링 엔진에 밀어 넣습니다. 엄격한 단방향입니다.** UI에서 불투명도 슬라이더를 드래그하면 MapLibre 객체가 아니라 스토어가 수정됩니다.

이 설계의 배당금: **포맷 추가가 저렴합니다.** 새 포맷을 레이어 레코드로 표현할 수만 있다면, 레이어 패널의 모든 기능(불투명도, 순서, 프로젝트 저장, 스타일 내보내기)을 자동으로 얻습니다.

공유 MapEngine 인터페이스 뒤에 있는 **네 가지 렌더링 엔진의 역할 분담:**

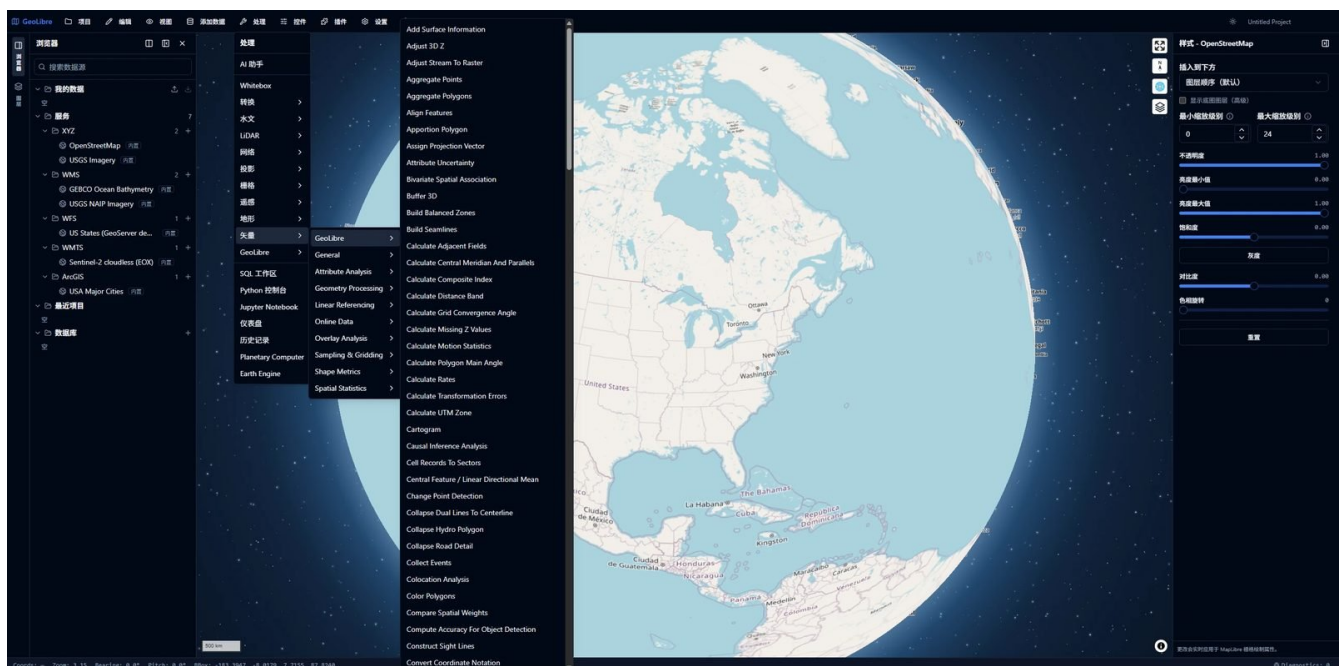
- **MapLibre**는 기본 엔진이며 가장 폭넓은 레이어, 도구, 플러그인 지원을 갖추고 있습니다.
- **Mapbox**는 Mapbox 스타일과 서비스를 위해 Mapbox GL JS를 사용하면서 MapLibre의 Style Specification 표면 대부분을 공유합니다.
- **Cesium**은 지형, 3D Tiles, CZML, ion 에셋, I3S, 지구본 중심 워크플로를 위한 네이티브 3D 지구본 엔진입니다.
- **ArcGIS**는 네이티브 ArcGIS 서비스와 2D 또는 3D Esri 뷰를 위해 ArcGIS Maps SDK를 사용합니다.

**deck.gl은 다섯 번째 엔진이 아닙니다.** COG, 포인트 클라우드, 3D, 시각화 레이어를 위해 호환되는 엔진이 호스팅하는 오버레이입니다. 같은 지도 위의 인터리브된 생산자들은 하나의 오버레이 인스턴스를 공유해야 하며, 그렇지 않으면 나중 인스턴스가 이전 인스턴스가 소유한 레이어를 대체할 수 있습니다.



주목할 만한 세부 사항: **Cesium의 카메라 동기화는 줌 레벨이 아니라 지상 해상도(미터/픽셀)로 정렬됩니다.** 그래서 서로 다른 높이의 분할 화면 패널이 같은 화면상 축척을 유지합니다.

다섯 가지 연산 엔진이 모두 같은 UI 아래에 있습니다. DuckDB-WASM Spatial(주력, 전용 Worker에서 실행), PGlite + PostGIS, Apache Sedona(사이드카 또는 브라우저 WASM 버전), Pyodide(브라우저에서 GeoPandas/Shapely 실행), Whitebox WASM(1,000개 이상의 도구).



영리한 접근: 벡터 연산자의 Python 구현은 "프레임워크에 의존하지 않는" 모듈이며, 사이드카와 브라우저 Pyodide가 정확히 같은 코드를 실행합니다. 따라서 두 경로의 결과가 완전히 일치하며, "로컬 연산이 서버 연산과 다르다"는 불일치가 없습니다.

## 12. 성능: 몇 가지 실제 수치

빠른 개요입니다. 모두 소스에서 검증 가능한 수치이며 추정치가 아닙니다.

**대용량 벡터 처리 방식.** 50,000 피처 이상에서는 MapLibre의 네이티브 GeoJSON 소스 사용을 멈추고 대신 클라이언트 측에서 즉석으로 타일링합니다. Web Worker에서 geojson-vt가 타일을 생성하고(포인트 레이어는 Supercluster로 집계), vt-pbf가 MVT로 인코딩한 뒤, 커스텀 프로토콜을 통해 MapLibre에 공급합니다. 최대 줌 16, 4096 extent.

엔지니어링의 세심함을 보여 주는 두 가지 세부 사항: 타일 인덱스 객체는 **의도적으로 스토어에서 제외됩니다**(너무 크고, 직렬화할 수 없으며, 프로젝트 파일에 기록할 수 없음). 인코딩은 진행하기 전에 중단 신호를 확인합니다. MapLibre가 화면 밖으로 스크롤된 타일 요청을 취소하기 때문입니다.

**기타 보호 임계값**(이 수치들 자체가 좋은 참고점입니다):

| 임계값                 | 값                                     |
|---------------------|---------------------------------------|
| 타일링 렌더 트리거          | 50,000 피처                             |
| DuckDB 결과 구체화 확인 요청 | 500,000 행                             |
| 브라우저 측 Sedona 상한    | 50,000 피처                             |
| OSM PBF 경고 / 타임아웃   | 50 MB / 5분                            |
| 원격 벡터 파일 상한         | 2 GiB(DuckDB-WASM이 원격 파일 크기에 32비트 사용) |
| 로컬 COG 상한           | 2 GiB                                 |
| 실행 취소 기록 소프트 예산     | 500,000 피처                            |

**속성 테이블은 가상화되어 있지만**, 정렬, 필터링, 선택은 전체 데이터 모델에 대해 동작합니다. 가상화는 렌더링만 관장합니다.

**실행 취소 기록에는 메모리 상한이 있습니다.** 모든 스냅샷이 레이어의 전체 GeoJSON을 보유하기 때문에, 반복 편집은 여러 복사본을 메모리에 고정시킵니다. 접근 방식: 피처 수 소프트 예산을 설정하고, 초과하면 가장 오래된 스냅샷부터 버리되 **항상 최신 스냅샷은 유지**하여 큰 편집 후에도 최소 한 단계의 실행 취소를 보장합니다. 슬라이더 드래그는 400 ms 안에서 하나의 실행 취소 기록으로 병합됩니다.

**번들 크기는 CDN 오프로딩으로 제어됩니다.** 세 가지 무거운 WASM 엔진(PGLite+PostGIS 약 25 MB, CereusDB 약 40 MB, gdal3.js 약 28 MB + 12 MB)은 번들되지 않습니다. 런타임에 정확한 버전으로 CDN에서 가져오며, 처음 사용한 뒤에는 Service Worker가 오프라인 사용을 위해 캐시합니다. 소스 주석의 두 가지 역사적 기록: **PGLite를 번들하면 바이너리가 42 MB에서 63 MB로 부풀고, CereusDB는 설치 파일을 27 MB에서 36 MB로 늘렸을 것입니다.** 30 MB 설치 파일이라는 수치는 그렇게 달성되었습니다.

**솔직한 성능 문제 하나.** 문서에는 Linux/WebKitGTK 조사에 대한 절이 통째로 있습니다. 빈 지도는 어떤 줌에서든 안정적인 60 FPS이지만, 타일 레이어가 하나라도 있으면 **타일 로딩 중 FPS가 한 자릿수로 떨어지고**, 로딩이 멈추면 즉시 60으로 복귀합니다. 근본 원인: WebKitGTK가 새 타일마다 메인 스레드에서 GPU 업로드(래스터는 텍스처, 벡터는 정점 버퍼)를 수행하며, **단일 타일 통합 사이클이 Chromium의 몇 ms에 비해 약 125 ms로 측정됩니다.** 작성자는 소프트웨어 렌더링, GPU 포화, Tauri IPC, JSON.parse, 컴포지터 지연 등 수많은 가능성을 배제했습니다. 완화책(타일 캐시 증가, 512px 래스터 타일, 페이드인 비활성화)은 문서화되어 있지만 **아직 구현되지 않았습니다.** macOS/Windows WebView는 테스트되지 않았습니다.

**편집자 주:** 이런 부정적 결과를 아키텍처 문서에 기꺼이 기록하는 프로젝트는 드뭅니다. 이 관행은 인정받을 만합니다.

## 13. 그 경계

강점을 다루었으니 이제 한계를 이야기하겠습니다.

### QGIS의 대체재가 아닙니다

이는 여러 리뷰에 걸친 공통된 의견이며, 작성자도 그렇게 주장한 적이 없습니다.

- 1. 기능 범위가 의도적으로 좁습니다.** 브라우저 워크플로, 로컬 처리, 클라우드 네이티브 포맷, 공간 SQL, 현대적 시각화, 이식성에 집중합니다. 복잡한 전문 워크플로는 여전히 QGIS의 영역입니다.
- 2. 이터레이션 속도는 양날의 검입니다.** 원문이 작성될 당시, 두 달 남짓 만에 0에서 2.4.0까지 갔고, 2.0.0과 2.1.0 사이의 간격은 약 19시간이었습니다. 장기 프로덕션 의존성으로 고려하고 있다면 이 변동 위험을 감안하세요.
- 3. 몇 가지 명확한 포맷 공백.** `.obj` 는 가져올 수 없고, HDF4는 지원하지 않으며, 원시 `.osm` XML도 지원하지 않습니다.
- 4. 플랫폼 기능 비대칭.** 10절의 표를 참조하세요. 브라우저 버전의 경험을 전체로 외삽하지 마세요.
- 5. 웹 버전 밖에서는 Cesium 지형과 영상에 ion 토큰이 필요합니다.** 호스팅된 웹 버전에는 데모 토큰이 번들되어 있지만, 데스크톱과 모바일 앱에는 자신의 토큰이 필요합니다. 토큰이 없어도 지구본은 동작하지만 Cesium World Terrain과 Ion 영상은 없습니다. 무료 티어는 개인 사용에 충분하며, 팀은 할당량과 가격을 예산에 반영해야 합니다.
- 6. 중국 특유의 환경.** 베이스맵, 지형, Photorealistic 3D Tiles의 기본 소스는 모두 방화벽 밖에 있고, 좌표는 표준 WGS84를 사용합니다. **GCJ-02 오프셋은 별도로 처리해야 합니다.** 진지하게 사용하려면 이 두 가지 문제를 먼저 해결해야 합니다. 이에 대한 신뢰할 수 있는 1차 데이터는 없으며, 실제 사용자의 추가 의견을 환영합니다.

## 맺으며

shp, tif, gpkg, tileset.json이 뒤섞인 zip 아카이브를 받았습니다. 예전에는 서버 가지 다른 애플리케이션이 필요했습니다. 이제 답은 이렇습니다. **30 MB짜리 애플리케이션(또는 그냥 웹페이지)을 열고, 전부 드래그해 넣으세요.**

GeoLibre의 진짜 가치는 QGIS보다 낫다는 것이 아니라(그렇지 않습니다), "**데이터를 잠깐 살펴보기**"의 비용을 거의 **0으로** 끌어내렸다는 것입니다. 이 틈새는 이전에는 비어 있었습니다.

다중 엔진 또는 3D GIS 개발을 하는 분들에게는 또 다른 층의 참고 가치가 있습니다. **엔진에 구매받지 않는 스토어 설계**입니다. 상태를 특정 엔진의 객체에 묶지 않고 평범한 레이어 레코드와 뷰 상태로 저장하면, 프로젝트 모델을 바꾸지 않고 다른 렌더러를 끼워 넣을 수 있습니다. 이 접근 방식은 채택할 가치가 있습니다.

시나리오별 사용 경로:

1. **그냥 살펴보고 싶다** — `web.geolibre.app` 을 바로 여세요. 설치도 등록도 필요 없습니다
2. **실제 작업을 해야 한다** — GitHub Releases에서 데스크톱 버전을 다운로드하거나 Microsoft Store / Homebrew / winget으로 설치하세요. 2분이면 됩니다
3. **Python 사용자** — `pip install geolibre`; GeoPandas 지원이 필요하다면 `pip install "geolibre[all]"`; Python 3.11 이상 필요
4. **인트라넷 / 오프라인 환경** — `VITE_PYODIDE_INDEX_URL` 과 `VITE_DUCKDB_SPATIAL_EXTENSION_PATH` 로 Pyodide와 DuckDB spatial 확장을 내부 미러로 향하게 할 수 있으며, **재빌드가 필요 없습니다**; 공식 Docker 이미지도 `ghcr.io/opengeos/geolibre:latest` 에서 제공됩니다
5. **2차 개발** — npm workspaces 모노레포이므로 **Node 22+**에서 **npm**을 사용하세요(저장소가 `package-lock.json` 을 추적합니다); 주 애플리케이션은 `apps/geolibre-desktop` 에 있습니다; MIT 라이선스

마지막 정리: 데이터 미리보기와 탐색에는 강력히 추천합니다. 프로덕션 배포는 조금 더 기다리세요. 적어도 버전 이터레이션 주기가 안정될 때까지는요.

<https://geolibre.app/tutorials/source-code-deep-dive-en/>

# GeoLibre 소스 코드 심층 분석: WebGIS 프로젝트에서 재사용할 만한 핵심 기법

**초록:** 이 글은 GeoLibre 소스 코드 아키텍처를 심층적으로 분석하며, 의존성 선택, DuckDB-WASM Spatial 핵심 엔진, 웹 측 성능 최적화, 상태 관리, 오프라인 PWA 캐싱, 그리고 클라우드 네이티브 지리공간 데이터 솔루션을 다룹니다. 개발자가 참고하고 재사용할 수 있도록 모든 소스 파일 경로를 명시했습니다.

**원문 출처:** "GIS开发手记", 원문 링크: <https://mp.weixin.qq.com/s/482yiH-VsKP7OmBw9UWG6g>

## 머리말

이 글은 GeoLibre 저장소의 소스 코드를 한 층씩 살펴보고, 의존성 선택, DuckDB-WASM 공간 연산 커널, 프런트엔드 성능 최적화 전략, 상태 관리 설계, 3단계 오프라인 캐싱 전략, 그리고 클라우드 네이티브 지리공간 데이터 솔루션을 다룹니다. 해당하는 모든 소스 파일 경로를 명시했습니다. 여기서 설명하는 모든 기술적 접근 방식은 여러분의 WebGIS 프로젝트에 바로 적용할 수 있습니다.

## 1. 도구 상자: 하나씩 살펴볼 가치가 있는 오픈 소스 라이브러리

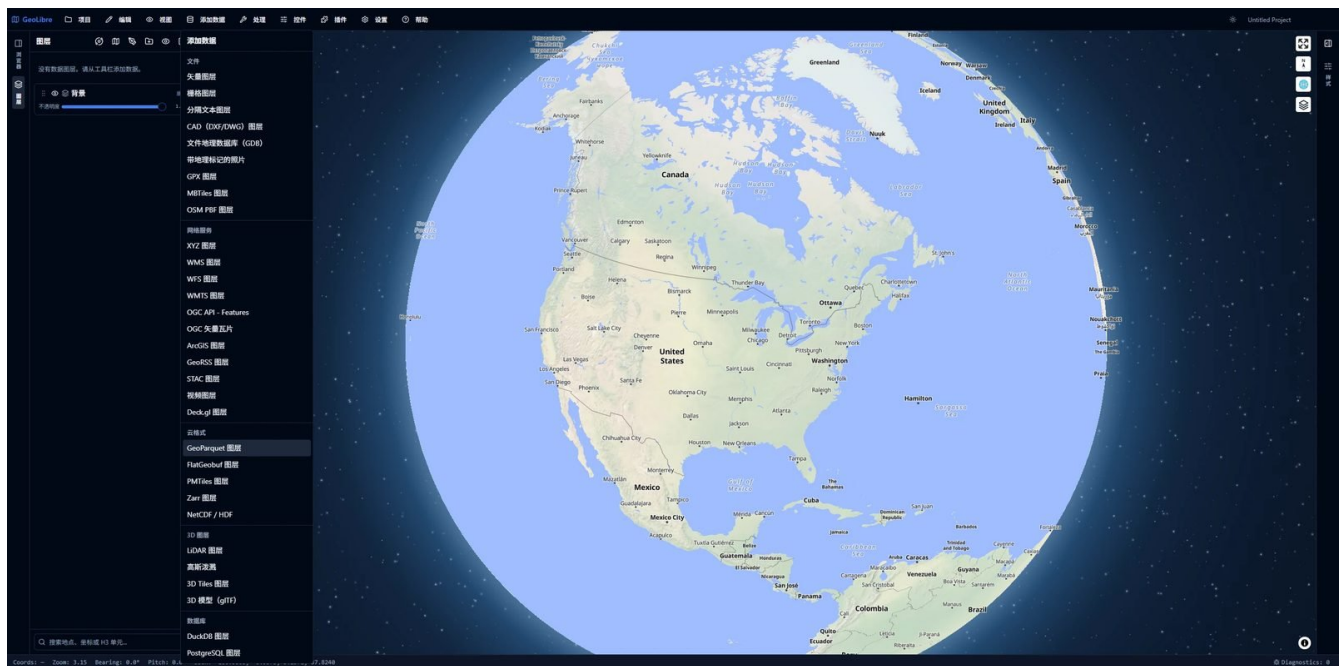
이 저장소는 7개의 패키지와 데스크톱 애플리케이션으로 구성된 npm workspaces 모노레포입니다. 아래에서 의존성을 용도별로 분류했으며, 주목하라고 표시한 것들은 집중적으로 살펴볼 가치가 있는 라이브러리입니다.

### 1.1 포맷 파싱: 경량 우선, 중화기는 폴백으로

| 라이브러리               | 버전     | 역할                                                | 왜 이것인가                                                            |
|---------------------|--------|---------------------------------------------------|-------------------------------------------------------------------|
| @duckdb/duckdb-wasm | 1.33.1 | GeoParquet, FlatGeobuf, GML, DXF, TAB, 그리고 공간 SQL | <b>포맷 드라이버와 연산 엔진</b> 역할을 동시에 하는 단일 라이브러리                         |
| shpjs               | 6.2    | Shapefile                                         | 순수 JS, 수십 KB; .prj 로 투영, .cpg 로 인코딩 처리( <b>중국어 속성 인코딩 문제 해결</b> ) |
| fflate              | 0.8    | zip / kmz / aprx 압축 해제                            | 매우 작고 빠름                                                          |
| sql.js              | 1.14   | GeoPackage 읽기 및 쓰기                                | <b>GPKG는 본질적으로 SQLite</b> — 가볍고 효율적                               |
| exifr               | 7.1    | 사진 EXIF GPS → 포인트 레이어                             | UAV/드론 시나리오에 실용적                                                  |
| gdal3.js            | 2.8    | 지리참조된 GeoTIFF/COG 내보내기에만 사용                       | WASM 약 28MB + 데이터 약 12MB, CDN에서만 로드하며 번들에 포함하지 않음                 |
| geotiff             | 3.0    | GeoTIFF 디코딩                                       | —                                                                 |

| 라이브러리                            | 버전 | 역할                 | 왜 이것인가                  |
|----------------------------------|----|--------------------|-------------------------|
| h5wasm / netcdfjs                | —  | HDF5 / NetCDF-3    | 명확한 역할 분담, 각각 하나의 포맷 담당 |
| @osmix/pbf @osmix/core           | —  | OSM PBF            | Web Worker에서 실행         |
| pmtiles proj4<br>fast-xml-parser | —  | 타일 아카이브 / 투영 / XML | 기반 유틸리티                 |

여기서의 핵심은 "포맷마다 가장 가벼운 경로를 선택한다"는 것입니다. 소스 파일 `packages/plugins/package.json`에는 60개가 넘는 의존성이 있지만, 거대한 "통합 읽기 계층"은 존재하지 않습니다. 각 포맷은 저마다 가장 짧은 경로를 택합니다.



KML 사례가 특히 시사적입니다. `docs/architecture.md`의 DuckDB-WASM 절에 명확히 적혀 있습니다. KML은 **임베디드 스타일을 보존하기 위해 직접 만든 파서**로 처리하며, `simplestyle-spec` 속성(`fill`, `stroke`, `stroke-width`)을 출력하여 스타일이 적용된 KML이 Google Earth에서와 똑같이 GeoLibre에서도 보이도록 합니다. 커스텀 파서가 실패할 때만 DuckDB Spatial로 폴백하며, **그 대가로 스타일 정보를 잃습니다**.

**핵심 통찰:** 범용 라이브러리는 데이터 모델을 통일하는 과정에서 포맷 고유의 정보를 필연적으로 버립니다. 그리고 그 정보야말로 사용자가 가장 중요하게 여기는 것인 경우가 많습니다.

Shapefile에서도 같은 패턴이 반복됩니다. **먼저 shpjs를 시도하고, 실패할 때만 DuckDB Spatial로 폴백합니다. 빠른 경로와 느리지만 포괄적인 폴백의 조합은 포맷 파싱의 보편적 패턴입니다.**

## 1.2 공간 연산: 네 단계의 엔진, 기본값은 가장 가벼운 것

이 부분은 오해하기 쉽습니다. 소스를 추적해 보면(`docs/architecture.md`의 Python 사이드카 절에 따르면) 다음과 같습니다.

| 엔진                                        | 실행 위치            | 포지셔닝                                 |
|-------------------------------------------|------------------|--------------------------------------|
| <b>Turf.js</b> ( @turf/* , 약 25개 하위 패키지)  | 브라우저, 순수 JS      | <b>벡터 도구의 기본 엔진</b> , 의존성 없음, 백엔드 없음 |
| <b>GeoPandas / Shapely</b>                | Python 사이드카      | <b>투영을 고려한</b> 결과가 필요할 때의 업그레이드 경로   |
| <b>GeoPandas / Shapely</b>                | Pyodide를 통한 브라우저 | <b>동일한 코드베이스</b> , 웹 버전에서도 사용 가능     |
| <b>DuckDB / PGlite+PostGIS / SedonaDB</b> | 브라우저 또는 사이드카     | SQL Workspace를 위한 세 가지 엔진            |

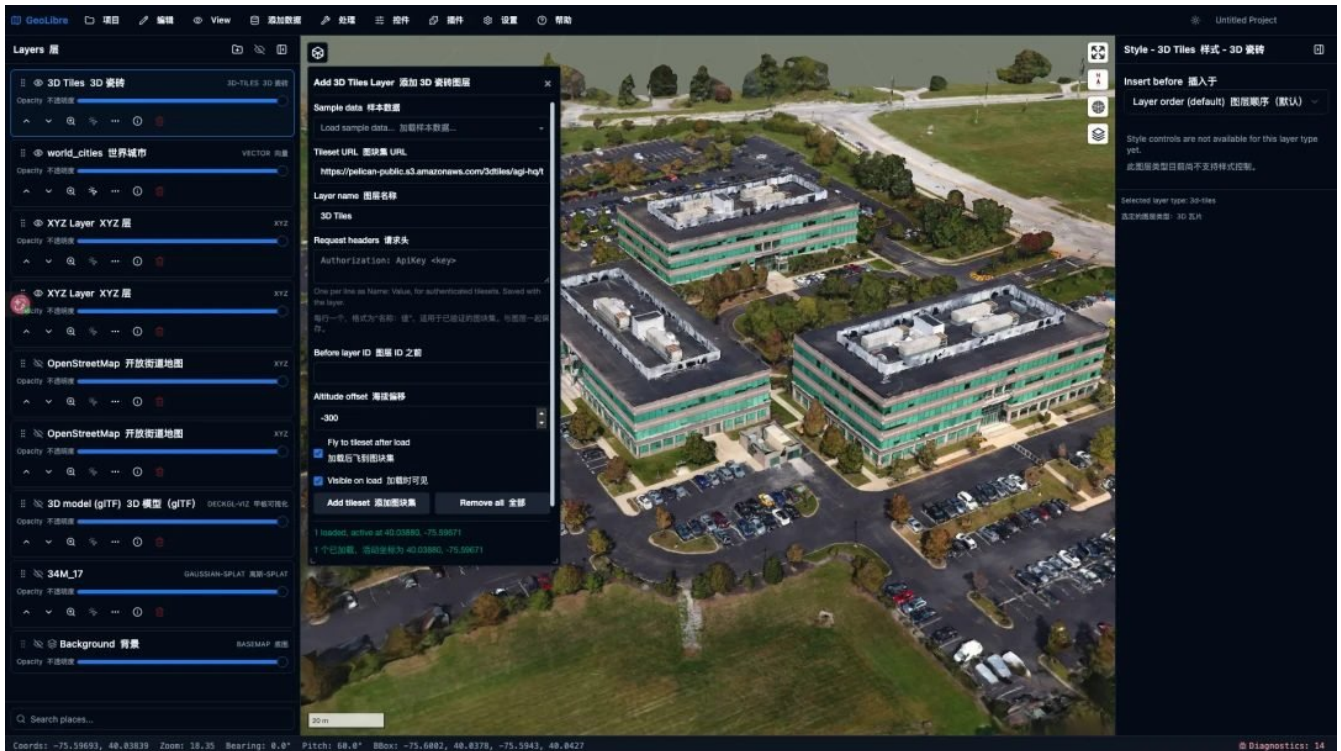
Turf는 전체가 아니라 **하위 패키지 단위**( @turf/buffer , @turf/intersect 등)로 임포트된다는 점에 주목하세요. 이는 중요합니다. Turf 전체를 한 번에 임포트하면 용량이 상당하지만, 패키지별 임포트 덕분에 실용적으로 사용할 수 있습니다.

"하나의 코드베이스, 두 개의 런타임" 설계는 같은 절에 문서화되어 있습니다. 지오메트리 로직은 **프레임워크에 의존하지 않는 모듈** backend/geolibre\_server/geolibre\_server/vector\_ops.py 에 있습니다. Vite 플러그인 ( vite-plugins/copy-vector-ops.ts )이 이를 프론트엔드 패키지로 복사하고, 브라우저 측은 클래식 Web Worker를 사용하여 Pyodide를 로드하고 geopandas 를 설치한 뒤 JSON 문자열 경계를 넘어 run\_vector\_tool 을 호출합니다.

**현장의 교훈:** GIS를 다뤄본 사람이라면 누구나 이 함정을 압니다. 프론트엔드에서 Turf.js로 버퍼를 계산하고 백엔드에서 PostGIS로 같은 버퍼를 계산하면 면적이 0.3% 다릅니다. 이를 뒤에야 두 구현의 기본 세그먼트 수가 다르기 때문이라는 걸 발견합니다.

### 1.3 렌더링 및 레이어: MapLibre 플러그인 생태계

| 라이브러리                                                                      | 역할                                                               |
|----------------------------------------------------------------------------|------------------------------------------------------------------|
| maplibre-gl 6.11                                                           | 주 지도                                                             |
| deck.gl 9.4 (core/layers/geo-layers/mesh-layers/aggregation-layers/mapbox) | COG, 3D Tiles, I3S, 시각화 레이어를 MapLibre 캔버스에 인터리브                  |
| maplibre-gl-3d-tiles / -lidar / -splat / -raster / -vector                 | <b>엔진을 바꾸지 않고 3D Tiles, 포인트 클라우드, 가우시안 스포를 MapLibre 위에 직접 추가</b> |
| @developmentseed/deck.gl-geotiff / -raster                                 | COG 렌더링                                                          |
| @carbonplan/zarr-layer                                                     | Zarr 과학 데이터                                                      |
| @loaders.gl/i3s / @esri/maplibre-arcgis                                    | Esri 생태계 통합                                                      |
| @geoman-io/maplibre-geoman-free                                            | 그리기 및 편집                                                         |
| maplibre-gl-time-slider / -swipe / -layer-control / -basemap-control       | 인터랙티브 컨트롤                                                        |
| @tanstack/react-virtual                                                    | 속성 테이블 가상화                                                       |
| cesium 1.146                                                               | 선택적 3D 지구본 분할 뷰, <b>별도 청크로 약 4.7 MB 지연 로드</b>                    |



많은 개발자가 `maplibre-gl-*` 계열에 대해 이 점을 모릅니다. 3D Tiles, COPC 포인트 클라우드, 가우시안 스플랫을 모두 MapLibre 캔버스 안에서 직접 로드할 수 있으며, Cesium이 필요 없습니다. 프로젝트에서 "3D Tiles를 잠깐 살펴보기"만 필요하다면, 이 경로가 3D 엔진 전체를 끌어오는 것보다 훨씬 가법합니다.

## 핵심 정리

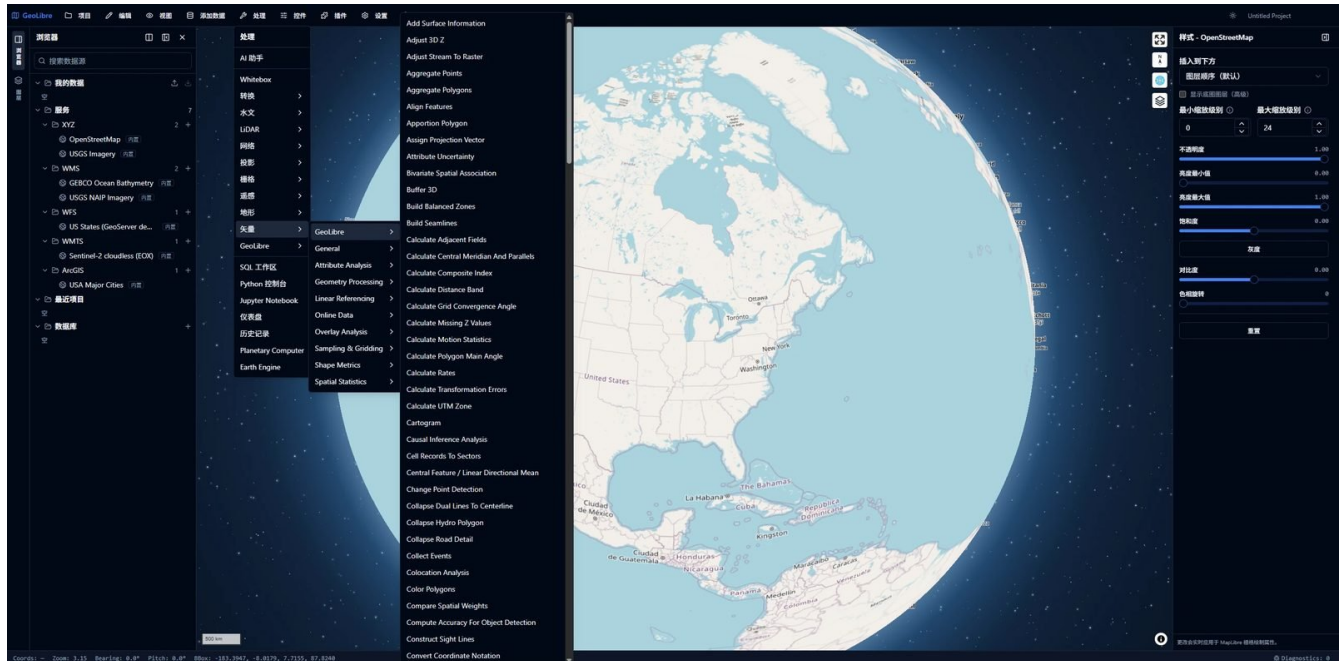
- 1. 포맷마다 가장 가벼운 경로를 선택하세요. 프론트엔드 라이브러리 간의 크기 차이는 자릿수 단위입니다 (shpjs 수십 KB 대 gdal3.js 40 MB). 라이브러리 크기의 비용은 사용자가 첫 화면 로딩 시간으로 지불합니다.
- 2. 빠른 경로 + 포괄적인 폴백: 정상 데이터의 90%는 경량 파싱으로 처리하고, 10%의 예외 상황은 무거운 엔진으로 폴백합니다.
- 3. 범용성보다 충실도. KML 스타일을 보존하기 위해 커스텀 파서를 만드는 것은 그만한 가치가 있습니다.
- 4. Turf는 하위 패키지 단위로 임포트하고, 절대 전체를 임포트하지 마세요.

## 2. DuckDB-WASM Spatial: 포맷 드라이버와 연산 엔진을 겸하는 하나의 라이브러리

이 절은 이 글의 핵심입니다. 단 하나만 읽어야 한다면 이 절을 읽으세요.

먼저 WebAssembly를 한 문장으로 정리하겠습니다. GeoLibre 소스에서 `wasm`이라는 식별자는 천 번이 훨씬 넘게 등장합니다. 데이터베이스, 언어 런타임, 네이티브 틀체인, 코덱, 머신러닝 — 이 다섯 가지 기능 범주 모두가 WASM 엔진으로 구동됩니다. DuckDB, sql.js(SQLite), PGLite + PostGIS, CereusDB(SedonaDB), Pyodide, geolibre-wasm (WASI로 컴파일된 Whitebox 엔진과 GeoLibre 자체 Rust 도구), gdal3.js, cog-tiler-wasm,

h5wasm, onnxruntime-web이 그것입니다. 네이티브 언어를 추적해 보면 그림이 분명해집니다. C, C++, Rust. 여기서 WASM은 JavaScript를 가속하기 위한 것이 아니라, 수십 년간 축적된 네이티브 GIS 생태계를 통째로 브라우저에 들여오기 위한 것입니다.



이 열 개 남짓한 엔진 중에서 지금 당장 시도해 볼 가치가 있는 것이 하나 있습니다. 바로 @duckdb/duckdb-wasm 과 그 spatial 확장입니다. 이유는 간단합니다. 다른 엔진들은 "특정 범주의 작업"을 해결하지만, 이것은 포맷 드라이버이자 연산 엔진입니다. 하나의 라이브러리가 "데이터 읽기"와 "데이터 연산"을 모두 맡을 수 있습니다.

## 2.1 먼저 세 계층의 관계를 명확히 하기

이 세 이름은 자주 혼동됩니다. 이들은 중첩 관계입니다.

| 계층                | 무엇인가                                    | 관계                                                    |
|-------------------|-----------------------------------------|-------------------------------------------------------|
| WebAssembly       | 브라우저의 바이너리 명령어 포맷                       | 단지 "네이티브 코드를 실행하는 능력"일 뿐, 그 자체로는 GIS와 관련된 일을 전혀 하지 않음 |
| DuckDB-WASM       | WASM으로 컴파일된 DuckDB(C++로 작성된 분석용 데이터베이스) | "WASM으로 구현된 DuckDB"이지, WASM 그 자체가 아님                  |
| Spatial Extension | DuckDB의 spatial 확장, 역시 별도의 .wasm 파일     | 별도의 INSTALL / LOAD 가 필요하며, 없으면 공간 함수가 전혀 없음           |

**가장 흔한 오해:** duckdb-wasm을 설치하면 공간 기능이 생긴다고 생각하는 것입니다. 그렇지 않습니다.

ST\_Read, ST\_Transform, ST\_AsWKB — 이들은 모두 spatial 확장에 있으며, 이 확장은 CDN에서 가져와 런타임에 로드되는 두 번째 WASM 산출물입니다.

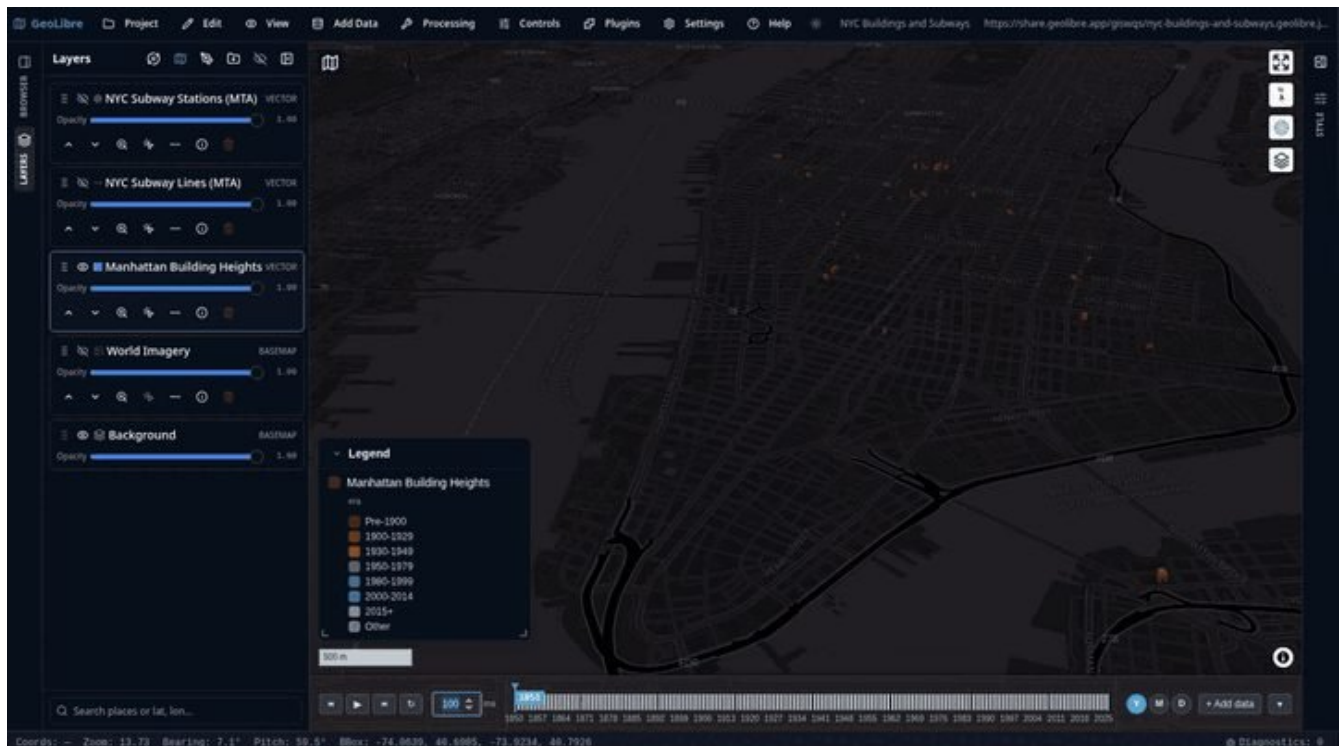
그리고 spatial 확장의 `ST_Read` 뒤에는 **GDAL의 부분 집합**이 있습니다. 즉, 브라우저에서 SQL 문 하나만 실행하면 그 부분 집합이 다루는 벡터 포맷을 읽을 수 있다는 뜻입니다. 이는 머신에 설치된 GDAL이 아니라 확장에 자체 번들된 GDAL이므로, 정확한 목록은 로드된 빌드가 포함하는 그대로입니다. `SELECT * FROM ST_Drivers()` 를 실행하면 확인할 수 있습니다.

## 2.2 GeoLibre는 이것을 어디에 사용하는가

소스 코드의 DuckDB 호출 지점은 다음과 같이 분류됩니다.

| 사용 사례                          | 사용 인터페이스                                 | 비고                                                  |
|--------------------------------|------------------------------------------|-----------------------------------------------------|
| GeoParquet                     | <code>read_parquet</code>                | 로컬과 원격 모두 이 경로를 통과하며, 원격은 HTTP Range 요청 사용          |
| FlatGeobuf / GML / DXF / TAB 등 | <code>ST_Read</code>                     | GDAL 백엔드; 포맷 범위는 확장이 로드하는 것에 따라 달라짐                 |
| Shapefile (zip)                | 먼저 <code>shpjs</code> , 실패 시 Spatial로 폴백 | 경량 우선, 중화기는 폴백으로                                    |
| KML                            | 먼저 커스텀 파서(스타일 보존), 실패 시 Spatial로 폴백      | Spatial은 지오메트리만 얻으며 스타일은 손실됨                        |
| WKT 지오메트리 열이 있는 CSV            | DuckDB SQL                               | 텍스트 지오메트리를 바로 레이어로 변환                               |
| 좌표계 변환                         | <code>ST_Transform</code>                | 레거시 최상위 <code>crs</code> 멤버가 있는 GeoJSON은 이를 통해 재투영됨 |
| SQL Workspace                  | 전체 DuckDB SQL                            | 로드된 레이어가 테이블로 등록되어 바로 JOIN 가능                       |

**핵심 통찰:** 마지막 행에 주목하세요. 이는 과소평가되기 쉬운 설계 선택입니다. 지도 위의 레이어가 동시에 SQL의 테이블입니다. 사용자는 두 레이어에 걸쳐 `JOIN` 과 `ST_Intersects` 를 작성할 수 있고, 그 결과는 곧바로 새 레이어가 됩니다. "지도"와 "데이터베이스"는 같은 것을 바라보는 두 가지 뷰입니다.



SQL Workspace에는 "사용자가 의도한 바를 말하도록 돕는" 세 가지 재작성 기능도 있습니다. SQL 콘솔을 만들고 있다면 이 접근 방식을 그대로 따를 수 있습니다.

- FROM / JOIN 뒤의 맨 URL이나 로컬 경로가 그대로 동작합니다. 소스 코드가 파일 확장자를 기준으로 적절한 리더( read\_parquet , read\_csv\_auto , ST\_Read 등)로 자동으로 감싸 줍니다. 사용자는 어떤 포맷이 어떤 함수에 대응하는지 외울 필요가 없습니다.
- s3:// , gs:// , az:// 는 해당하는 공개 HTTPS 엔드포인트로 변환되어, 같은 HTTP Range 채널을 통해 전송됩니다.
- 리더 매개변수의 HTTP URL은 DuckDB 파일 핸들로 재작성되며, WASM 내장 httpfs를 거치지 않고 JS 측에서 Range 요청을 발행합니다. 후자는 많은 서버에서 아예 실패하기 때문입니다.

**구현 세부 사항:** 이 세 가지 재작성은 실제 리더 호출 인자에만 적용됩니다. 문자열 리터럴과 주석 안에 있는 URL처럼 보이는 문자열은 건너뛰니다(소스 코드는 완전한 SQL 리터럴 마스킹 패스를 구현합니다). **SQL을 재작성할 때 절대 날것의 정규식을 사용하지 마세요.**

## 2.3 빌려 쓸 만한 네 가지 구현 세부 사항

첫째, 브라우저 기능에 따라 번들을 선택합니다. duckdb-wasm은 기능과 크기가 다른 여러 WASM 빌드(mvp, eh 등)를 제공합니다. duckdb-wasm-bundles.ts 는 딱 한 가지 일만 합니다. Vite의 ?url 로 얻은 mvp와 eh 산출물을 duckdb.selectBundle() 에 넘겨 런타임에 선택하게 하는 것입니다.

```
const MANUAL_BUNDLES: duckdb.DuckDBBundles = {
  mvp: { mainModule: duckdbWasmMvp, mainWorker: mvpWorker },
  eh: { mainModule: duckdbWasmEh, mainWorker: ehWorker },
};
export async function selectDuckDbBundle() {
```

```
// Imported here so DuckDB-WASM's JS stays off the startup path
const { selectBundle } = await import("@duckdb/duckdb-wasm");
return selectBundle(MANUAL_BUNDLES);
}
```

**기본 CDN 해석 대신 번들을 수동으로 나열하면 Vite가 WASM과 워커에 대해 콘텐츠 해시가 붙은 로컬 산출물을 생성하게 됩니다.** 이는 다시 Service Worker의 CacheFirst 전략에 의한 안전한 캐싱을 가능하게 합니다(5절 참조). (npm run lite:build 변형은 duckdb-wasm-bundles.cdn.ts 로 교체하여, 파일별 크기 상한이 있는 호스트를 위해 같은 고정 버전을 jsDelivr에서 로드합니다.)

**둘째, 확장 로딩은 "인스턴스별로 한 번만 실행"되어야 합니다.** INSTALL spatial 은 네트워크를 거치고 LOAD 는 상태를 가지므로, 동시 호출은 충돌합니다. GeoLibre의 접근 방식은 살펴볼 가치가 있습니다.

```
// Memoized per-instance: the shared library and the SQL Workspace
// each have their own DB, and each tracks its own loading state
const spatialExtensionByDb = new WeakMap<duckdb.AsyncDuckDB, Promise<void>>();

export async function ensureSpatialExtension(db, connection, beforeLoad?) {
  let promise = spatialExtensionByDb.get(db);
  if (!promise) {
    promise = (async () => { /* ...INSTALL / LOAD... */ })();
    spatialExtensionByDb.set(db, promise);
  }
  try {
    await promise;
  } catch (error) {
    // Only clear if the memoized entry still points to this failed attempt,
    // otherwise retries would be permanently impossible
    if (spatialExtensionByDb.get(db) === promise) spatialExtensionByDb.delete(db);
    throw error;
  }
}
```

이 짧은 코드에는 세 가지 설계 결정이 담겨 있습니다. 불리언이 아니라 Promise를 캐시합니다. 동시 호출이 같은 로딩 작업을 공유합니다. 모듈 수준 싱글톤이 아니라 인스턴스를 키로 하는 WeakMap 을 사용합니다. 인스턴스가 재구성되면 캐시가 자동으로 무효화됩니다. 실패 시 메모이저된 항목을 지우되, 동일성을 확인한 뒤에만 지웁니다. 다른 누군가가 방금 기록한 새 로딩 시도를 지워 버리는 일을 막습니다. **이 셋 중 하나라도 빠뜨리면 프로덕션에서 "공간 함수가 간헐적으로 존재하지 않음" 문제를 만나게 됩니다.**

**셋째, 네트워크 접근 없이 확장을 로드할 수 있는 탈출구를 남겨 둡니다.** INSTALL spatial 은 기본적으로 DuckDB 의 확장 저장소에서 다운로드합니다. GeoLibre는 환경 변수 VITE\_DUCKDB\_SPATIAL\_EXTENSION\_PATH 를 추가했습니다. 이 값이 설정되면 INSTALL 을 완전히 건너뛰고 바로 LOAD '<로컬 경로>' 를 실행합니다.

인트라넷, 정부 기관, 오프라인 데스크톱 배포에서는 이 설정 하나가 동작하느냐 마느냐를 가릅니다. 같은 패턴이 h3 커뮤니티 확장( INSTALL h3 FROM community )에도 재사용됩니다. DuckDB의 확장 생태계를 브라우저에서 사용할 수 있다는 사실을 많은 개발자가 모릅니다.

넷째, **Worker와 수명 주기를 직접 관리합니다.** DuckDB는 전용 Worker에서 실행됩니다. **WASM 힙은 Worker의 수명에 묶여 있으며, `terminate()` 만이 이를 시스템에 진정으로 반환합니다.** GeoLibre는 SQL Workspace 인스턴스에 대해 참조 카운팅을 구현합니다. 쿼리가 아직 실행 중이면 인스턴스를 "유휴 상태가 되면 파괴"로 표시하고, 마지막 쿼리가 해제될 때까지 기다린 뒤 종료합니다. **인스턴스를 재생성해야 하는 시나리오에서 이는 필수적입니다.** 바로 `terminate()` 를 호출하면 진행 중인 쿼리가 죽어 버립니다.

## 2.4 어떤 튜토리얼보다 값진 실제 버그

소스 코드 주석에서 발견한 주목할 만한 기록이 하나 있는데, "브라우저에서 데이터베이스를 실행한다는 것이 어떤 것인지"를 위의 모든 내용보다 잘 보여 줍니다.

duckdb-wasm 1.33.1-dev45는 인스턴스가 첫 원격 읽기를 완료하기 전에 `LOAD spatial` 을 실행하면 해당 인스턴스의 원격 `read_parquet` 을 **영구적으로 망가뜨립니다.** 이후 모든 원격 Parquet 읽기가 `stoi: no conversion` 으로 실패하며, 이는 **제자리에서 복구할 수 없습니다.**

GeoLibre는 두 갈래 접근으로 이에 대응합니다.

**첫째, 워밍업.** `ensureSpatialExtension` 의 `beforeLoad` 훅이 바로 이를 위한 것입니다. `LOAD spatial` 전에, 이 쿼리가 사용하는 원격 리더에 대해 `SELECT 1 FROM read_parquet(...) LIMIT 0` 을 실행합니다. `LIMIT 0` 은 **Parquet 푸터만 가져오므로 추가 트래픽이 거의 없습니다.** 이 쿼리에 원격 Parquet이 없으면 작은 공개 샘플 파일을 읽는 것으로 풀백합니다.

**둘째, 재구성.** 그래도 버그가 발생하면(예: 이전에 워밍업이 실패한 경우) `runSqlQuery` 가 `stoi: no conversion` 을 잡아 `resetSqlDatabase(db)` 를 호출해 인스턴스 전체를 교체한 뒤 **한 번 재시도** 합니다. 재구성은 워밍업을 다시 거치므로 두 번째 시도는 깨끗합니다.

두 가지 추가 방어 세부 사항이 있습니다. 재시도는 문장이 **실제로 원격 리더 호출을 포함할 때만** 트리거되며(문자열 리터럴 안에 나타나는 URL은 포함되지 않음), `resetSqlDatabase` 는 행동하기 전에 "교체할 인스턴스가 여전히 현재 인스턴스인지" 확인합니다.

### 전이 가능한 세 가지 교훈

- **WASM 확장 로딩 순서에는 부작용이 있습니다.** 특히 네트워크 하위 시스템과 관련된 확장의 경우 "언제 LOAD하든 상관없다"고 가정하지 마세요.
- **"워밍업"과 "재구성"을 땀질이 아닌 일급 시민으로 다루세요.** 되돌릴 수 없는 불량 상태에서 우아하게 복구하려면 인스턴스가 교체 가능하도록 설계되어 있어야만 합니다.
- **업스트림 버그에 대한 탈출구를 남기되, 범위를 증상으로 좁히세요.** 특정 오류 메시지 + 특정 조건 + 한 번만 재시도로 매칭하세요. 무턱대고 모두 잡아서 재시도하지 마세요.

## 2.5 그 경계는 어디인가

많은 글이 DuckDB-WASM을 찬양합니다. 이 절은 소스 코드에서 드러난 실제 한계를 특별히 살펴봅니다. 이러한 제약을 이해하면 눈먼 채로 운영하는 것보다 더 큰 확신을 가질 수 있습니다.

| 한계          | 값            | 이유                                           |
|-------------|--------------|----------------------------------------------|
| 원격 파일 크기    | <b>2 GiB</b> | DuckDB-WASM의 HTTP 파일 시스템이 원격 파일 크기를 32비트로 저장 |
| 단일 탭 메모리    | 약 4 GiB      | WASM 주소 공간 상한                                |
| 피처 수 확인 임계값 | 500,000      | 이 이상을 구체화하기 전에 사용자가 확인해야 함                   |
| 스레딩         | 단일 스레드       | 매우 큰 데이터는 서버로 돌아가야 함                         |

2 GiB 제한에는 교훈적인 세부 사항이 있습니다. 가드 검사는 **수집이 시작되기 전에** 일어납니다(`_registerSource`가 스트리밍 분기보다 먼저 실행되므로 "스트리밍 읽기"도 이를 우회할 수 없습니다). GeoLibre의 접근 방식은 사용자가 Add를 클릭하고 피할 수 없는 실패를 기다리게 하는 대신, 파일 브라우저 패널에서 이를 명확하게 알려 주는 것입니다. **미리 판단할 수 있는 실패를 런타임으로 미뤄서는 안 됩니다.**

**복사 대 스트리밍**의 트레이드오프도 있습니다. GeoParquet만이 "복사 없이 제자리에서 쿼리"를 진정으로 지원합니다. 다른 포맷은 스트리밍 옵션이 제공되어도 조용히 전체 복사로 폴백합니다. 그래서 GeoLibre는 GeoParquet에 대해서만 이 버튼을 보여 줍니다. **클릭해도 아무 차이가 없는 버튼은 버튼이 없는 것보다 나쁩니다.** 복사 모드에서 메모리 사용량은 대략 **압축 해제된** 데이터 크기를 따르며, 이는 Parquet 디스크 크기의 몇 배가 될 수 있습니다. 100 MB를 넘으면 UI가 사용자에게 스트리밍으로 전환하라고 안내합니다.

마지막으로 교훈적인 비교가 하나 있습니다. GeoLibre의 데스크톱 빌드는 네이티브 DuckDB를 대신 사용하는 `native-duckdb` 컴파일 타임 스위치를 노출합니다. 그러나 Rust 측에는 엄격한 제약이 있습니다.

```
compile_error!("the `mas` (Mac App Store) build must not enable `native-duckdb`: \
  DuckDB loads its spatial extension as unsigned native code at runtime, \
  which App Sandbox and App Store guideline 2.5.2 forbid.");
```

네이티브 버전이 더 빠르지만, 서명되지 않은 네이티브 확장을 런타임에 로드하는 것은 Mac App Store 규칙을 직접 위반합니다. 반대로 **WASM 버전은 샌드박스 안에서 실행되기 때문에 "어디서나 배포되는" 쪽이 됩니다.** 이는 기술 선택에서 좀처럼 고려되지 않는 요소입니다.

## 2.6 핵심 정리

- **1. 먼저 DuckDB-WASM Spatial을 시도하고, 그다음에 대안을 고려하세요.** "포맷 읽기"와 "공간 연산"을 모두 해결하는 하나의 라이브러리는 오늘날 웹 측 GIS에서 ROI가 가장 높은 단계입니다. 원격 GeoParquet + `read_parquet` — 코드 열두어 줄이면 되고 백엔드는 필요 없습니다.
- **2. 확장 로딩은 "인스턴스별 메모이저된 Promise + 실패 시 동일성 검증 후 캐시 삭제"로 구현하세요.** 이 패턴은 DuckDB뿐 아니라 모든 "비동기 일회성 초기화" 시나리오에 적용됩니다.
- **3. 항상 로컬 확장 경로 탈출구를 제공하세요.** 인트라넷 / 오프라인 / 데스크톱 — 이 설정 하나가 전부입니다.
- **4. 모든 WASM 엔진에 대해: 지연 로드 + 별도 청크 + Worker 실행 + 고정 버전 + 교체 가능한 수명 주기.** 메인 스레드와 핵심 렌더링 경로에서 떨어뜨려 두세요.

- **5. 엄격한 한계를 이룰 있는 상수로 명문화하고 미리 검증하세요.** 2 GiB, 50만 피처, 약 4 GiB 힙 — 사용자에게 미리 알릴 수 있는 실패를 런타임으로 미뤄서는 안 됩니다.

### 3. 웹 측 성능: 다섯 가지 최적화, 그리고 솔직한 부정적 결과 하나

이 절의 모든 수치는 특정 파일의 줄 번호까지 추적할 수 있습니다.

#### 3.1 임계값 주도: 명확한 수치를 정하고, 경계에서 구현을 전환

핵심 아이디어를 한 문장으로 정리하면 이렇습니다. 모든 것을 지배하는 하나의 방식을 추구하지 말고, 잘 정의된 임계값에서 구현을 전환하세요.

소스의 임계값 상수(모두 검증 가능):

| 상수                                            | 값       | 위치                                          | 보호 대상             |
|-----------------------------------------------|---------|---------------------------------------------|-------------------|
| <code>LARGE_VECTOR_FEATURE_THRESHOLD</code>   | 50,000  | <code>core/src/types.ts:783</code>          | 메인 스레드 GeoJSON 파싱 |
| <code>maxHistoryFeatureCount</code>           | 500,000 | <code>core/src/history.ts:29</code>         | 실행 취소 스택 메모리      |
| <code>DUCKDB_VECTOR_FEATURE_WARN_COUNT</code> | 100,000 | <code>core/src/types.ts:2540</code>         | 결과 구체화 메모리        |
| <code>MAX_CEREUS_FEATURES</code>              | 50,000  | <code>lib/sedona-workspace.ts:25</code>     | WASM 힙            |
| <code>MAX_DERIVED_FEATURES</code>             | 50,000  | <code>map/src/derived-geometry.ts:42</code> | 파생 지오메트리 연산       |
| <code>historyCoalesceMs</code>                | 400 ms  | <code>core/src/history.ts:6</code>          | 실행 취소 기록 폭증       |
| 원격 파일                                         | 2 GiB   | <code>plugins/remote-file-formats.ts</code> | DuckDB-WASM 32비트  |

이 임계값들은 모두 이룰 있는 상수이고, 모두 문서화 주석이 있으며, `historyCoalesceMs` 와 `maxHistoryFeatureCount` 는 런타임 조정을 위한 세터를 갖추고 있다는 점에 주목하세요(테스트에서는 0으로 설정). 이는 코드 곳곳에 흩어진 매직 넘버보다 한 단계 위입니다.

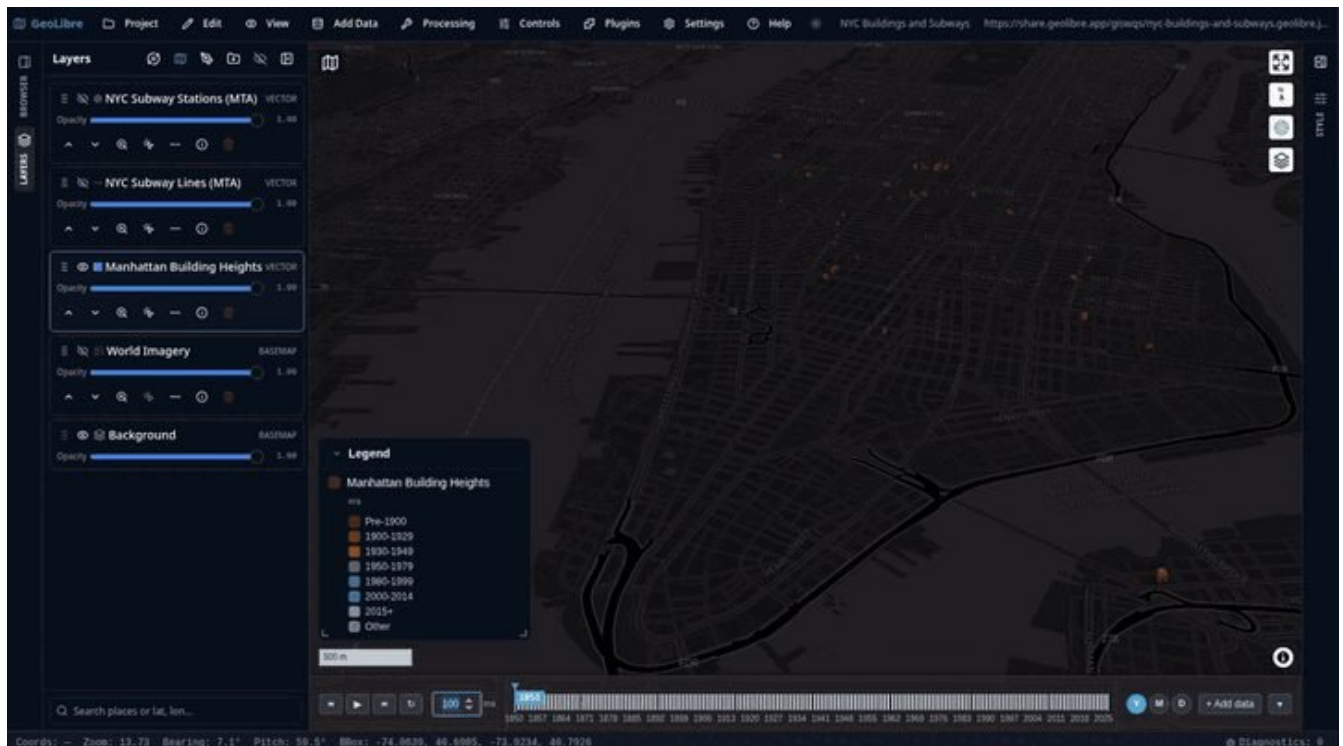
**정리:** 모든 임계값은 "무엇을 보호하는지"를 말할 수 있어야 합니다. 보호 목적을 정당화할 수 없는 임계값은 신뢰할 수 있는 근거가 없으며, 장기 유지보수에서 살아남기 어렵습니다.

#### 3.2 클라이언트 측 타일링: 5만 피처 이상에서 즉석 타일 생성

파이프라인은 `packages/map/src/geojson-vt-protocol.ts` (프로토콜 및 레지스트리)와 `geojson-vt-index.ts` (인덱스 구축 및 인코딩)에 있으며, 단계별로 다음과 같습니다.

- 인덱싱에는 `@maplibre/geojson-vt` 를 사용합니다(참고: 이는 MapLibre의 포크이며, **Supercluster가 이 패키지에 포함되어 있습니다**. 주석에는 "MapLibre가 내부적으로 사용하는 것과 같은 엔진"이라고 적혀 있습니다)

- 포인트 레이어는 Supercluster 로 인덱싱하고, 나머지는 모두 GeoJSONVT 를 사용합니다
- 인코딩에는 @maplibre/vt-pbf 의 fromGeojsonVt 를 사용합니다
- 커스텀 프로토콜 geolibre-gjvt 를 통해 MapLibre에 공급합니다
- TILE\_EXTENT = 4096 , TILE\_MAX\_ZOOM = 16 (이를 넘어서면 MapLibre가 오버Zoom하도록 돕니다)
- 인덱스 구축과 타일 인코딩은 Web Worker( geojson-vt.worker.ts )에서 실행되므로, 큰 데이터를 로드해도 더 이상 메인 스레드가 멈추지 않습니다. Worker를 사용할 수 없는 곳에서는 같은 코드가 인라인으로 실행됩니다



따로 짚어 볼 만한 세부 사항 두 가지가 있습니다.

**첫째, 레이어별 타일 레지스트리는 스토어가 아니라 모듈 수준 Map에 있습니다.** 소스 주석은 이렇게 설명합니다.

레이어 id를 키로 합니다. Zustand 레코드가 아닌 모듈 수준에 두는 이유는 **타일 인덱스가 크고 직렬화할 수 없는 객체**여서 앱 상태에 들어가거나 `.geolibre.json` 에 기록되어서는 안 되기 때문입니다.

**이는 훌륭한 휴리스틱을 제공합니다. 전역 상태에 무엇이 속하는가? — 프로젝트 파일에 기록할 수 있는 것만.** 파생 산출물(타일 인덱스, 공간 인덱스, WebGL 버퍼, 디코딩된 비트맵)은 항상 모듈 수준 캐시나 엔진 측에 두어야 합니다.

**둘째, 인코딩 전에 중단 신호를 확인합니다.** 한 줄입니다.

```
// packages/map/src/geojson-vt-protocol.ts
if (signal?.aborted) return empty;
```

MapLibre는 화면 밖으로 스크롤된 타일 요청을 취소하며, 그 결과는 어차피 버려질 것이므로 연산을 낭비하지 않습니다.

**일반화하면: 모든 "주문형 생산" 비동기 파이프라인은 취소를 지원해야 합니다.** 실제 프로젝트에 적용하면 이렇습니다. 사용자가 지도를 빠르게 드래그하는 동안 화면 밖 타일을 재투영하는 것, 속성 테이블이 빠르게 페이지를 넘기는 동안 이전 페이지의 쿼리가 여전히 실행 중인 것, 레이어를 빠르게 전환하는 동안 이전 레이어의 파싱 결과가 늦게 도착해 새 레이어를 덮어쓰는 것(**전형적인 경쟁 조건**).

`AbortController` 는 `fetch` 만을 위한 것이 아닙니다

한 프레임 이상에 걸치는 모든 루프는 루프 본문에서 `signal.aborted` 를 확인하고 일찍 빠져나와야 합니다.

### 3.3 실행 취소 스택: 살펴볼 만한 세 가지 개선

`packages/core/src/history.ts` 파일은 직접 열어 볼 가치가 있습니다. 200줄이 안 되지만 밀도가 높습니다. `zundo` (Zustand의 시간 여행 미들웨어)를 사용합니다.

문제: **모든 스냅샷이 레이어의 전체 GeoJSON을 보유합니다.** 반복 편집은 여러 복사본을 메모리에 고정시킵니다 (주석은 이슈 #341을 참조합니다).

**처방 1: 피쳐 수로 측정하는 소프트 예산.** 주석은 왜 바이트가 아닌지 설명합니다. "피쳐 수는 페이로드 크기의 저렴한 대리 지표이며, 편집할 때마다 지오메트리를 직렬화하는 일을 피합니다" — **메모리를 측정하려고 지오메트리를 직렬화하는 것 자체가 성능 문제입니다.**

**처방 2: 항상 최신 스냅샷을 유지:**

```
// trimHistoryBySize: walk from newest to oldest, stop when the accumulated budget is exceeded
// The newest snapshot is always retained, even if it alone exceeds the budget.
let total = distinctFeatureCount(pastStates[lastIndex], seen);
```

"메모리가 부족하면 실행 취소 스택을 비운다"는 것이 많은 프로젝트의 접근 방식이지만, 그것은 사용자가 실행 취소를 가장 필요로 하는 바로 그 순간(큰 편집 직후)에 실행 취소가 정확히 하나도 없다는 뜻입니다.

**처방 3: 객체 참조로 중복 제거.** `distinctFeatureCount` 는 `Set` 을 사용해 이미 본 페이로드를 추적합니다. 변경되지 않은 레이어는 스냅샷 간에 같은 참조를 공유하며 한 번만 세어집니다. **따라서 "작은 레이어의 스냅샷을 많이 유지"하는 데는 추가 메모리 비용이 거의 없습니다.**

간과하기 쉬운 세부 사항이 하나 더 있습니다. 그 400 ms는 평범한 디바운스가 아니라 `zundo`의 `handleSet` 역할을 하는 **리딩 에지 디바운스**입니다.

리딩 에지에서만 발화하여 폭주 이전 상태를 한 번 기록합니다

이 구분은 결정적입니다. **기록해야 하는 것은 "이번 연속 작업 폭주 이전의 상태"**이므로, 상황이 안정된 뒤가 아니라 첫 트리거 시점에 기록해야 합니다. **슬라이더를 드래그하기 시작하기 전의 값이 바로 되돌리고 싶은 값입니다.** 평범한 트레일링 디바운스를 사용하면 드래그 도중의 어떤 중간값을 잡게 됩니다.

### 3.4 가상화는 렌더링만 관장한다

속성 테이블은 가상화를 위해 `@tanstack/react-virtual` 을 사용하지만, 정렬, 필터링, 선택은 전체 데이터 모델에 대해 동작합니다.

소박하게 들리지만 이는 고빈도 버그입니다. 많은 가상 목록 구현이 "렌더링된 행만 정렬"하여 스크롤 위치에 따라 정렬 결과가 바뀌거나, "모두 선택"이 보이는 행만 선택하는 문제를 일으킵니다. 그리고 이런 버그는 소규모 데이터 셋 테스트에서는 재현이 불가능합니다.

### 3.5 번들 크기: 지연 로딩은 어디까지 가는가?

`apps/geolibre-desktop/vite.config.ts` 의 코드 분할 설정(`manualChunks` 와 `Rollup`의 `CODE_SPLITTING_GROUPS`)이 이 접근 방식의 핵심입니다. 가장 대표적인 예인 Cesium을 보겠습니다. 소스 주석에는 Cesium 청크가 한때 공유 헬퍼를 삼켰을 때 "약 4.7 MB의 Cesium 청크 전체가 부팅할 때마다 `modulepreload`되었다"고 기록되어 있습니다. 지금은 "Cesium은 패널이 3D 지구본으로 전환될 때만 로드된다"는 이유로 자체 그룹에 들어 있습니다.

부팅 경로에서 떨어뜨려 두는 것이 핵심입니다. 웹 GIS에서 이는 가장 쉬운 성능 개선입니다. 실제로 3D가 필요한 사용자는 20%에 불과할 수 있습니다. 100%의 사용자가 그것 때문에 첫 화면 비용을 지불할 이유가 없습니다.

소스 주석에서 가져온 상세 크기 분석을 여기에 정리했습니다.

| 무거운 리소스            | 크기                                          | 전략                                              |
|--------------------|---------------------------------------------|-------------------------------------------------|
| CesiumJS           | 약 4.7 MB                                    | 별도 청크, 지구본 뷰로 전환할 때만 <code>import()</code>      |
| PGlite + PostGIS   | 약 25 MB(데스크톱에 번들하면 거의 압축되지 않는 약 22 MB가 추가됨) | 기본적으로 jsDelivr CDN을 통해 제공, 빌드에 포함되지 않음          |
| gdal3.js           | WASM 약 28 MB + 데이터 약 12 MB                  | 번들하지 않으며, 항상 CDN에서 로드; CDN을 비활성화하면 해당 기능이 비활성화됨 |
| Pyodide / CereusDB | 각각 수십 MB                                    | CDN 로드; PGlite와 CereusDB는 빌드 스위치로 번들에 포함 가능     |

그러니 "설치 파일이 30 MB에 불과하다"는 수치는 "무거운 엔진이 거의 번들되지 않음"으로 달성된 것입니다. 빌려 쓸 만한 철학은 이렇습니다. 무거운 엔진의 기본 위치는 "혹시 몰라 패키지에 포함"이 아니라 "사용자가 처음 클릭할 때 다운로드"여야 합니다.

함께 가는 관행이 있습니다. 모든 외부 리소스 URL은 런타임에 설정할 수 있습니다. `VITE_PYODIDE_INDEX_URL`, `VITE_DUCKDB_SPATIAL_EXTENSION_PATH`, `VITE_CESIUM_TOKEN` 은 모두 런타임에 내부 미러로 향하게 할 수 있으며, 재빌드가 필요 없습니다.

인트라넷/오프라인 납품 시나리오에서 이는 생명을 구하는 설계입니다. 번들에 하드코딩된 CDN URL은 현장에서 네트워크가 끊겼을 때 긴급 재빌드를 의미합니다.

잇기 쉬운 세부 사항이 하나 더 있습니다. `packages/processing/src/ort.ts` 는 `onnxruntime-web`의 WASM 산출물이 CDN에서 로드되며, **CDN의 버전이 npm에 고정된 버전과 정확히 일치해야 한다고** 적어 두었습니다. 그렇지 않으면 런타임 크래시입니다. **"JS 글루는 패키지에, WASM 산출물은 CDN에"** 패턴을 따르는 모든 라이브러리에서 이것이 함정입니다. 해결책은 버전을 양쪽이 함께 사용하는 단일 상수로 인코딩하는 것입니다.

결론: CDN URL, WASM 버전, 미러 경로 — 이 셋 중 어느 하나라도 하드코딩하면, 어느 금요일 저녁의 긴급 패키징 작업이 기다리고 있습니다.

### 3.6 솔직한 부정적 결과 하나

`docs/architecture.md`의 마지막 절은 말 그대로 "Performance: map rendering on Linux (WebKitGTK)"라는 제목입니다. 세부 내용은 전부 읽어 볼 가치가 있습니다.

- 어떤 줌 레벨에서든 빈 지도: 안정적인 60 FPS
- 타일 레이어가 하나라도 있으면(벡터 또는 래스터 XYZ): **타일 로딩 중 FPS가 한 자릿수로 떨어지고**, 로딩이 멈추면 즉시 60으로 복귀
- 근본 원인: WebKitGTK가 새 타일마다 메인 스레드에서 동기 WebGL 호출로 GPU 업로드(래스터는 텍스처, 벡터는 정점 버퍼)를 수행하고, 이어서 페이드인 다시 그리기까지 수행
- **단일 타일 통합 렌더 사이클이 약 125 ms로 측정됨; Chromium은 몇 ms에 불과**
- **명시적으로 배제됨:** 벡터 타일 파싱과 버킷 구축은 MapLibre의 Worker에서 실행되며 병목이 아님

배제 목록은 철저하게 상세합니다. 소프트웨어 렌더링(Intel i915 GPU 사용 확인), GPU 포화(렌더링 엔진이 여전히 약 20% 유향), Tauri IPC 파일 읽기(22 MB GeoJSON 약 126 ms), `JSON.parse` (약 36 ms), KWin 컴포지터 지연, `renderWorldCopies`, 지구본 대 메르카토르 투영, `preserveDrawingBuffer`.

문서는 재현을 위한 한 줄짜리 FPS 카운터까지 제공하며, **아직 구현되지 않은** 세 가지 완화책(`maxTileCacheSize` 증가, 512px 래스터 타일, `fadeDuration: 0` — Chromium 빌드에 불이익을 주지 않도록 WebKitGTK에서만 적용하라는 메모와 함께)도 제시합니다.

두 가지 교훈:

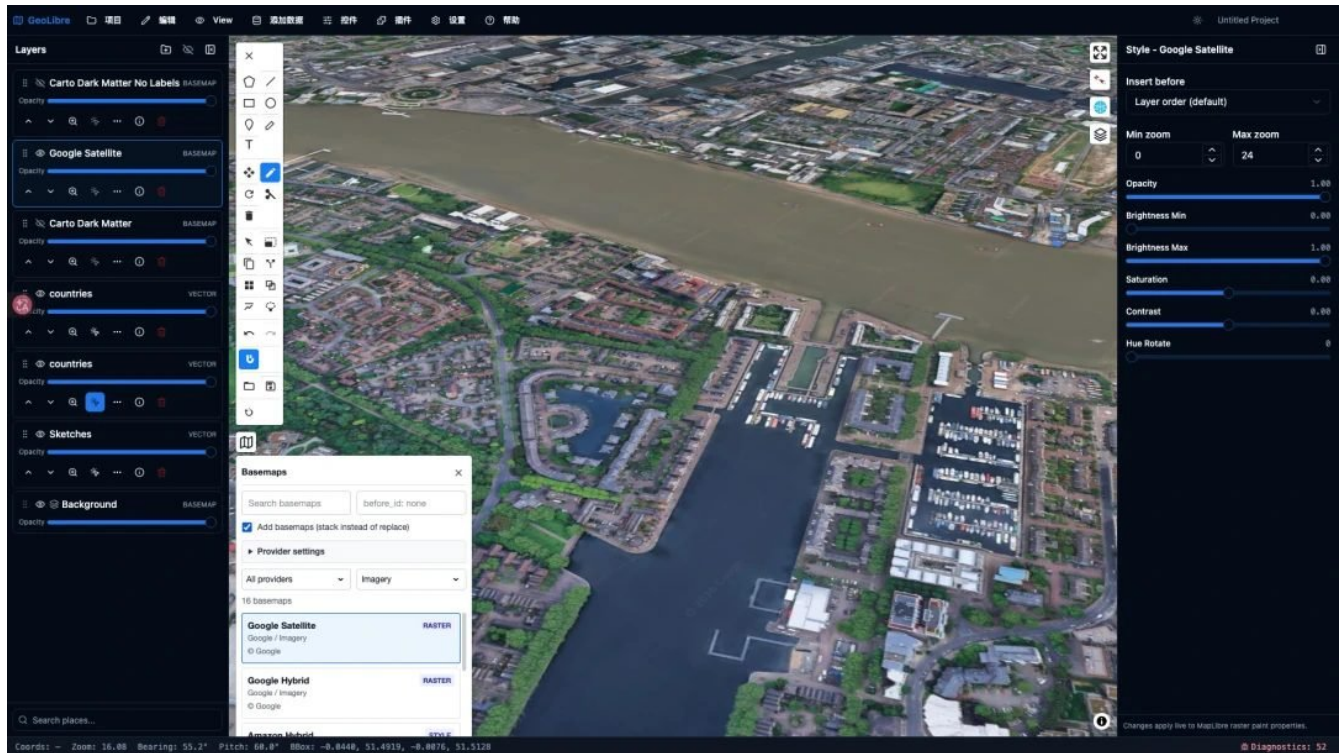
- **WebView 간 성능은 외삽되지 않습니다.** Chrome에서 측정한 60 FPS가 Electron의 오래된 Chromium, 국산 브라우저 커널, WKWebView에서도 같다고 보장하지 않습니다. **클라이언트 측 납품 요구 사항이 있는 모든 프로젝트는 대상 WebView에서 테스트해야 합니다.**
- **"무엇이 배제되었는지"와 함께 조사 과정을 문서화하는 것이 결론만 적는 것보다 더 가치 있습니다.** 그 8개 항목의 배제 목록은 다음 사람의 이틀을 아껴 줄 수 있습니다.

**편집자 주:** 이런 부정적 결과를 재현 단계와 미구현 TODO까지 포함해 아키텍처 문서에 기꺼이 적어 두려는 자세는 이 프로젝트의 엔지니어링 성숙도를 잘 보여 줍니다.

## 4. 상태 관리: 바로 재사용할 수 있는 세 가지 패턴

이 절은 순수한 웹 엔지니어링으로 GIS 고유의 내용은 적지만, 이식 비용이 가장 낮습니다.

@geolibre/core 의 의존성은 `zustand`, `zundo`, `uuid`, `@maplibre/maplibre-gl-style-spec` 네 가지뿐입니다. `maplibre-gl` 자체에는 의존하지 않는다는 점에 주목하세요(style-spec 패키지는 스타일 표현식을 평가하는 데만 사용됩니다). 따라서 상태 계층과 렌더링 계층이 진정으로 분리되어 있습니다.



**패턴 1: 단일 진실 공급원으로서의 상수 배열.** `packages/core/src/types.ts:63` 에 있는 20개의 레이어 타입:

```
export const LAYER_TYPES = [
  "geojson", "raster", "wms", "wmts", "xyz", "vector-tiles", "arcgis",
  "pmtiles", "mbtiles", "zarr", "lidar", "gaussian-splat", "3d-tiles",
  "cog", "flatgeobuf", "geoparquet", "duckdb-query", "deckgl-viz",
  "video", "image",
] as const;
export type LayerType = (typeof LAYER_TYPES)[number];
```

주석은 그 근거를 설명합니다. "신뢰할 수 없는 입력(가져온 **Layer Library** 번들, 손으로 편집한 프로젝트)을 검증할 수 있도록 런타임 목록으로" 두었으며, `LayerType` 은 배열에서 파생되어 "둘이 어긋날 수 없게" 했습니다.

**정리:** 단일 정의가 런타임 검증 데이터와 컴파일 타임 타입을 동시에 제공하면서, 둘이 서로 어긋나는 것을 막습니다. "유리한 열거형 + 외부 입력 검증 필요" 시나리오가 있는 모든 TypeScript 프로젝트는 이 패턴을 따라야 합니다.

**패턴 2: 상태는 평면 레코드 + 최소한의 뷰 상태.** `MapViewState` 는 다섯 개의 필드만 가집니다.

```
export interface MapViewState {
  center: [number, number];
  zoom: number; bearing: number; pitch: number;
  bbox?: [number, number, number, number];
}
```

이 미니멀리즘은 의도적입니다. 상태가 작을수록 프로젝트 파일로 직렬화하는 데 대한 확신이 커지고, 두 번째 소비자(두 번째 지도 패널, 두 번째 렌더러, 내보내기 도구)를 연결하는 비용이 낮아집니다.

**패턴 3: SYNC: 로 표시된 언어 간 경계.** 이는 소스를 읽으며 발견한 가장 놀라운 점이었습니다.

17개의 벡터 파일 확장자는 TypeScript에서는

`VECTOR_FILE_DIALOG_EXTENSIONS (lib/file-io/paths.ts:39)`로, Rust에서는

`RESTORABLE_VECTOR_EXTENSIONS: [&str; 17] (src-tauri/src/lib.rs:800)`로 존재합니다. **상수를 언어 간에 공유할 수 없기 때문에 두 개의 복사본이 있습니다.** 그들의 해결책은 양쪽에 주석을 다는 것입니다. `lib/file-io/paths.ts:35-38`에서 그대로 인용합니다.

SYNC: src-tauri/src/lib.rs의 `RESTORABLE_VECTOR_EXTENSIONS`는 같은 확장자를 나열해야 합니다. 그렇지 않으면 여기에 추가된 포맷이 프로젝트를 다시 열 때마다 Rust 복원 가드에 의해 거부됩니다(**이 PR이 수정하는 버그**). "SYNC:"를 grep하여 짝이 되는 목록을 찾으세요.

괄호 안의 "이 PR이 수정하는 버그"에 주목하세요. 이는 해당 주석을 추가한 풀 리퀘스트를 가리키므로 미래의 독자에게는 큰 의미가 없겠지만, 그것이 기록하는 실패는 실제입니다. TypeScript에 새 포맷이 추가되었는데 Rust 측 가드가 업데이트되지 않아, 프로젝트를 다시 열 때마다 거부되었습니다.

**정리:** 하나의 소스로 통합할 수 있다면 그렇게 하세요. 그럴 수 없다면(언어 간, 프로세스 간, 저장소 간), 통일된 grep 가능한 마커로 둘을 묶어 두고, 어긋나면 정확히 무슨 일이 일어나는지 주석에 문서화하세요. 이는 "모두 동기화를 잊지 말아 주세요"보다 훨씬 효과적입니다.

## 5. 오프라인 기능: Workbox 3단계 캐싱 전략

이 절은 인트라넷, 오프라인, 정부 기관 배포 시나리오에 특히 관련이 있어 따로 다룹니다. `docs/architecture.md`의 "Offline support (PWA)" 절에 포괄적으로 문서화되어 있습니다.

웹 빌드는 `vite-plugin-pwa` + Workbox를 사용하는 설치 가능한 PWA입니다. 캐싱은 **의도적으로 세 단계로 나뉘어** 있습니다.

| 단계              | 전략    | 내용                           | 왜 이렇게 나누는가                                                             |
|-----------------|-------|------------------------------|------------------------------------------------------------------------|
| <b>Precache</b> | 사전 캐시 | 지도 시작에 필수적인 HTML + JS/CSS 청크 | <b>첫 방문 이후 셀은 네트워크 없이 동작</b> ; 무거운 청크는 대규모 첫 화면 다운로드를 피하기 위해 <b>제외</b> |

| 단계           | 전략                                     | 내용                                                                    | 왜 이렇게 나누는가                                                                                               |
|--------------|----------------------------------------|-----------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------|
| 동일 출처 런타임 캐시 | CacheFirst                             | /assets/ 아래의 콘텐츠 해시 산출물: MapLibre, DuckDB-WASM과 그 spatial 확장, 플러그인 청크 | 콘텐츠 해시 파일명이 CacheFirst를 안전하게 만들 — 재배포하면 새 URL이 생성되므로 오래된 항목이 새것으로 제공되지 않음                                |
| CDN 엔진 캐시    | CacheFirst(별도 규칙 geolibre-cdn-engines) | jsDelivr의 Pyodide, PGLite/PostGIS, CereusDB, gdal3.js                 | URL에 정확한 버전 번호가 포함되어 마찬가지로 오래된 버전 제공을 방지; jsDelivr의 CORS 헤더 덕분에 이들은 불투명하지 않고 제대로 검증 및 제거할 수 있는 200 응답이 됨 |

따라서 이 CDN 엔진들에 대한 정확한 설명은 이렇습니다. 웹 PWA에서는 첫 번째 성공적인 가져오기에 네트워크가 필요하며(CacheFirst는 일치하는 응답이 실제로 저장된 뒤에야 캐시에서 제공합니다), 그 이후부터는 오프라인에서 사용할 수 있습니다. 데스크톱 빌드는 Service Worker를 전혀 설치하지 않으므로 이 내용이 적용되지 않습니다. 아래 항목을 참조하세요.

왜 그냥 모든 것을 번들하지 않을까요? 다음 세부 사항이 이를 명확히 해 줍니다.

특히 실용적인 세부 사항 몇 가지:

- "첫 사용 시 네트워크 필요"를 없애려면 `GEOLIBRE_PGLITE_CDN=0` 과 `GEOLIBRE_CEREUS_CDN=0` 을 설정하여 `/assets/` 에 번들할 수 있습니다. 그 대가로 PGLite만으로도 Tauri 바이너리에 약 22 MB가 다시 추가됩니다
- Pyodide와 gdal3.js에는 그런 스위치가 없습니다. Pyodide는 항상 CDN에서 로드됩니다 (`VITE_PYODIDE_INDEX_URL` 로 미러를 변경할 수 있지만, 그 미러는 두 CacheFirst 규칙의 범위에 들지 않습니다. `/assets/` 아래에 두지 않는 한 일반 HTTP 캐싱으로 폴백합니다). gdal3.js는 절대 번들링되지 않습니다. `GEOLIBRE_GDAL_CDN=0` 은 해당 내보내기 기능을 비활성화하는 것이지 번들하는 것이 아닙니다
- GeoLibre가 제어하는 모든 외부 CDN 참조를 제거하려면(신뢰할 수 없는 CDN을 전혀 참조할 수 없는 배포의 경우) `GEOLIBRE_NO_EXTERNAL_CDN=1` 을 설정하세요. 이는 모든 `*_CDN=0` 플래그를 함축하며, 추가로 소스 코드에 CDN URL을 포함하는 기능(스토리맵 HTML 내보내기, 내장 탐지 모델, ONNX WASM, 3D Tiles 디코더 경로, Pyodide 기본 URL)을 비활성화합니다. 서드파티 npm 패키지에는 GeoLibre가 제어할 수 없는 내부 CDN URL 문자열이 여전히 포함되어 있을 수 있습니다
- 데스크톱 빌드는 Service Worker를 설치하지 않으므로(Tauri 자체 리소스는 이미 오프라인입니다) 데스크톱 설치할 때마다 CDN 엔진을 다시 가져옵니다
- 베이스맵은 CORS 친화적인 기본 소스(OpenFreeMap, CARTO)만 캐시합니다. 다른 원격 타일과 WMS/WFS는 설계상 오프라인에서 사용할 수 없습니다
- 새 배포는 `registerType: "autoUpdate"` + `skipWaiting` 을 사용하지만, Workbox의 기본 "활성화 시 강제 새로고침"은 의도적으로 억제합니다. `/demo/` 같은 상대 베이스 하위 경로에서 잘못 트리거되어 사용자가 진행 중인 지도 상태를 날려 버리기 때문입니다. 페이지 복구는 `installStaleChunkReload` 가 담당하며, 고아가 된 지연 청크가 404를 반환할 때만 새로고침하고 쿨다운 보호를 갖추고 있습니다

### 가장 가치 있는 세부 사항

이 마지막 세부 사항은 이 절 전체에서 가장 가치 있는 것으로, "실제 프로덕션 문제를 겪은 뒤에야 작성되는" 종류의 코드입니다. 자동 업데이트로 사용자가 저장하지 않은 작업을 잃게 만드는 것은 가장 흔하고 가장 치명적인 PWA 실수입니다.

## 핵심 정리

- 1. 캐싱을 "셀 / 동일 출처 무거운 리소스 / CDN 엔진"의 세 단계로 나누세요. 모든 것에 하나의 규칙을 쓰지 마세요.
- 2. 콘텐츠 해시나 버전 번호는 CacheFirst가 안전하기 위한 전제 조건입니다. 그것이 없으면 CacheFirst는 "영원히 옛 버전 제공"입니다.
- 3. "오프라인에서 확실히 사용할 수 없는 것"(원격 서비스, 허용 목록에 없는 베이스맵)을 명시적으로 문서화하세요. 사용자가 추측하게 하지 마세요.
- 4. 자동 업데이트가 페이지를 강제로 새로고침해서는 안 됩니다. 고아 청크가 감지될 때만, 쿨다운과 함께 새로고침하세요.

## 6. 클라우드 네이티브 포맷: 대체되는 것은 프론트엔드 라이브러리가 아니라 배포 아키텍처 전체

앞의 다섯 절은 코드를 다뤘습니다. 이 절은 **아키텍처와 비용**에 영향을 미치는 것을 다룹니다.

전통적인 파이프라인: 데이터 → 데이터베이스에 적재 → GeoServer / ArcGIS Server로 발행 → 타일 생성 → **프론트엔드가 서비스 요청**. 서버, 발행 워크플로, 타일 캐시를 유지해야 하고, 그 모든 것에 대한 모니터링, 백업, 확장까지 필요합니다.

클라우드 네이티브 파이프라인: 데이터 → COG / GeoParquet / PMTiles / FlatGeobuf로 변환 → **Range 요청에 응답하는 아무 정적 저장소에 올리기** → **프론트엔드가 필요한 바이트 범위를 주문형으로 가져오기**. 서버 계층이 완전히 사라지고, 남는 것은 객체 저장소 트래픽 비용뿐입니다.

**핵심 문구는 HTTP Range입니다.** 이 포맷들이 공유하는 것은 "더 나은 압축"이 아니라, **각 파일이 자체 레이아웃 메타데이터(헤더 또는 푸터, 타일 디렉터리, R-tree 인덱스, 오버뷰, 로우 그룹 통계)를 가지고 있다는 점**입니다. 클라이언트는 그 메타데이터 몇 KB를 읽고, 어떤 바이트 범위가 필요한지 계산한 뒤, **Range** 요청을 발행해 가져옵니다. 정적 저장소로 충분하며 "서비스"는 필요 없습니다.

### 6.1 무엇이 좋은가: 다섯 가지 구체적인 이점

**이점 1: 다운로드 양이 "파일이 얼마나 큰가"가 아니라 "무엇을 봐야 하는가"로 수렴합니다.**

이것이 가장 근본적인 변화입니다. 전통적인 포맷(GeoJSON, Shapefile, 스트라이프 GeoTIFF)은 **유용한 부분 범위 접근을 제공하지 않습니다**. 인덱스가 있더라도 **사이드카 파일에 있으므로 클라이언트는 여전히 모든 데이터를 가져와야 합니다**. 클라우드 네이티브 포맷은 그 인덱스를 데이터 파일 자체 안에 두어, 세 가지 차원에서 가지치기를 가능하게 합니다.

| 가지치기 차원 | 메커니즘                                    | 효과                                           |
|---------|-----------------------------------------|----------------------------------------------|
| 공간      | FlatGeobuf의 R-tree 인덱스, PMTiles 타일 레이아웃 | <b>뷰포트 안의 피쳐/타일만 가져옴</b>                     |
| 해상도     | COG 피라미드(오버뷰), PMTiles 줌 레벨             | 가장 거친 레벨에서 전국을 보며, 1억 픽셀을 디코딩하지 않음           |
| 속성 열    | GeoParquet의 열 지향 저장                     | 3개 열만 필요하면 그 3개 열의 바이트만 읽고, 다른 열은 전혀 건드리지 않음 |

세 번째 차원은 GeoParquet의 가장 과소평가된 강점입니다. **열 지향 저장 + 로우 그룹 통계(min/max)**는 `WHERE` 조건을 푸시다운할 수 있다는 뜻입니다. DuckDB는 통계만으로 로우 그룹 전체가 조건을 만족하지 않는다고 판단하고, 그 바이트를 다운로드하지 않고 건너뛸 수 있습니다.

**이점 2: 파싱 비용이 "메인 스레드를 막는 수십 밀리초"에서 거의 0으로 떨어집니다.**

텍스트 포맷의 숨은 비용은 `JSON.parse` 입니다. 3.6절의 조사 체크리스트는 두 가지 측정 수치를 제공합니다. **22 MB GeoJSON은 디스크에서 읽는 데 약 126 ms, 그다음 `JSON.parse` 에 약 36 ms가 더 걸립니다**. 모두 메인 스레드에서입니다.

바이너리 열 지향 포맷은 이 단계를 없앱니다. 바이트 레이아웃이 곧 메모리 레이아웃입니다. DuckDB는 벡터화된 열을 직접 읽습니다. 문자 단위 파싱도 없고, 수십만 개의 JS 객체를 만들지도 않습니다. **피쳐 수가 많을 때 실제로 페이지를 멈추게 하는 것은 렌더링이 아니라 파싱과 GC인 경우가 많습니다**.

**이점 3: 캐싱이 거의 공짜가 됩니다.**

클라우드 네이티브 포맷은 **정적 파일**이므로, 기존 HTTP 캐싱 파이프라인 전체를 자연스럽게 활용합니다. **CDN 엣지 캐싱, 브라우저 캐시, Service Worker, ETag / 불변 URL** — 어느 것도 직접 구현할 필요가 없습니다.

동적 타일 서비스와 비교해 보세요. 타일 캐시는 직접 만들어야 하고, 무효화 전략은 직접 작성해야 하며, 캐시 적중률은 직접 모니터링해야 합니다. **같은 불변 파일의 같은 바이트 범위에 대한 Range 요청은 CDN 계층에서 그저 평범한 캐시 적중일 뿐입니다**. 5절의 Workbox 3단계 캐싱 전략이 애초에 동작하는 이유는 바로 리소스가 정적이고 콘텐츠 해시가 붙어 있기 때문입니다.

**이점 4: 동시성 병목이 여러분의 서버에서 벗어납니다.**

타일 서비스의 용량 병목은 서버 측 CPU입니다. 동시 사용자 100명은 렌더링 비용 100벌을 의미합니다. **정적 저장소 + CDN은 완전히 다른 확장 곡선을 따릅니다**. 사용자별로 확장할 렌더링이 없으므로 추가 부하는 대부분 트래픽 비용으로 나타납니다. 그리고 엣지 캐싱이 사용자와 가장 가까운 노드에서 대다수의 요청을 가로챍니다. 사라지지 않는 것은 용량 계획입니다. 캐시 미스는 여전히 오리진에 도달하며, 객체 저장소에는 자체적인 접두사별 요청 속도 제한과 송신 요금이 있습니다.

이는 정부 대시보드와 공공 서비스 포털에 특히 중요합니다. "대부분의 시간은 유휴 상태지만 회의 중에는 모두가 동시에 본다"는 트래픽 패턴 말입니다. 이런 형태를 타일 서비스로 처리하려면 장기적으로 피크 용량에 과다 지불하거나 피크 때 다운되거나 둘 중 하나입니다.

**이점 5: 체인에서 고리 하나가 통째로 사라지고, 운영 비용도 그와 함께 무너집니다.**

**발행 워크플로도, 서비스 프로세스도, 데이터베이스 커넥션 풀도, 패치할 미들웨어도 없습니다.** 데이터 업데이트는 그저 파일을 덮어쓰는 것입니다. GeoLibre의 "브라우저에서 원격 GeoParquet / COG / PMTiles 직접 읽기" 기능 전체가 이 경로가 동작한다는 증거입니다. 연결되는 모든 온라인 카탈로그(STAC, Source Cooperative, Overture Maps, Planetary Computer)가 이 모델을 따릅니다.

**그러나 트레이드오프는 실재합니다. 다음 네 가지는 진짜입니다.**

- **작은 데이터는 실제로 더 느립니다.** 헤더 읽기, 인덱스 읽기, 그다음 데이터 가져오기 — 네트워크 왕복이 여러 번입니다. 수백 KB의 GeoJSON은 요청 한 번이면 끝입니다. 이를 클라우드 네이티브 포맷으로 감싸는 것은 역효과입니다. **임계값은 대략 "단일 요청으로는 너무 크다"입니다.**
- **Range 요청 수가 크게 늘어납니다.** HTTP/1.1도 Range 를 잘 처리하지만, 이 정도 요청량에서는 HTTP/2 또는 HTTP/3을 강력히 권장합니다. 그렇지 않으면 연결 오버헤드가 이점을 갉아먹습니다.
- **서버가 Range 와 CORS를 지원해야 합니다**(Access-Control-Expose-Headers 도 포함). 이는 예상보다 자주 발목을 잡습니다. 다음 하위 절에서 자세히 다룹니다.
- **연산이 서버에서 클라이언트로 이동합니다.** 서버가 하던 디코딩, 필터링, 스티칭이 이제 사용자의 브라우저에서 실행되며, 약 4 GiB 메모리와 단일 스레드 제약을 받습니다(2.5절 참조). **"더 빨라진" 것이 아니라 "연산이 일어나는 곳이 바뀐" 것이며, 새 위치가 더 비용 효율적으로 연산할 때만 이득입니다.**

따라서 클라우드 네이티브 포맷에는 잘 정의된 최적 지점이 있습니다. 크고, 정적이며, 다중 사용자 읽기이고, 뷰포트로 제한된 접근 — 이 넷이 모두 충족될 때 최대 이득을 얻습니다. 반대로 고빈도 쓰기의 동적 운영 데이터는 이 모델에 맞지 않으며, 그래서 6.7절의 체크리스트가 "PostGIS 경로는 건드리지 마세요"라고 명시하는 것입니다.

## 6.2 GeoLibre는 이 포맷들을 어떻게 읽는가

| 포맷         | 읽기 경로                                                                               | 핵심 포인트                                      |
|------------|-------------------------------------------------------------------------------------|---------------------------------------------|
| PMTiles    | MapLibre 커스텀 프로토콜( <code>pmtiles</code> 패키지)                                        | 파일 하나가 타일 서비스 전체를 대체; 프론트엔드는 프로토콜 하나만 추가 등록 |
| COG        | <code>cog-tiler-wasm</code> (기본값) / <code>deck.gl GPU</code> / <code>TiTiler</code> | 전환 가능한 세 가지 엔진; 앞의 둘은 완전히 클라이언트 측           |
| GeoParquet | DuckDB-WASM <code>read_parquet</code>                                               | 선택적 "제자리 스트리밍 쿼리" — 데이터셋 전체를 메모리에 복사할 필요 없음 |
| FlatGeobuf | DuckDB Spatial <code>ST_Read</code>                                                 | 내장 공간 인덱스, 범위 읽기에 자연스럽게 적합                  |

| 포맷                     | 읽기 경로                               | 핵심 포인트                          |
|------------------------|-------------------------------------|---------------------------------|
| MosaicJSON / STAC 카탈로그 | Raster 컨트롤이 카탈로그를 읽고 읽기 시점에 장면을 스티칭 | 카탈로그 자체에는 데이터가 없고 에셋 URL 목록만 있음 |
| 클라우드 최적화 NetCDF/HDF5   | kerchunk 참조 매니페스트 → Zarr 렌더링 파이프라인  | 6.5 참조                          |

### 6.3 진짜 장벽은 포맷 변환이 아니라 CORS

포맷 변환은 일회성 작업입니다. CORS는 매일같이 부딪히는 벽입니다. Source Cooperative에 대한 소스 코드의 설명은 정확합니다. 데이터 엔드포인트 `data.source.coop` 은 `Access-Control-Allow-Origin: *` 을 보내고 바이트 범위 요청을 지원하므로, **PMTiles 프로토콜, DuckDB-WASM, COG 리더가 모두 프록시 없이 직접 연결할 수 있습니다.** 그러나 그 메타데이터 API는 CORS 헤더를 전혀 보내지 않습니다. 그렇다면 브라우저는 이를 어떻게 처리할까요? 데스크톱 버전은 Tauri의 네이티브 HTTP를 통합합니다(서버 대 서버이므로 CORS 같은 것은 없습니다). 웹 버전은 CORS 헤더를 추가해 JSON을 다시 제공하는 자체 구축 Cloudflare Worker를 통합합니다.

"데이터는 직접 연결 가능, 메타데이터는 프록시 필요"라는 판단은 어떤 클라우드 네이티브 데이터 소스를 통합하든 가장 먼저 내려야 할 결정입니다.

같은 Worker에는 행성 베이스맵을 왜 프록시해야 하는지에 대한 더욱 교훈적인 주석이 있습니다.

MapLibre는 `fetch()` 를 사용해 래스터 타일을 가져오는데, 이는 CORS 검사를 거치므로 지도가 검은 화면으로 렌더링됩니다. 한편 `openplanetarymap.org` 자체는 잘 동작하는데, Leaflet이 `<img>` 태그로 타일을 로드하기 때문이며, `<img>` 는 CORS 검사를 수행하지 않습니다.

#### 치명적인 함정

같은 타일 URL이 Leaflet에서는 표시되지만 MapLibre에서는 표시되지 않습니다. 근본 원인은 코드도 타일 자체도 아닐 수 있습니다. 두 라이브러리가 이미지를 가져오는 방식이 다르기 때문입니다.

이 파이프라인에서 빌려 쓸 만한 세 가지 관행: 서버 측에서 데이터를 가져와 CORS 헤더를 추가(브라우저가 벽에 직접 부딪히지 않게 함), 엣지에서 결과 캐시(반복 요청이 오리진으로 돌아가지 않음), 엄격한 허용 목록, 절대 열린 프록시 금지(소스 코드의 표현: "keyed to a tight allowlist so it is never an open proxy"). 세 번째는 보안 기준선입니다. 임의의 URL을 전달할 수 있는 공개 프록시는 결국 악용되며, 그 결과는 여러분의 몫입니다.

기억할 만한 세부 사항이 하나 더 있습니다. Source Cooperative의 알 수 없는 API 경로는 404를 반환하지 않고, 포괄 페이지 라우트로 넘어가 상태 코드 200과 함께 HTML을 반환합니다. 그래서 소스 코드는 읽을 때마다 파싱된 구조를 검증합니다. `response.ok` 는 네트워크 계층에서 오류가 없었다는 뜻일 뿐, 요청한 것을 받았다는 뜻이 아닙니다.

## 6.4 "COG로 변환"은 확장자를 .tif 로 바꾸는 것이 아니다

COG를 Range 요청으로 읽을 수 있는 능력은 **파일 내부의 타일링과 피라미드 오버뷰**에 달려 있습니다. 스트라이프 GeoTIFF도 기술적으로는 범위 읽기가 가능합니다. 스트립 오프셋과 바이트 수가 헤더에 있으니까요. 하지만 스트립은 이미지 전체 너비에 걸쳐 있으므로 작은 지도 범위를 가져오는 데 필요한 것보다 훨씬 많은 바이트를 끌어오고, 오버뷰가 없으니 좀 아웃할 거친 레벨도 없습니다. .tif 라는 이름을 붙여도 이 중 어느 것도 바뀌지 않으며, GeoLibre의 클라이언트 측 리더는 내부 타일을 아예 요구합니다.

GeoLibre의 처리 방식은 교훈적입니다. 패널은 **먼저 Range 요청으로 파일 헤더를 읽어** 내부 타일이 존재하는지 판단합니다. 없으면 로드하기 전에 브라우저에서 파일을 COG로 변환하겠다고 제안합니다(geolibre-wasm의 `CogBuilder`, `packages/processing/src/cog-convert.ts`). **사용자가 필연적으로 느린 로딩을 기다리게 하는 대신, 몇 KB의 헤더로 확실한 답을 얻습니다.**

세 가지 래스터 엔진 간의 트레이드오프도 소스에 문서화되어 있습니다. `cog-tiler-wasm` (기본값, 브라우저 WASM, **내장 색상 램판만 사용할 수 있다는 대가** — 커스텀 분류는 손실됨), `maplibre-gl-raster` (GPU, 전체 심볼로지), `titiler` (서버 측). 기본값은 **기능 완전성이 아니라 안정성을 기준으로 선택되었습니다.**

## 6.5 데이터가 클라우드 네이티브가 아니라면? 인덱스를 외부화하라

NetCDF / HDF5는 과학 데이터의 사실상 표준이지만 HTTP를 위해 설계되지 않았습니다. GeoLibre의 접근 방식은 **kerchunk 참조 매니페스트**입니다. 매니페스트는 Zarr 키를 `[url, offset, length]` (원본 파일 내의 바이트 범위)에 매핑하고, **최소한의 zarrita Readable (get(key) 만 있으면 됨)**을 구현한 뒤, 이를 기존 Zarr 렌더링 파이프라인에 그대로 공급합니다. 소스 주석의 "with no rewrite"라는 문구가 핵심입니다. **렌더링 코드는 단 한 줄도 바뀌지 않았습니다.**

이 아이디어는 일반화됩니다. 데이터도 바꾸지 말고 렌더러도 바꾸지 말고, 그 사이에 "키 → 바이트 범위" 매핑 계층만 끼워 넣으세요. 스트리밍 읽기에 부적합한 과거의 포맷도 오프셋을 계산할 수만 있다면 주문형으로 읽을 수 있게 됩니다.

## 6.6 카탈로그 브라우저는 딱 한 가지 일만 해야 한다: URL 생산

GeoLibre는 STAC, Source Cooperative, Overture Maps, Planetary Computer, Hugging Face 등 수많은 온라인 데이터 소스를 통합하면서도 하나의 원칙으로 관리 가능한 상태를 유지합니다. **레이어를 추가할 때는 각 포맷을 이미 이해하는 컨트롤에 의도적으로 위임합니다.** PMTiles는 `addPMTilesLayerFromUrl` 로, GeoParquet은 `addVectorLayerFromUrl` 로, COG는 `app.addCogLayer` 로. 그 결과 카탈로그에서 클릭해 들어온 데이터와 Add Data(데이터 추가)로 수동 추가한 데이터는 **완전히 동등합니다.** 같은 레이어 패널, 같은 스타일링 기능, 같은 프로젝트 영속화.

데이터 소스 통합 하나 = "URL을 찾는 방법" 어댑터 하나. 읽기와 렌더링은 항상 단일 구현입니다. 이것이 통제를 잃지 않고 N개의 데이터 소스를 통합하는 유일한 방법입니다.

## 6.7 도입 순서 및 핵심 정리

도입 순서(투자가 적은 것부터 많은 것 순):

1. 외부에 공개되는 정적 데이터를 PMTiles로 변환 — 타일 서비스 하나를 통째로 제거
2. 래스터 산출물을 진짜 COG로 변환(내부 타일과 피라미드 확인) — 프론트엔드 직접 읽기 또는 titiler와 결합
3. 큰 벡터 테이블을 GeoParquet으로 변환 — DuckDB로 직접 쿼리
4. 변환할 수 없는 과거 포맷은 외부 인덱스(kerchunk 접근 방식)를 고려
5. 동적 운영 데이터는 PostGIS에 그대로 둡니다. 이 경로는 건드리지 마세요
6. 1. 본질은 "파일 내 인덱스 + HTTP Range"이므로, 대체되는 것은 서버 측이지 프론트엔드 라이브러리가 아닙니다.
7. 2. 먼저 "데이터는 직접 연결 가능"과 "메타데이터는 프록시 필요"를 구분하세요. 프록시는 허용 목록 + 오프지 캐시여야 하며, 절대 열린 프록시여서는 안 됩니다.
8. 3. 외부 API를 상대로 프로그래밍할 때는 파싱된 구조를 검증하세요. `response.ok` 를 믿지 마세요.
9. 4. 다중 소스 통합은 단일한 포맷 읽기 구현 세트로 수렴해야 하며, 카탈로그 패널은 URL 생산만 책임집니다.

1번 항목의 ROI가 가장 높습니다. PMTiles 파일 하나 + 정적 서버가 타일 발행 워크플로 전체를 대체합니다. 많은 프로젝트가 짧은 시간 안에 이 전환을 완료할 수 있습니다.

## 맺으며: 투자 규모 순으로 정리한 도입 체크리스트

### 하루 안에 검증 가능

1. DuckDB-WASM Spatial을 한 번 시도해 보세요. 원격 GeoParquet을 직접 읽으며 "백엔드 불필요"를 경험하세요
2. 프로젝트의 무거운 엔진, 특히 3D 엔진을 주문형 지연 로딩 + 별도 청크로 전환하세요. 첫 화면 경로에서 떼어내세요
3. 모든 CDN / 리소스 URL을 런타임에 설정 가능하게 만드세요

### 이번 이터레이션에 달성 가능

1. 대용량 데이터 레이어에 명확한 임계값을 정하고, 그 선을 넘으면 클라이언트 측 타일링으로 전환하세요 (`@maplibre/geojson-vt` + `@maplibre/vt-pbf`)
2. 네트워크가 아닌 긴 루프를 포함한 모든 비동기 파이프라인에 `AbortController` 를 연결하세요
3. 실행 취소 스택에 메모리 예산이 있는지, 최신 스냅샷이 항상 유지되는지 확인하고, 가상 목록 정렬이 전체 데이터 모델에 대해 동작하는지 확인하세요
4. 열거형을 `as const` 배열로 다시 작성하여 런타임 검증과 타입 정의가 단일 소스를 공유하게 하세요

5. 언어 간 / 프로세스 간 중복 상수를 통일된 SYNC: 마커로 묶어 두세요

## 프로젝트로서 평가할 가치가 있음

1. DuckDB-WASM Spatial을 "포맷 리더"에서 "쿼리 계층"으로 업그레이드하세요. 레이어가 동시에 SQL 테이블이 되게 하여 사용자가 두 레이어에 걸쳐 JOIN 과 ST\_Intersects 를 작성할 수 있게 하세요
2. 연산 계층을 프레임워크로부터 분리하여 브라우저와 서버가 같은 코드를 실행하게 하세요. 결과를 다시는 대조할 필요가 없습니다
3. 정적 데이터 파이프라인을 PMTiles부터 시작해 클라우드 네이티브 포맷으로 옮기세요. 저장소가 Range 와 CORS를 지원하는지 확인하고, 어떤 메타데이터가 프록시를 거쳐야 하는지 명확히 하세요
4. 웹 애플리케이션을 만들고 있다면 Service Worker 캐싱을 3단계 전략으로 다시 구축하세요

이 체크리스트의 어떤 항목도 GeoLibre 코드를 단 한 줄도 가져올 필요가 없습니다. GeoLibre의 가장 큰 기여는 또 하나의 지도 애플리케이션이 아니라, 실행하고 설치하고 소스를 읽을 수 있는 완전한 프로젝트로써 브라우저에서 실행되는 공간 데이터베이스만으로 예전에 서버가 필요했던 작업의 절반을 대체하기에 충분하다는 것을 증명했다는 점입니다.